Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Робототехника и комплексная автоматизация»

КАФЕДРА «Компьютерные системы автоматизации производства»

РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

К ВЫПУСКНОЙ КВАЛИФИКАЦИОННОЙ РАБОТЕ

НА ТЕМУ:

«Разработка серверной части системы управления гибкой производственной системой»

Студент РК9-83Б _____ (Подпись, дата) Зубков М.С.

Руководитель ВКР _____ (Подпись, дата) Мурашов М.В.

Консультант _____ (Подпись, дата) Буханов С.А.

Нормоконтролер _____ (Подпись, дата) Шеин А.П.

2024 г.

РЕФЕРАТ

Расчётно-пояснительная записка содержит 172 страницы, 6 разделов, 12 рисунков, 2 таблицы, 35 листингов кода, 29 приложений, 15 источников.

Целью ВКР является разработка серверной части системы управления гибкой производственной системой. В данной работе проведен анализ существующей системы, выявлены точки роста. Проект содержит описание процесса выбора сетевого протокола, инфраструктуры, конкретной облачной платформы, архитектуры приложения. В ходе работы описывается разработка системы управления: разработка алгоритма коммуникации оператора гибкой производственной линии с ее модулями, алгоритма внутреннего взаимодействия модулей, организация кодовой базы согласно рекомендуемым практикам, апробация итогового решения. Заключение посвящено основным выводам по проделанной работе.

Ключевые слова: Интернет Вещей, протокол MQTT, клиент, брокер, облачные технологии, Yandex.Cloud, централизованная система, многопоточное программирование, Golang, чистая архитектура, серверное приложение.

СОДЕРЖАНИЕ

РЕФЕРАТ 1

СОДЕРЖАНИЕ 3

ОПРЕДЕЛЕНИЯ 6

ВВЕДЕНИЕ 7

1. Предпроектное обследование 10

1.1. Анализ существующей системы 10

2. Концептуальное проектирование 12

2.1. Постановка задачи 12

2.2. Выбор сетевого протокола коммуникации 13

2.3. Выбор платформы 17

2.4. Обзор платформы Yandex Cloud и описание необходимого сервиса 18

3. Техническое задание 21

4. Структурное проектирование 23

4.1. Разработка алгоритма взаимодействия клиента ГПС с системой 23

4.2. Концепция распределенных систем 25

4.3. Концепция централизованных систем 27

4.4. Микросервисная и монолитная архитектура приложения 28

4.5. Параллельные вычисления в контексте монолитной архитектуры 31

4.6. Разработка алгоритма внутренней коммуникации элементов ГПС 33

5. Рабочее проектирование 35

5.1. Настройка Yandex IoT Core и выбор языка программирования 35

5.2. Структура приложения 39

5.3. Архитектура проекта, организация кодовой базы 41

5.4. Слой Entity 45
5.5. Слой Service 49
5.6. Слой Delivery 52
5.6.1. Инициализация MQTT клиента 53
5.6.2. Описание промежуточного слоя функций, выполняющихся перед началом основного алгоритма обработки 56
5.6.3. Описание обработчика, отправляющего метрики модулей с определенной периодичностью 61
5.6.4. Описание обработчиков сообщений топиков, на которые подписаны модули ГПС 62
5.6.5. Описание формата ответов внешнему клиенту, метода сериализации полезной нагрузки 65
6. Апробирование 68
6.1. Понятие интеграционного тестирования 68
6.2. Подготовка среды для проведения интеграционного тестирования программного кода приложения 68
6.3. Сценарий работы и результаты интеграционного тестирования 70
ЗАКЛЮЧЕНИЕ 73
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 75
ПРИЛОЖЕНИЕ А 76
ПРИЛОЖЕНИЕ Б 82
ПРИЛОЖЕНИЕ В 85
ПРИЛОЖЕНИЕ Г 87
ПРИЛОЖЕНИЕ Д 89
ПРИЛОЖЕНИЕ Е 91
ПРИЛОЖЕНИЕ Ж 92

ПРИЛОЖЕНИЕ З 94
ПРИЛОЖЕНИЕ И 98
ПРИЛОЖЕНИЕ К 100
ПРИЛОЖЕНИЕ Л 102
ПРИЛОЖЕНИЕ М 104
ПРИЛОЖЕНИЕ Н 109
ПРИЛОЖЕНИЕ О 113
ПРИЛОЖЕНИЕ П 117
ПРИЛОЖЕНИЕ Р 121
ПРИЛОЖЕНИЕ С 124
ПРИЛОЖЕНИЕ Т 125
ПРИЛОЖЕНИЕ У 128
ПРИЛОЖЕНИЕ Ф 130
ПРИЛОЖЕНИЕ Х 134
ПРИЛОЖЕНИЕ Ц 137
ПРИЛОЖЕНИЕ Ч 139
ПРИЛОЖЕНИЕ Ш 146
ПРИЛОЖЕНИЕ Щ 154
ПРИЛОЖЕНИЕ Ы 155
ПРИЛОЖЕНИЕ Э 157
ПРИЛОЖЕНИЕ Ю 163
ПРИЛОЖЕНИЕ Я 167

ОПРЕДЕЛЕНИЯ

Протокол UDP (User Datagram Protocol) — один из ключевых элементов пакета протоколов интернета. Он используется для отправки сообщений (датаграмм) в сетях, работающих по принципу коммутации пакетов. Протокол TCP (Transmission Control Protocol) — это один из основных протоколов в наборе интернет-протоколов. Он обеспечивает надежную, упорядоченную и безошибочную передачу данных между отправителем и получателем в сети. TCP используется для контроля передачи данных в интернете.

API (Application Programming Interface) — набор определений и протоколов для создания и интеграции программного обеспечения. API позволяет разным программам взаимодействовать друг с другом, обеспечивая стандартизированный способ доступа к функциональности, предоставляемой одной программой для использования в другой.

TLS (Transport Layer Security) и его предшественник SSL (Secure Sockets Layer) — это криптографические протоколы, предназначенные для обеспечения безопасности коммуникации в компьютерных сетях.

Сертификаты X.509 — это стандарт, используемый для создания цифровых сертификатов, которые устанавливают идентичность организации, сервера или отдельного лица в сети.

Фреймворк — это стандартизированный набор концепций, практик и критериев для решения определенного типа задач, который служит в качестве основы для разработки программных приложений и систем.

IP (Internet Protocol) — это основной протокол сетевого уровня в модели OSI, который используется для маршрутизации и адресации пакетов данных в сетях, работающих по технологии коммутации пакетов, включая интернет. IP-адрес предоставляет уникальный идентификатор для каждого устройства в сети, позволяя правильно маршрутизировать данные от отправителя к получателю.

ВВЕДЕНИЕ

В настоящее время инженерное дело меняется быстрыми темпами. С каждым годом появляется все больше практик, способствующих повышению эффективности, автономности и безопасности предприятий. Современные технологии позволяют порой практически полностью исключить человека из непосредственного производственного процесса, тем самым уменьшая вероятность производства непригодных изделий, возникновения разного рода недоразумений в процессе работы.

Одним из ярчайших примеров вышеуказанных технологий является концепция «Интернета вещей». Интернет вещей (IoT – The Internet Of Things) представляет собой технологическую концепцию, в основе которой лежит идея объединения физических устройств и объектов сетью, чтобы они могли обмениваться данными и взаимодействовать друг с другом без прямого человеческого вмешательства. Наладив инфраструктуру, необходимую для IoT, можно обеспечить взаимодействие в том числе и бытовых приборов в рамках локальной и глобальной сетей.

Применение технологии интернета вещей разнообразно и включает в себя множество областей, таких как умные дома, здравоохранение, промышленность, сельское хозяйство, транспорт и многие другие. Концепция Интернета вещей продолжает эволюционировать, и ее влияние ощущается в различных аспектах повседневной жизни и различных отраслях.

Термин "Интернет вещей" впервые был сформулирован Кевином Эштоном в 1999 году при представлении технологии внедрения радиочастотных меток в логистическую систему компании Procter & Gamble. В 2000-х годах концепция IoT начала привлекать все больше внимания. Компании и исследовательские организации начали активно работать над интеграцией устройств в единую сеть. Более широкое внедрение концепции "Интернет вещей" стало возможным благодаря широкому распространению разнообразных электронных устройств и носителей, таких как смартфоны, микроконтроллеры, развитию методов

обработки данных и совершенствованию беспроводных сетей. Повсеместное распространение мобильных устройств, таких как смартфоны и планшеты, сделало IoT более доступным и функциональным. Мобильные устройства стали управляющими центрами для подключенных устройств и платформами для взаимодействия с ними.

Ключевым аспектом данной работы является одно из ответвлений рассмотренной концепции – промышленный Интернет Вещей. Промышленный интернет вещей представляет собой подразделение интернета вещей, объединяющее системы компьютерных сетей с промышленными объектами, оборудованными встроенными датчиками и программным обеспечением. [4] Эти системы предназначены для сбора и обмена данными с возможностью удаленного контроля и управления в автоматизированном режиме, минимизируя вмешательство человека. Эта технологии способствует повышению эффективности в управлении производственными процессами, техническом обслуживании, мониторинге состояния оборудования и оптимизации управления элементами системы. На сегодня промышленный интернет вещей активно применяется в различных отраслях, таких как добыча полезных ископаемых, электростанции, авиация и многое другое.

Технически Интернет Вещей возможен благодаря делегированию задачи связывания устройств в единую инфраструктуру некоему удаленному серверу. Причем весьма часто в качестве сервера выступает та или иная облачная платформа. В конечном счете взаимодействие между устройствами(клиентами) становится возможным благодаря различным протоколам коммуникации.[7]

Рисунок 1— Схематичное представление концепции IoT

1. Предпроектное обследование

1.1. Анализ существующей системы

В результате анализа существующей системы была построена idef0-диаграмма (рисунок 2), демонстрирующая процессы, происходящие в ГПС. Из склада заготовка поступает на конвейер. Конвейер доставляет заготовку к токарному станку. После обработки на токарном станке заготовка доставляется к фрезерному станку и также обрабатывается. Затем обработанное изделие идентифицируется модулем распознавания, далее изделие поступает на склад, процесс обработки заканчивается.

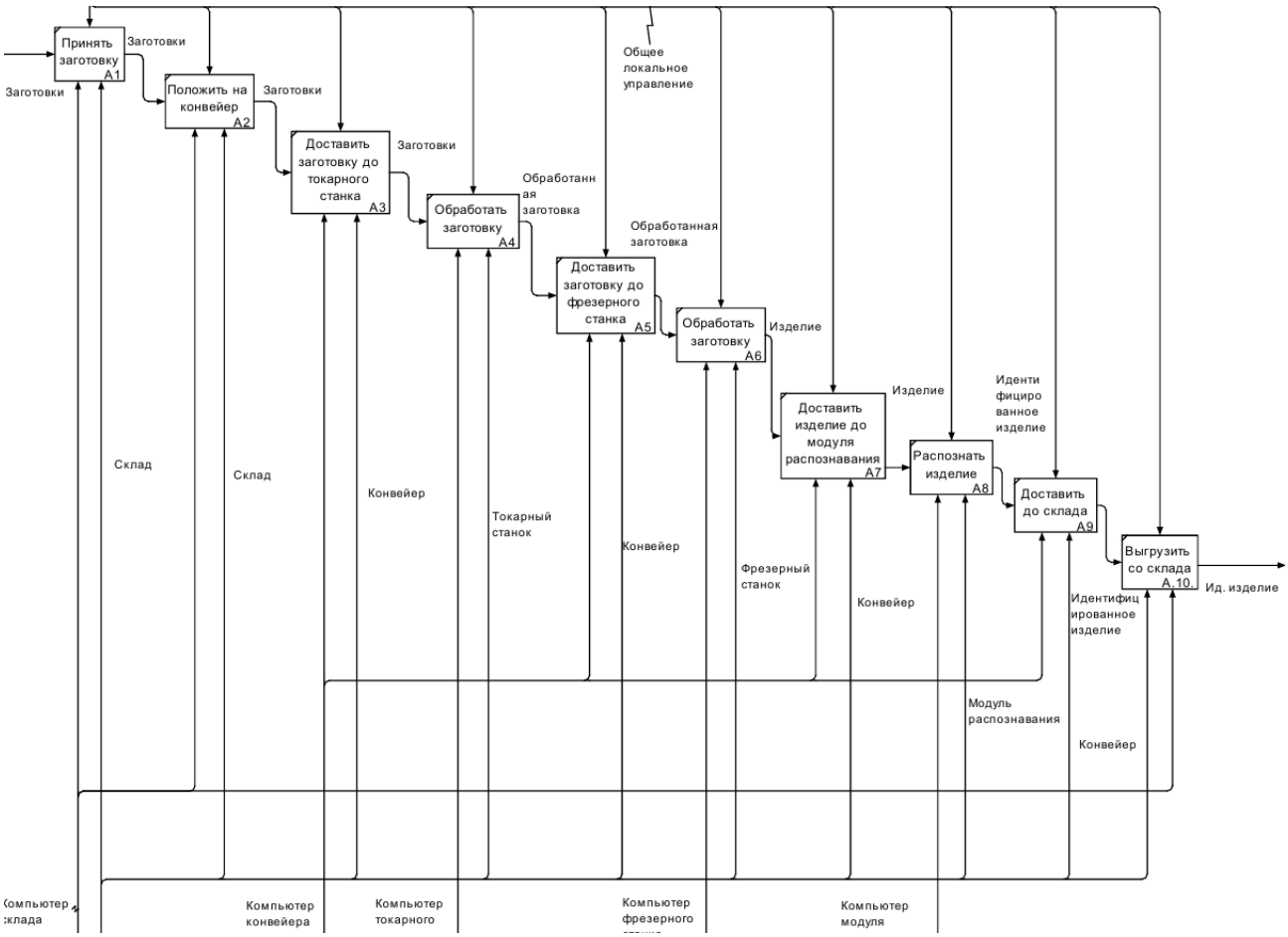

Рисунок 2 — Диаграмма процесса функционирования существующей системы

Модулями ГПС можно управлять как через единую точку доступа, так и с помощью отдельных пунктов управления(компьютер/контроллер).

«Узкие места» описанного производственного процесса: управление в ручном режиме, как следствие, необходимость постоянного присутствия человека, отсутствие удаленного управления.

2. Концептуальное проектирование

Отраженный в диаграмме производственный процесс можно улучшить путем внедрения систем автоматического удаленного управления и мониторинга, что положительно скажется на результативности комплекса и сведет человеческий фактор к минимуму. Технологии Интернета вещей и современное ПО для разработки серверных приложений позволят реализовать указанный функционал.

2.1. Постановка задачи

Цель данной ВКР: разработка серверной части системы управления гибкой производственной системой.

В ходе работы над проектом были поставлены следующие задачи:

1) анализ компонентов гибкой производственной системы, определение объектов автоматизации;
2) выбор сетевого протокола для взаимодействия устройств между собой и клиентом ГПС;
3) выбор платформы для развертывания компонентов разрабатываемой системы;
4) разработка алгоритма взаимодействия клиента ГПС с системой;
5) разработка алгоритма внутренней коммуникации элементов ГПС;
6) выбор языка программирования для разработки программного обеспечения;
7) разработка серверного ПО, работающего согласно обозначенным алгоритмам;
8) интеграционное тестирование программных модулей.

2.2. Выбор сетевого протокола коммуникации

В первую очередь, необходимо выбрать протокол(ы) для обеспечения обмена сообщениями между клиентом гибкой производственной системы, иначе говоря, оператором ГПС и самой системой, а также для поддержания взаимодействия самих устройств гибкой производственной системы между собой. Требования, предъявляемые к протоколу: асинхронная обработка данных в реальном времени, гарантия доставки сообщений, поддержка очередности сообщений, имплементация механизмов снабжения клиентов, инициализировавших новую сессию, актуальными данными, совместимость с криптографическими протоколами, обеспечивающими защищенную передачу данных между узлами в сети (предпочтителен SSL/TLS), наличие механизмов аутентификации.

Основываясь на характеристиках распространенных решений, можно выбрать наиболее подходящую технологию.

1) XMPP (Extensible Messaging and Presence Protocol) — протокол представляет собой открытый стандарт обмена сообщениями в режиме реального времени и управления присутствием в сетях. В основном используется для мгновенного обмена сообщениями. Обладает высокой масштабируемостью и поддерживает большое количество одновременных подключений. Протокол работает на основе моделей издатель-подписчик и клиент-сервер. Стандартом предусмотрены механизмы аутентификации с помощью имени пользователя и пароля. Для обмена сообщениями используется XML.
2) DDS (Data Distribution Service) — протокол основан на концепции «издатель - подписчик». Узлы-издатели публикуют информацию в определенный «topic» (строка, необходимая для разделения сообщений на категории, то есть для маршрутизации, примеры: температура, местоположение, давление), а узлы-подписчики получают данные из нужных разделов. Может работать как поверх протокола UDP (User

Datagram Protocol) для обеспечения низкой задержки в условиях реального времени, так и поверх TCP (Transmission Control Protocol) для обеспечения гарантии доставки.

3) MQTT (Message Queue Telemetry Transport) - это протокол коммуникации, который был разработан для передачи сообщений и данных в сетях с ограниченной пропускной способностью и низкой задержкой. [2] MQTT широко используется в сфере Интернета вещей (IoT) и приложениях, где требуется эффективная и масштабируемая передача данных. MQTT основан на модели издатель-подписчик. [16] Устройства-издатели публикуют сообщения в топики, а устройства-подписчики подписываются на топики, чтобы получать соответствующие сообщения. Это позволяет организовать эффективный обмен данными. Помимо этого, MQTT является легковесным и эффективным протоколом, что делает его подходящим для устройств с ограниченными ресурсами, такими как датчики и микроконтроллеры. Он не имеет больших накладных расходов и минимизирует использование сетевого трафика. MQTT поддерживает три уровня качества обслуживания (Quality of Service):
    a. QoS 0 (At Most Once) – сообщение отправляется клиентом без каких-либо подтверждений и повторной отправки, то есть сообщение может быть потеряно в процессе доставки;
    b. QoS 1 (At Least Once) – данный уровень предполагает подтверждение получения сообщения брокером, если подтверждение не поступает, клиент повторяет отправку сообщения, это гарантирует, что сообщение будет доставлено как минимум один раз, но могут возникнуть дубликаты;
    c. QoS 2 (Exactly Once) – обеспечивает наивысший уровень надежности. Клиент отправляет сообщение и ожидает подтверждения от брокера. Если подтверждение не поступает, клиент повторяет отправку. Однако брокер и клиент обмениваются

уникальными идентификаторами сообщений, чтобы избежать дублирования сообщений. Это гарантирует, что сообщение будет доставлено только один раз. QoS 2 наиболее подходит для сценариев, где надежность и исключение дубликатов сообщений являются критически важными.

Эти уровни позволяют контролировать надежность доставки сообщений.

MQTT может сохранять сообщения в брокере (сервере MQTT) для доставки в случае, если устройство-подписчик не доступно в момент публикации сообщения. Это обеспечивает надежность обмена данными. Более того, при установке специальных опций можно гарантировать порядок доставки. Также функционал брокера позволяет использовать механизм аутентификации по имени пользователя и паролю. Поскольку MQTT работает поверх TCP, интеграция SSL/TLS для обеспечения конфиденциальности и целостности данных не является сложной задачей.

MQTT – это мощный инструмент для организации обмена данными в условиях с ограниченной пропускной способности сети и требований к надежности обмена данными, что делает его популярным выбором для разработки в сфере IoT и не только.

4) CoAP (Constrained Application Protocol) — основан на архитектуре REST, подобной HTTP, что облегчает интеграцию с веб-службами и позволяет использовать стандартные методы, такие как GET, POST, PUT и DELETE. Предоставляет опциональные подтверждения доставки сообщений, что позволяет увеличить надёжность передачи данных. На транспортном уровне реализуется UDP. Хотя CoAP поддерживает безопасность через DTLS (Datagram Transport Layer Security), это может быть недостаточно для некоторых приложений, требующих более высокого уровня безопасности.

5) Websocket – протокол позволяет устанавливать двустороннее взаимодействие постоянного соединения между клиентом и сервером, что полезно для реального времени в IoT, однако неизбежен дополнительный расход ресурсов, поскольку протокол работает поверх HTTP (Hypertext Transfer Protocol). [3]
6) Sigfox - это технология беспроводной связи, специально разработанная для Интернета вещей (IoT). Это низкоэнергетическая, широко охватывающая сеть (LPWAN - Low Power Wide Area Network), ориентированная на передачу малых объемов данных на большие расстояния с минимальным энергопотреблением. Вследствие чего протокол особенно подходит скорее для устройств, работающих от батарей, нежели от бесперебойного электропитания. Ограниченная пропускная способность не совсем подходит для разрабатываемого приложения, поскольку планируется частое обновление данных.
7) AMQP (Advanced Message Queuing Protocol) – открытый протокол для передачи сообщений между компонентами системы. AMQP обеспечивает высокий уровень надёжности доставки сообщений, включая механизмы подтверждения доставки и возможности восстановления после сбоев. Протокол поддерживает разные уровни качества обслуживания, что позволяет гарантировать доставку сообщений даже в условиях нестабильных сетей. AMQP предоставляет сложные возможности маршрутизации и распределения сообщений, что позволяет эффективно управлять потоками данных. Включает в себя поддержку шифрования и аутентификации, обеспечивая безопасный обмен данными. Благодаря стандартизации, AMQP легко интегрируется с различными системами и платформами. [12] Однако из-за своей надежности и многофункциональности протокол может потребовать больше ресурсов, что может быть недопустимо для очень ограниченных по ресурсам систем. К тому же, AMQP является более сложным в настройке и

использовании по сравнению с некоторыми более легковесными протоколами (например, MQTT).

В результате анализа был выбран протокол MQTT. Возможности протокола отвечают поставленным требованиям. Данная технология разрабатывалась преимущественно для реализации коммуникации между устройствами, а потому она подразумевает наиболее необходимый функционал. Широкая распространенность и активное сообщество именно в сфере Интернета Вещей делают этот вариант наиболее предпочтительным.

2.3. Выбор платформы

Поскольку для проекта был выбран протокол MQTT, в первую очередь необходимо сконфигурировать и развернуть MQTT-сервер (брокер), через который будет циркулировать весь трафик. Можно приобрести вычислительные мощности на доступных открытых платформах и настроить брокера вручную, чтобы, имея «белый» IP адрес – напрямую видный в интернете, обрабатывать входящие от клиентов сообщения и отправлять некие данные в реальном времени. Минусы такого подхода: трудоемкость процесса, более того, потенциально низкая масштабируемость системы при потребности развернуть не один сервер, а кластер серверов.

Поэтому можно воспользоваться готовыми публичными решениями, с необходимым предварительными настройками и удобным графическим интерфейсом для отладки и мониторинга. То есть рационально будет обратиться к облачным технологиям.

Рассмотрим перечень наиболее популярных доступных решений:

Таблица 1 — Сравнение облачных платформ

Платформа	Преимущества	Недостатки
Amazon Web Services (AWS)	• Лидирующие позиции на рынке • Мощная инфраструктура • Доступ к платформе по всему миру	• Непроста в освоении для начинающих • Возможны высокие затраты при неправильном управлении

Платформа	Преимущества	Недостатки
		ресурсами
Microsoft Azure	• Удобная интеграция с продуктами Microsoft • Обширный набор сервисов	• Некоторые сервисы менее развиты по сравнению с AWS • Ограниченный набор инструментов для администрирования
Google Cloud	• Широкий перечень услуг в области анализа данных и машинного обучения • Достойная производительность сервисов • Конкурентоспособные цены	• Меньший выбор географических регионов по сравнению с аналогами • Документация на иностранном языке
Yandex.Cloud	• Прозрачное ценообразование с возможностью бесплатного использования и скидок при предоплате • Удобные инструменты для миграции из других облачных провайдеров • Ориентированность на российский рынок • Оперативная поддержка клиентов	• Хорошее качество связи на территории РФ • Меньшее разнообразие услуг

Каждая из представленных платформ предоставляет необходимый функционал и обеспечивает достаточный уровень гибкости и масштабируемости, однако Yandex Cloud представляется наиболее оптимальным решением: вся документация адаптирована под российскую клиентуру, имеются бесплатные тарифы, гарантирована оперативная поддержка клиентов на российском рынке. Перечисленные преимущества оптимизируют процесс разработки проекта в том числе и в будущем.

2.4. Обзор платформы Yandex Cloud и описание необходимого сервиса

Yandex Cloud подходит для широкого круга задач, от хостинга веб-сайтов и разработки приложений до анализа больших данных и машинного обучения. Ниже представлен перечень всех доступных на данный момент сервисов.

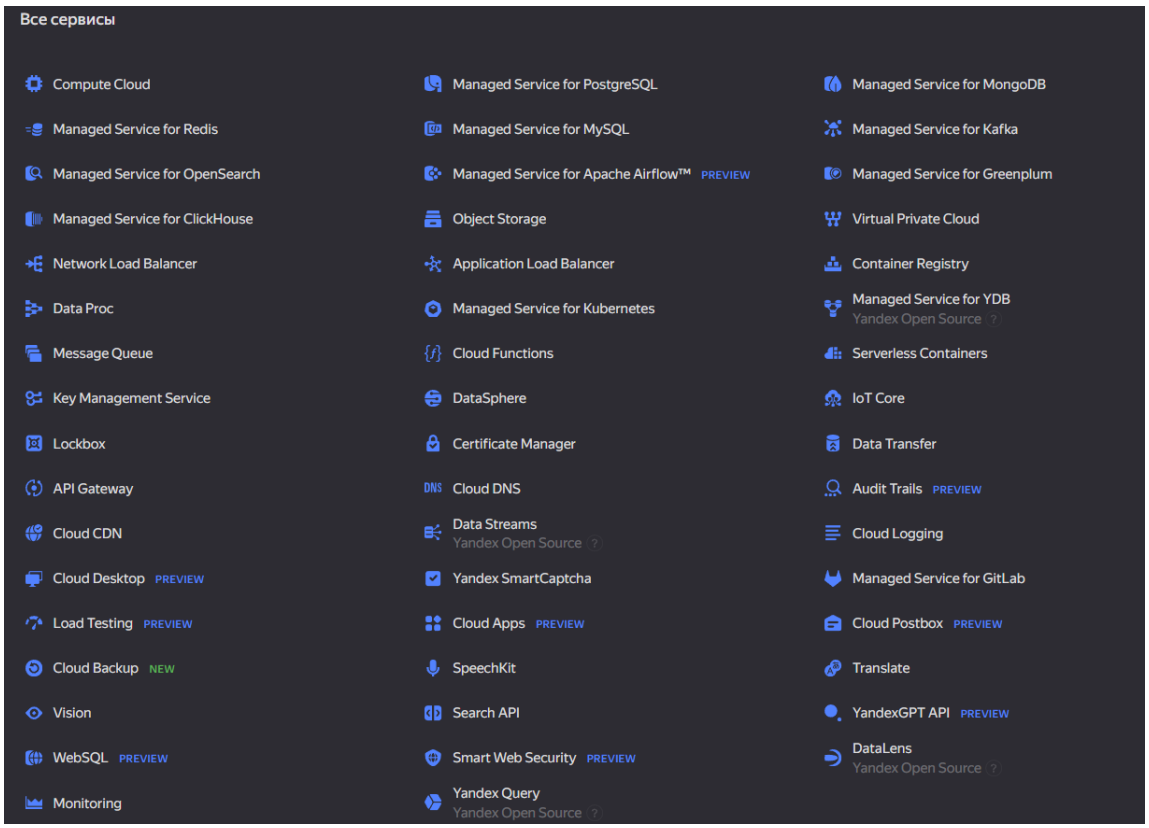

Рисунок 2 — список сервисов Yandex Cloud

Из указанного списка необходимо выбрать технологию Yandex IoT Core - сервис интернета вещей для двустороннего обмена сообщениями между реестрами и устройствами по протоколу MQTT. Платформа поддерживает автоматическое балансирование нагрузки и горизонтальное масштабирование, то есть не составит труда развернуть дополнительный сервер в кластере при необходимости. Помимо гарантий безопасности, предоставляемых стандартом MQTT, в Yandex IoT Core устройства могут обмениваться данными только после аутентификации с помощью логина и пароля, либо сертификатов формата x.509, обеспечивая защиту данных на всех точках доступа. Интерфейс сервиса позволяет просматривать информацию о состоянии брокеров, логи за указанный период в удобном формате.[6]

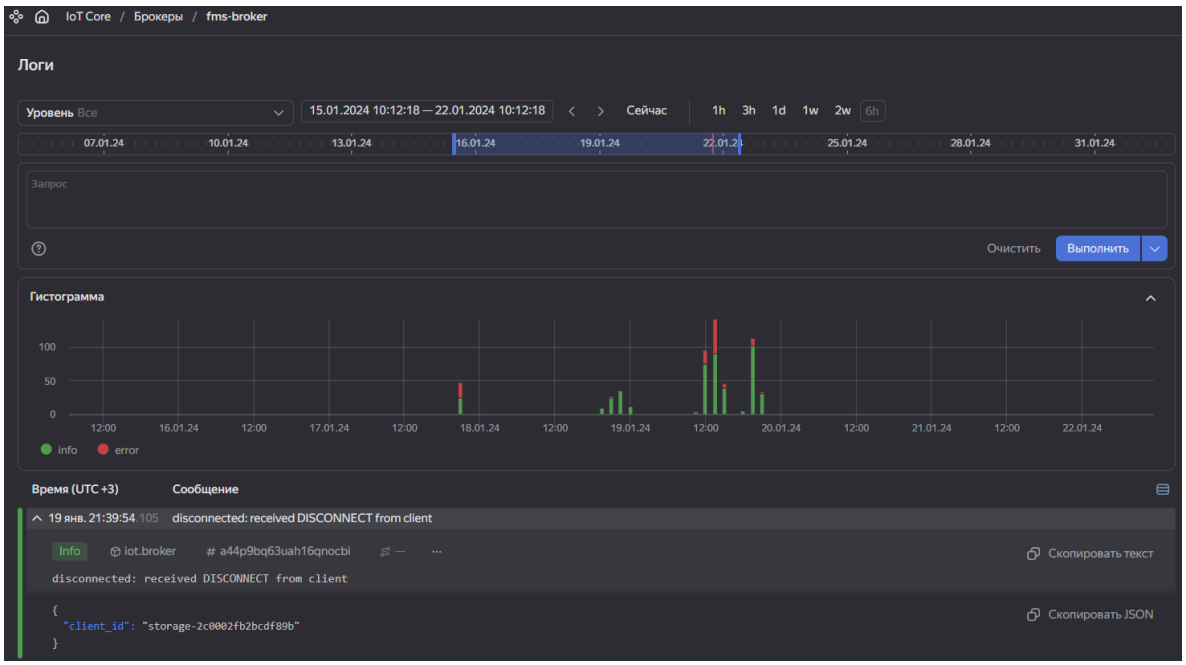

Рисунок 3 — интерфейс Yandex IoT Core (раздел «Брокеры»)

3. Техническое задание

Техническое задание на данную систему выполняется в соответствие с ГОСТ 34.602-89 “Техническое задание на создание автоматизированной системы” [1]. Документ включает в себя следующие разделы:

1) Общие сведения:

Полное наименование: серверная часть системы управления гибкой производственной системой.

2) Назначение и цели создания системы:

Назначение системы: автоматизация технологических процессов гибкой производственной системы.

Цели создания системы: автономное функционирование системы, исключение человеческого фактора из процесса производства, уменьшение травматизма на производстве.

3) Характеристика объекта автоматизации:

Основные сведения об объекте автоматизации: гибкая производственная линия состоит из склада, кольцевого конвейера, модуля распознавания и двух станков. Оборудование склада взаимодействует с конвейером, выставляя паллеты с заготовками на него. Конвейер доставляет заготовки к замкам, стоящим около станков и затем, после обработки на соответствующем оборудовании, заготовка идентифицируется модулем распознавания и доставляется обратно к замку около склада.

Сведения об условиях эксплуатации: оборудование эксплуатируется непосредственно из производственного помещения.

4) Требования к системе:

Система должна выполнять следующие функции:

- Управлять гибкой производственной линией автоматически, в соответствие с разработанным алгоритмом.
- Запуск системы должен осуществляться из любого места, где присутствует Интернет-соединение.

- Остановка системы должна осуществляться по соответствующей команде незамедлительно из любого места, где присутствует Интернет-соединение.
- Система должна обеспечивать схематичное представление метрик текущего состояния.
- Система должна обладать достаточной долей надежности и отказоустойчивости.
- Разработка программного кода системы должна вестись в соответствие с наиболее актуальными версиями языков программирования. Код должен быть читабельным и лаконичны.

Этапы разработки:

- Проанализировать существующую систему управления, выявить ее достоинства и недостатки.
- Сформулировать постановку задачи, выбрать протокол интернета вещей, платформу для реализации, удовлетворяющую всем заданным требованиям, выбрать вид системы управления.
- Разработать алгоритм взаимодействия оператора с системой, разработать алгоритм коммуникации модулей внутри системы.
- Написание программного кода для обозначенных алгоритмов, настройка подключения и конфигурация.

4. Структурное проектирование

4.1. Разработка алгоритма взаимодействия клиента ГПС с системой

В процессе анализа возможных решений была выбрана следующая схема коммуникации:

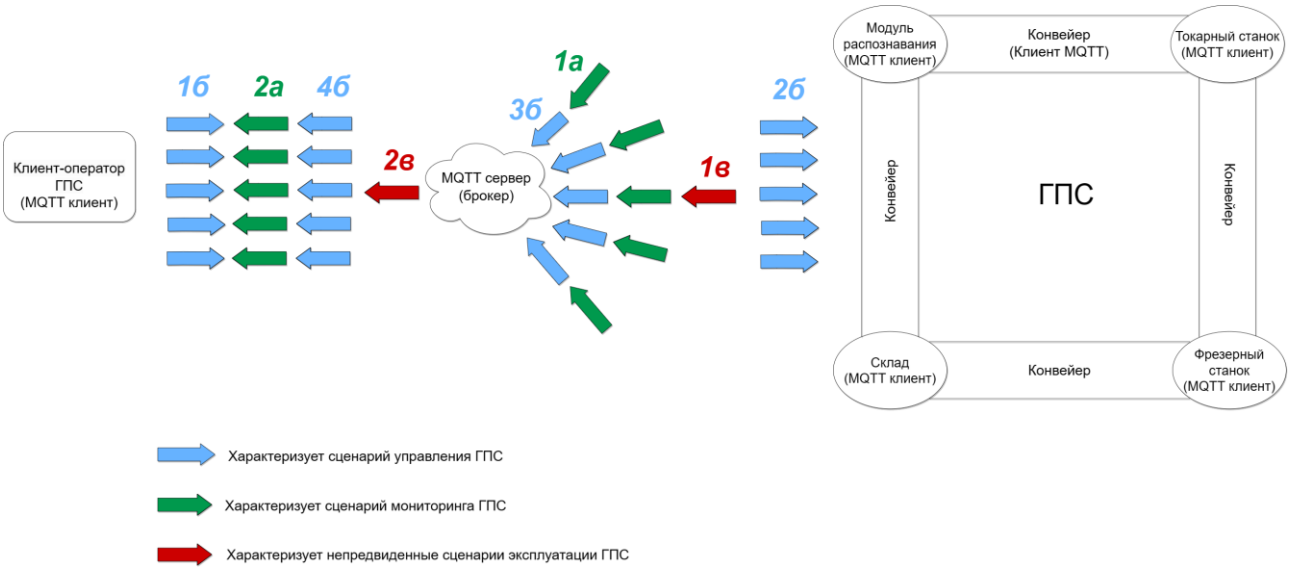

Рисунок 4 — схематичное представление взаимодействия оператора с ГПС посредством разрабатываемого ПО

Поскольку ГПС состоит из 5-ти модулей (склад, конвейер, фрезерный станок, токарный станок, модуль распознавания), необходимо написать программный код для конфигурирования, инициализации, настройки и запуска 5-ти MQTT клиентов, которые будут работать независимо друг от друга, подобный подход гарантирует максимальную гибкость и эффективность решения: все клиенты могут быть подключены либо к единому центру, либо каждый к своему локальному пункту управления, оба сценария возможны при подобном подходе, более того, трафик будет циркулировать не через одного клиента, играющего в таком случае роль «единой точки отказа», а через пять самодостаточных узлов, это наиболее оптимально, производительно и надежно.

Выделяется три различных сценария эксплуатации системы:

1. Мониторинг метрик модулей ГПС внешним клиентом

Каждый модуль ГПС будет отправлять метрики состояния в MQTT брокер с определенным временным интервалом. Предполагаются специальные топики, в которые модули ГПС как MQTT клиенты будут публиковать сообщения с информацией о состоянии в виде полезной нагрузки (рисунок 4, пункт 1а). Внешний клиент предварительно подпишется на эти топики при инициализации подключения на клиентской стороне. Как только сообщение попадет в топик, брокер незамедлительно опубликует его для подписчика (рисунок 4, пункт 2а), клиент в свою очередь автоматически обработает содержимое в программном коде. В результате пользователь будет видеть актуальную информацию о системе, меняющуюся в реальном времени в зависимости от сценария работы.

2. Отправка внешним клиентом команд каждому из модулей ГПС, ожидание результата.

   Пользователю будет доступен функционал управления каждым из модулей. При инициации команды клиентский код формирует сообщение с деталями команды и публикует его в предусмотренный топик (рисунок 4, пункт 1б). Модули гибкой производственной системы подписываются на соответствующие топики, при поступлении команды (рисунок 4, пункт 2б) отрабатывает логика сервиса, формируется результат команды, публикующийся в соответствующий топик результата обработки (рисунок 4, пункт 3б). Предполагается, что пользователь будет подписан на все подобные топики и получит ответ (рисунок 4, пункт 4б).

3. Фиксация внешним клиентом оповещений о непредвиденных сбоях, посылаемых модулями ГПС.

В случае возникновения внештатной ситуации предусмотрены специальные служебные топики, куда каждый модуль публикует сообщения о внутренних ошибках сервиса (рисунок 4, пункт 1в), тем самым оповещая пользователя о возможных сбоях (рисунок 4, пункт 2в).

Теперь необходимо определиться с системным дизайном изложенного решения. Чтобы обеспечить независимую, при этом синхронизированную работу клиентов, необходимо сделать выбор между 2 ключевыми концепциями: концепцией распределенных систем и концепцией параллельных вычислений в контексте централизованных монолитных систем.

4.2. Концепция распределенных систем

Распределенная система представляет собой комплекс компьютерных программ, которые используют вычислительные мощности различных независимых вычислительных узлов для достижения единой цели. Такие системы, известные также как распределенные вычисления, полагаются на взаимодействие и синхронизацию данных между узлами через общую сеть. Эти узлы часто являются отдельными физическими устройствами, но могут также быть программными процессами или другими вложенными системами. [8] Цель распределенных систем — устранить узкие места и единственные точки отказа.

Основные характеристики распределенных систем включают:

- Совместное использование ресурсов - в такой системе ресурсы, будь то аппаратное обеспечение, программное обеспечение или данные, могут использоваться совместно.
- Параллельная обработка на разных узлах - позволяет одновременно обрабатывать одни и те же задачи на множестве машин.
- Масштабируемость - возможность увеличения вычислительной мощности и производительности системы с добавлением новых машин.

- Распределенные системы характеризуются большей модульностью. Они состоят из множества компонентов, которые, в свою очередь, могут быть дополнительно разделены на подкомпоненты. Каждый из этих компонентов функционирует независимо и обладает полным контролем над своими сегментами. В таких системах отсутствует центральный элемент управления, регулирующий работу всех остальных частей. Для обеспечения единообразия и согласованной работы распределённой системы, необходимо определить интерфейсы, через которые компоненты могут взаимодействовать между собой.

- Компоненты распределенной системы не обязательно должны быть однородными. Один узел может быть реализован при помощи одних технологий, другой – на основе иных решений.

- В рамках распределенных систем возможно наличие отдельных компонентов, предназначенных для эксплуатации лишь одним пользователем. Когда компонент станет перегружен из-за большого числа пользователей или запросов от других компонентов, можно ввести в систему дополнительный компонент, который предоставляет аналогичные услуги и помогает распределить нагрузку. Более того, компоненты могут быть организованы таким образом, чтобы они находились ближе к своим пользователям и другим взаимодействующим с ними компонентам. Такое местоположение компонентов способствует повышению общей производительности распределенной системы, поскольку локальное взаимодействие обычно более эффективно, чем коммуникация между удаленными друг от друга элементами.

- В отличие от централизованных систем, где сбои обычно связаны с одной точкой отказа, в распределенных системах существует несколько потенциальных точек отказа. Неполадки в любом из компонентов могут привести к сбою всей системы. Это может произойти из-за сетевых

проблем или из-за чрезмерной нагрузки на отдельный компонент, превышающей его способность своевременно реагировать. Таким образом, в распределенной системе возможна ситуация, когда некоторые ее части функционируют исправно, в то время как другие, зависящие от в данный момент неработоспособных компонентов, не способны нормально функционировать.

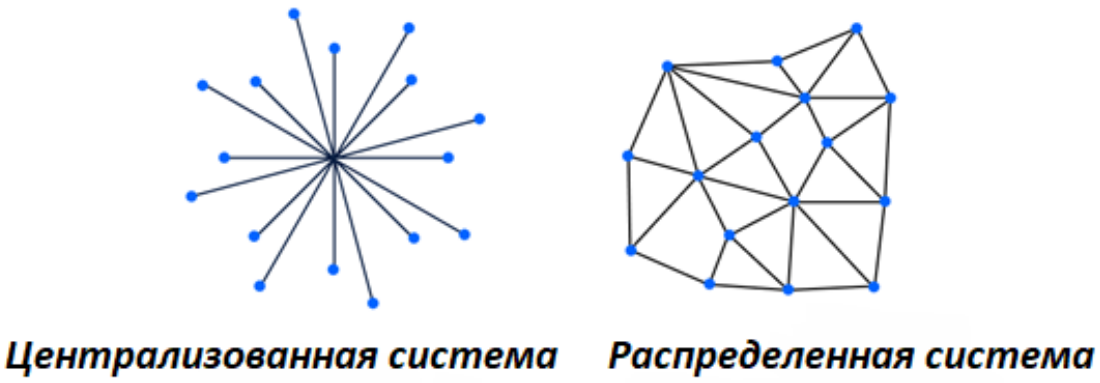

Рисунок 4 — концептуальное представление централизованных и распределенных систем

4.3. Концепция централизованных систем

Централизованные системы — это тип компьютерных систем, в которых основная обработка данных и управление ресурсами выполняются на одном центральном узле или сервере. В этих системах клиенты или периферийные устройства зависят от центрального сервера для обработки запросов, хранения данных и доступа к общим ресурсам

Основные характеристики централизованных систем:

- Централизованная система может включать в себя разнообразные элементы или компоненты. Тем не менее, эти элементы не обладают

автономностью; система в любое время управляет ими полностью. Эти компоненты системы активируются при её запуске и будут отключены или исключены в случае выключения или удаления самой системы.

- Централизованные системы обычно характеризуются схожестью в своей структуре. В процессе их создания часто используется одинаковая технология. В большинстве случаев разработка всех компонентов системы осуществляется на одном языке программирования с использованием одного компилятора. Этот компилятор производит машинный код одного типа, обеспечивая единообразие представления данных во всех элементах системы. Эффективное взаимодействие между различными частями системы может быть достигнуто благодаря программным примитивам, таким как вызовы процедур или методов.

4.4. Микросервисная и монолитная архитектура приложения

Частный случай распределенных систем – микросервисная архитектура. Это подход к разработке программного обеспечения, при котором приложение строится как набор небольших, независимых сервисов, каждый из которых выполняет определенную функцию или задачу. Эти сервисы обычно разрабатываются, разворачиваются и управляются независимо друг от друга.

Основные характеристики:

- Модульность - приложение разбивается на множество маленьких сервисов, каждый из которых отвечает за определенную функциональность.

- Независимость - каждый микросервис независим в разработке, развертывании, масштабировании и управлении. Это позволяет быстро и независимо вносить изменения в отдельные части системы.

- Распределенное управление данными - каждый микросервис может использовать свою собственную базу данных или систему управления данными.

- Связь через API (Application Programming Interface, или программный интерфейс приложения) - микросервисы общаются друг с другом посредством публичных программных интерфейсов, в рамках определенного архитектурного стиля (REST, RPC и др.)

Иной подход к разработке программного обеспечения – монолитная архитектура, при которой приложение строится как единая и неделимая единица. [5]В монолитной архитектуре все компоненты приложения (например, интерфейс пользователя, бизнес-логика, управление данными) тесно связаны и составляют единое целое.

Основные характеристики:

- Централизованное управление - все аспекты приложения, от пользовательского интерфейса до доступа к данным, управляются централизованно.

- Горизонтальное масштабирование - монолитные приложения обычно масштабируются путем запуска копий приложения на нескольких серверах.

Монолитная архитектура, несмотря на свою относительную простоту и меньшую гибкость по сравнению с микросервисной архитектурой, обладает рядом преимуществ:

- Простота разработки и развертывания – при монолитной архитектуре все компоненты приложения тесно связаны и управляются как единое целое. Это упрощает процесс разработки, тестирования и развертывания, так как

требуется работать только с одной кодовой базой и одним исполняемым файлом.

- Единообразие и согласованность - монолитные приложения обеспечивают единообразие кодовой базы.

- Отсутствие проблем синхронизации узлов - при монолитной архитектуре не требуется управлять сложной сетью взаимодействующих сервисов, что уменьшает сложность связи и координации между различными частями приложения.

- Относительное простое тестирование функциональности - тестирование монолитного приложения часто бывает более прямолинейным, так как не требуется учитывать взаимодействие между отдельными сервисами и их зависимостями.

- Транзакционная целостность - монолитные системы часто легче обеспечивают транзакционную целостность данных, поскольку все операции с данными выполняются в рамках одного и того же приложения.

- Стабильность - монолитные приложения могут быть более стабильными, так как они меньше подвержены проблемам, связанным с сетевыми задержками и разрывами связи, которые могут возникнуть в распределенных системах.

- Меньшие затраты на обслуживание - в краткосрочной перспективе монолитные приложения могут обходиться дешевле в разработке и поддержке по сравнению с микросервисами, особенно для небольших и средних проектов. Для малых приложений монолитная архитектура может быть более эффективной с точки зрения ресурсов и

производительности

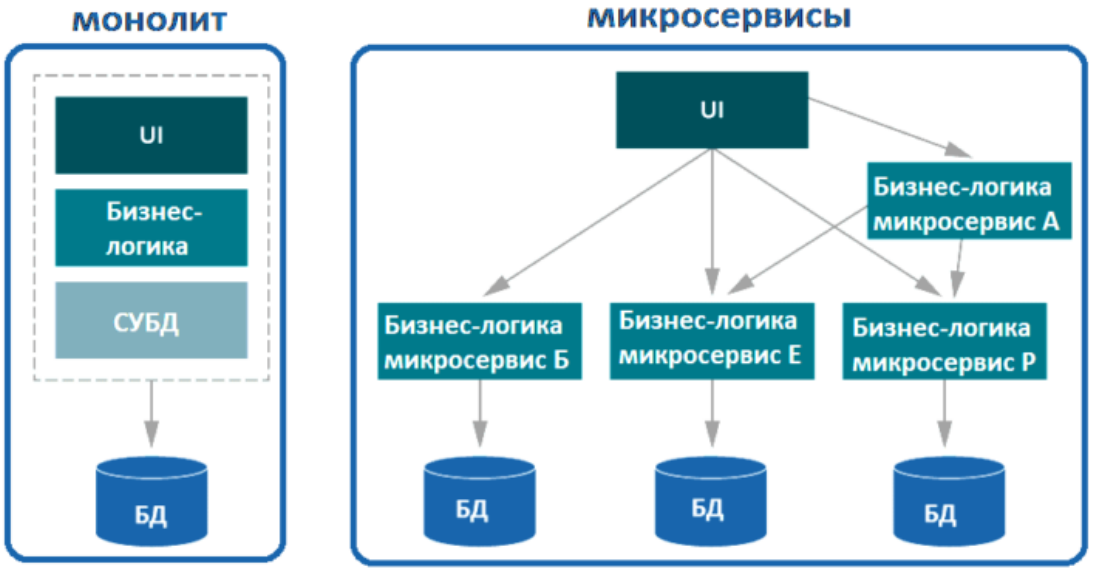

Рисунок 5 — иллюстрация микросервисной и монолитной архитектуры

В результате анализа представленных концепций была выбрана монолитная архитектура. Поскольку изложенные преимущества микросервисной архитектуры не являются критически важными для проекта с одним разработчиком, следовательно, в контексте рассматриваемой системы достоинства распределенной концепции не оправдывают недостатки в виде большей кодовой базы, более сложной конфигурации и синхронизации модулей, а также дополнительных потерь производительности в процессе коммуникации микросервисов друг с другом при выполнении клиентских команд.

4.5. Параллельные вычисления в контексте монолитной архитектуры

Для обеспечения одновременной работы всех модулей в рамках одного приложения необходимо распараллелить работу модулей ГПС.

Рассмотрим основные стратегии распараллеливания вычислений в программировании:

Многопоточность - метод распараллеливания вычислений, при котором для решения определенной задачи запускаются и выполняются несколько

независимых потоков вычислений. В операционных системах, где существует различие между понятиями "поток" и "процесс", под "потоком" понимается поток выполнения, в то время как ресурсы контролируются сущностью, известной как "процесс". Этот подход часто используется для распределения вычислительных процессов между несколькими вычислительными устройствами, такими как процессоры или их ядра.

Поток — это наименьшая единица обработки, которая может быть выполнена операционной системой. В контексте программирования поток является последовательностью исполняемых инструкций, которая может работать параллельно с другими потоками в рамках одного и того же приложения. Потоки делят с процессом, в котором они запущены, его ресурсы, такие как память и файловые дескрипторы. [15]

Многопроцессность - вариант реализации вычислений, при котором для решения поставленной задачи запускается несколько независимых процессов. В системах, где под процессом понимается сущность, владеющая ресурсами (памятью, открытыми файлами, сетевыми подключениями), несколько процессов запускаются с целью повышения отказоустойчивости приложения, а также с целью повышения безопасности.

Многопоточность позволяет нескольким потокам в рамках одного процесса осуществлять совместное разделение ресурсов, что увеличивает эффективность их использования, в случае многопроцессности для подзадачи создается новый процесс, что может быть избыточным, к тому же переключение контекста между процессами требует больше процессорного времени, что увеличивает накладные расходы. Многопроцессность создает дополнительные трудности для синхронизации данных процессов, поскольку у каждого процесса свое адресное пространство.

На основании вышеизложенных фактов была выбрана модель вычислений, основанная на многопоточности. Таким образом, каждый из

MQTT клиентов, ассоциированных с конкретным модулем ГПС, будет запускаться в отдельном потоке при старте приложения.

4.6. Разработка алгоритма внутренней коммуникации элементов ГПС

Выполнение команд клиента одним определенным элементом ГПС предполагает наличие алгоритма проверки зависимостей модуля системы. Взаимные зависимости модулей определяются на этапе проработки бизнес-логики приложения. Сначала првоеряется готовность самого исполнительного модуля. Если хотя бы одна из зависимостей возвращает сигнал неготовности выполнения какой-либо операции, основной исполнительный модуль текущей команды публикует сообщение о неготовности в соответствующий топик.

Рассмотрим алгоритм выполнения команд и проверки зависимостей на примере склада и модуля распознавания.

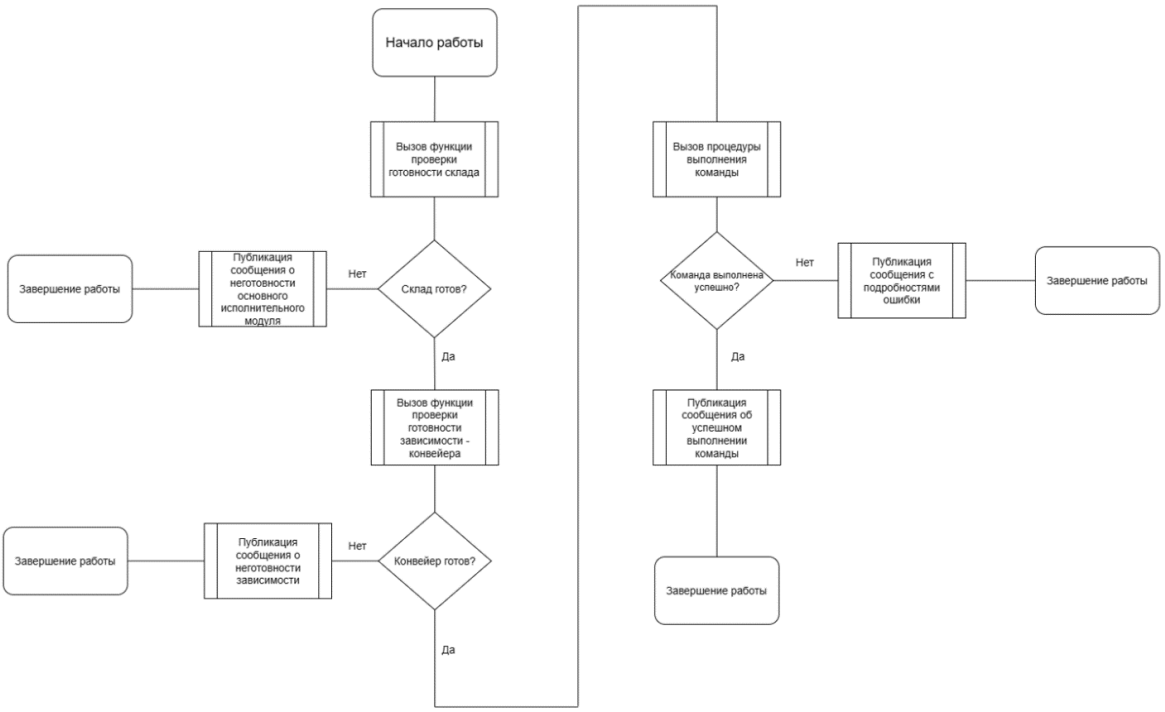

Рисунок 6 – алгоритм выполнения команды «положить заготовку на конвейер» складом

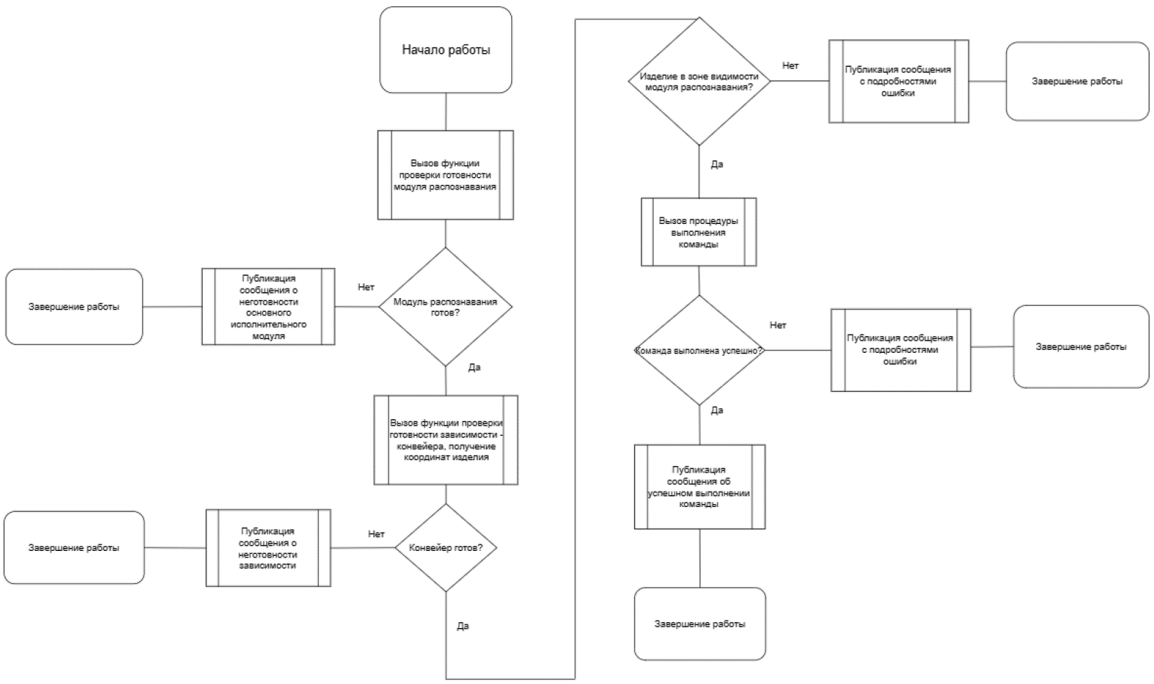

Рисунок 7 – алгоритм выполнения команды «идентифицировать изделие» модулем распознавания

5. Рабочее проектирование

5.1. Настройка Yandex IoT Core и выбор языка программирования

Для начала работы с брокером Yandex IoT Core необходимо создать сертификат формата x.509 с помощью утилиты OpenSSL:

```
openssl req -x509 \
-newkey rsa:4096 \
   -keyout private-key.pem \
   -out cert.pem \
   -nodes \
   -days 365 \
   -subj '/CN=localhost'
```

Листинг 1 — создание x.509 сертификата

Далее создается MQTT брокер, с которым будут взаимодействовать как клиент-оператор ГПС, так и клиенты-модули ГПС.

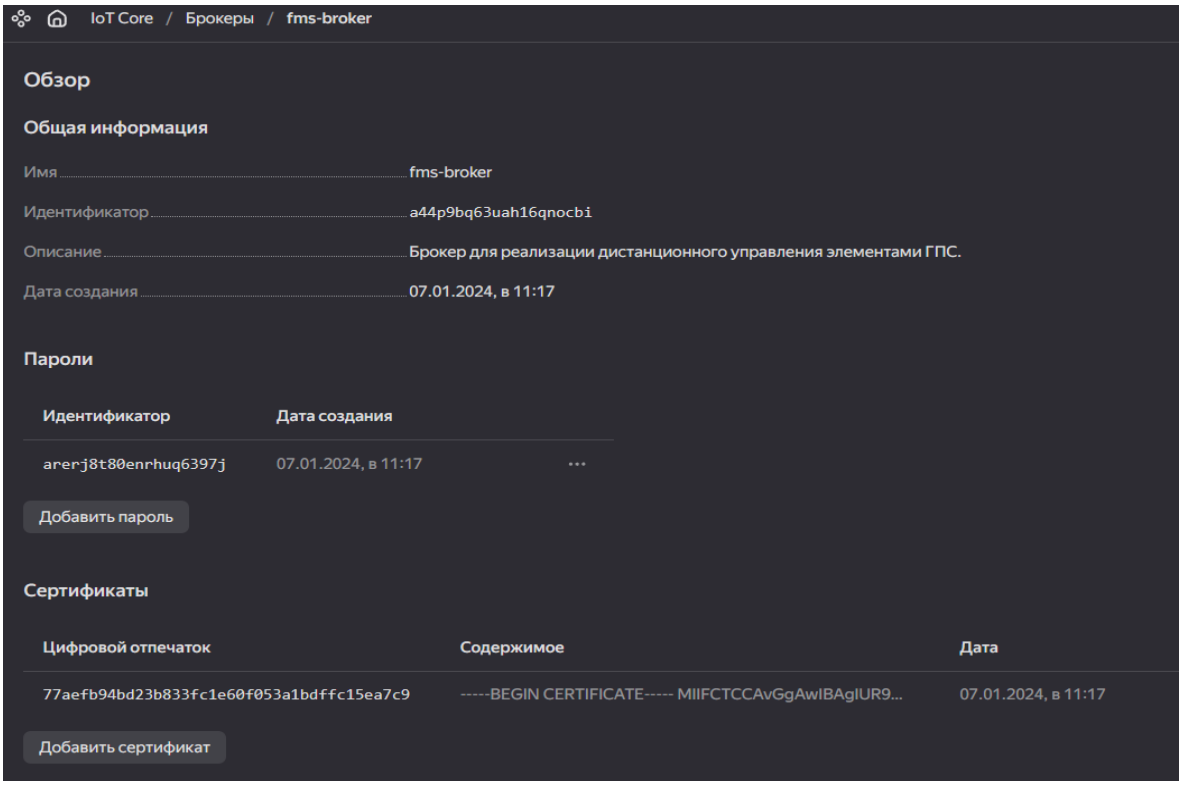

Рисунок 8 — панель управления Yandex IoT Core после настройки брокера

В настоящее время существует множество языков для написания серверных приложений, рассмотрим наиболее часто используемые:

Таблица 2 — Сравнение языков программирования

Язык программирования	Преимущества	Недостатки
JavaScript (Node.js)	• Широкое распространение • Однородность для веб-разработки(может использоваться как на клиентской, так и на серверной стороне)	• Однопоточность, как следствие, низкая производительность в многозадачных вычислениях
Python	• Обширное сообщество • Несложный синтаксис	• Невысокая производительность
Java	• Кроссплатформенность • Масштабируемость и высокая производительность • Многофункциональность	• Относительно непростой синтаксис • Требовательность к ресурсам оперативной памяти
Ruby	• Простота и читаемость синтаксиса • Удобство веб-разработки благодаря фреймворку Ruby On Rails	• Невысокая производительность • Ограниченное применение в крупных корпоративных системах
C#	• Интеграция с экосистемой Microsoft • Достойная производительность	• Зависимость от инструментария Microsoft • Ограниченная кроссплатформенность
Golang	• Кросскомпиляция • Очень быстрая компиляция • Богатая стандартная библиотека • Высокая производительность • В синтаксисе нет ничего лишнего • Крайне эффективная и лаконичная конкурентная модель, как следствие, наибольшее повышение производительности при распараллеливании вычислений по сравнению с аналогами	• Существует меньшее количество готовых решений, упрощающих работу

Наиболее предпочтительным вариантом стал язык Go, также известный как Golang, является статически типизированным языком программирования, разработанным компанией Google.

Golang широко используется в различных сценариях, таких как разработка веб-серверов, микросервисной архитектуры, инструментов для администрирования систем, а также в областях, где требуется эффективная параллельная обработка и высокая производительность. Go крайне эффективен в контексте параллельных вычислений благодаря конкурентной модели CSP. [13]

Концепция CSP (Communicating Sequential Processes) — это парадигма в программировании, предложенная Тони Хоаром в 1978 году, которая предлагает модель для параллельного исполнения процессов с использованием каналов обмена сообщениями для коммуникации между ними. Эта модель стала фундаментом для разработки многих современных систем параллельного программирования, включая язык программирования Golang. [14]

В языке программирования Go, концепция CSP реализована через горутины и каналы. Горутина — это легковесный поток, управляемый средой выполнения языка, а не операционной системой, иначе говоря, это не системный поток. Стоимость переключения контекста и расход памяти в таком случае намного ниже. Запуск горутины равносилен запуску функции, которая будет выполняться конкурентно с другими горутинами в том же адресном пространстве. Старт горутины не требует много ресурсов, потому среда выполнения Go может поддерживать одновременно десятки тысяч горутин. Каналы в Go — это средство, которое позволяет горутинам обмениваться данными. Они обеспечивают синхронизированную передачу сообщений и могут быть использованы для блокирования горутин до готовности данных.

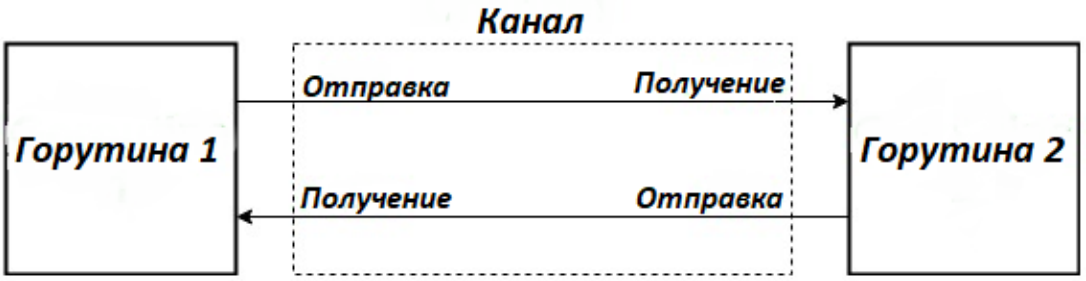

Рисунок 9 — визуализация горутин и каналов языка Go

```
func CheckTokenMQTT(ctx context.Context, token mqtt.Token, msg string, log *zap.SugaredLogger, wg *sync.WaitGroup, errCh chan<- struct{}) {
	defer wg.Done()
	select {
	case <-token.Done():
		if token.Error() != nil {
			errCh <- struct{}{}
		}
	case <-ctx.Done():
		log.Warnf("%s: OS signal was received before token release", msg)
	}
}
```

Листинг 2 — пример использования каналов и горутин для проверки MQTT токенов

Листинг 2 демонстрирует один из вариантов использования каналов и горутин: библиотека для работы с протоколом MQTT является асинхронной, то есть при совершении любого действия в адрес MQTT брокера клиентской стороне возвращается MQTT токен, который необходимо обработать для получения гарантии успешного подключения, подписки на топик, публикации сообщения. Чтобы проверка множества токенов не производилась в синхронном блокирующем режиме, применяются специальные обработчики, которые либо успевает обработать токен до получения сигнала завершения программы от операционной системы и записать результат в канал ошибок, емкость которого проверяется в дальнейшем, либо экстренно заканчивают работу и дополняют логи сервиса предупреждением о некорректном завершении.

5.2. Структура приложения

Была выбрана стандартная структура приложения, согласующаяся с современными требованиями стиля Go-разработки и лучшими практиками.[10]

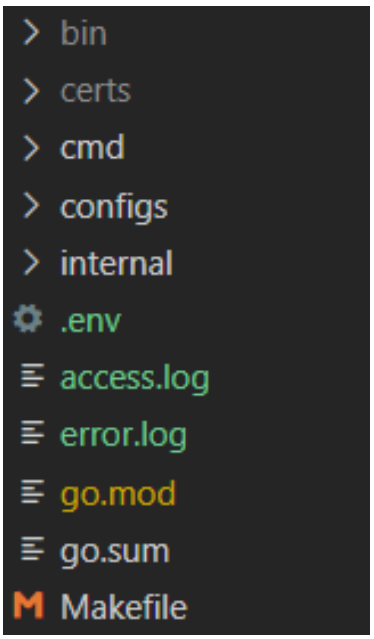

Рисунок 10 — демонстрация внешней структуры проекта

- **/bin** – директория с исполняемым файлам проекта
- **/certs** – директория с TLS сертификатами клиентов и брокера
- **/cmd** – директория с кодом точки входа в приложение
- **/configs** – в данной директории располагаются файлы конфигурации
- **/internal** – внутренний код приложения и библиотек, код данной директории не должен использоваться в других приложениях и библиотеках. Данный шаблон поддерживается самим компилятором языка
- **.env** – файл формата «ключ - значение» с переменными окружения, используемыми приложением (данные для инициализации подключений, прочая конфиденциальная информация). Содержимое файла не принято делать публичным.
- **access.log** – файл со штатными логами приложения. Лог - текстовый файл, куда автоматически записывается важная информация о работе системы или программы. По умолчанию логи записываются в указанные файлы и потоки вывода/ошибок.
- **error.log** – файл с логами ошибок приложения

- **go.mod, go.sum -** служебные файлы со списком и контрольной суммой всех зависимостей проекта
- **Makefile –** набор инструкций для утилиты make, позволяющей быстрее и удобнее собирать проект и запускать проверки стиля кода[18]

```
PROJECT_BIN = $(CURDIR)/bin
$(shell [ -f bin ] || mkdir -p $(PROJECT_BIN))
GOLANGCI_LINT = $(PROJECT_BIN)/golangci-lint

build:
    go build -o $(PROJECT_BIN)/app cmd/app/*.go

.PHONY:
run: build
    bin/app $(ARGS)

.install-linter:
    [ -f $(PROJECT_BIN)/golangci-lint ] || curl -sSfL
https://raw.githubusercontent.com/golangci/golangci-
lint/master/install.sh | sh -s -- -b $(PROJECT_BIN) v1.55.2

.PHONY:
lint: .install-linter
    $(GOLANGCI_LINT) run ./... --config=configs/.golangci.yml

.PHONY:
lint-fast: .install-linter
    $(GOLANGCI_LINT) run ./... --fast --
config=configs/.golangci.yml
```

Листинг 3 – содержимое Makefile с инструкциями сборки приложения и запуском необходимых проверок стиля

5.3. Архитектура проекта, организация кодовой базы

Перед началом работы над любым проектом крайне важно определиться с архитектурным стилем системы, правильно выбранная архитектура упрощает

тестирование, поддержку, модификацию, разработку и развертывание, а также обеспечивает независимость.

“Задача архитектуры ПО — минимизация человеческих ресурсов при разработке и последующем сопровождении системы”, — Роберт С. Мартин. [9]

Выше приведены слова Роберта С.Мартина, американского инженера и программиста, консультанта и автора в области разработки ПО. Он является создателем концепции под названием «Чистая архитектура», по канонам которой данный организован проект.

"Чистая архитектура" (Clean Architecture) — это концепция в разработке программного обеспечения, направленная на создание систем, которые являются независимыми от UI (пользовательского интерфейса), баз данных, фреймворков и любых внешних ресурсов. Основная цель этого подхода — создать систему с высокой степенью организации, где бизнес-логика и другие важные аспекты приложения не зависят от внешних факторов и легко адаптируются к изменениям.

Внедрение "Чистой Архитектуры" Роберта Мартина в проект приносит ряд значительных преимуществ, которые могут улучшить качество программного обеспечения и упростить процесс его разработки и поддержки.

Выделим основные преимущества:

- Улучшенная тестируемость
- Гибкость и легкость внесения изменений
- Независимость от сторонних библиотек
- Улучшенная отзывчивость и производительность
- Облегчение масштабирования и поддержки
- Повышение качества кода
- Улучшенная сопровождаемость

Основные принципы «Чистой архитектуры»:

- Независимость от фреймворков - система не должна зависеть от используемых фреймворков, что позволяет минимизировать влияние изменений во фреймворках на бизнес-логику.
- Тестируемость - бизнес-правила можно тестировать без UI, баз данных, веб-сервера или любых других внешних элементов.
- Независимость от UI - UI можно легко изменять без изменения остальной системы. Интерфейс пользователя должен быть отделен от бизнес-логики и данных.
- Независимость от базы данных - бизнес-правила не связаны с базой данных, так что логика приложения не зависит от конкретных механизмов хранения данных.

Рисунок 11 — иллюстрация чистой архитектуры

Изложенные выше принципы строго подчиняются ключевому правилу «Чистой Архитектуры» - правилу зависимостей. Правило гласит, что направление зависимостей в исходном коде должно следовать от внешних

кругов диаграммы к внутренним. Это подразумевает, что элементы, расположенные во внутренней части круга, не могут знать что-либо о внешних слоях системы, то есть они полностью независимы от элементов внешней части. На диаграмме это иллюстрируется направлением черных стрелок, которые следуют указанному принципу.

Описанному правилу помогает следовать паттерн «Инъекция зависимостей». Одна из трактовок этого подхода утверждает следующее: вместо того, чтобы предоставить каждому внутреннему слою конкретную реализацию сущности внешнего уровня, тем самым создав зависимость внутренней логики от слоя уровнем выше, необходимо соблюсти на стороне зависимого (внешнего) слоя некий контракт, набор правил/методов, который нужен внутренней бизнес-логике сервиса, иначе говоря, инвертировать зависимости при выполнении программы при помощи интерфейсов. А при инициализации системы следует динамически подставлять нужную имплементацию интерфейса - сущность, выполняющую контракт. В результате мы добьемся гибкости и простой сменяемости модулей: если появится необходимость адаптировать проект для работы с новым внешним сервисом (например, любым хранилищем или каким-либо микросервисом), будет намного проще реализовать новое требование – достаточно просто дописать код, который будет работать с нужной технологией, а бизнес-логика останется неизменной, поскольку она независима и самодостаточна.

На практике внутренняя структура проекта может быть организована следующим образом:

- Слой **entity (domain) –** содержит доменные сущности приложения, является ключевым компонентом, описывающим наиболее значимую и неизменяемую логику сервиса.

- Слой **usecase (service) –** характеризует бизнес-правила приложения, основные сценарии работы, определяет взаимодействие между

различными сущностями уровня **entity**, преобразовывает данные из формата, понятного внешним слоям, в формат, требуемой для бизнес-логики и наоборот.

- Слой **delivery (adapters/controllers) –** слой доставки. Служит промежуточным звеном между внешними сервисами (пользовательский интерфейс, хранилища, микросервисы и т.д.) и слоем **service.** Основная функция – прием и валидация данных от сторонних сервисов и их доставка во внутренние слои приложения, а также отправка обработанных данных обратно в том формате, в котором они изначально пришли.

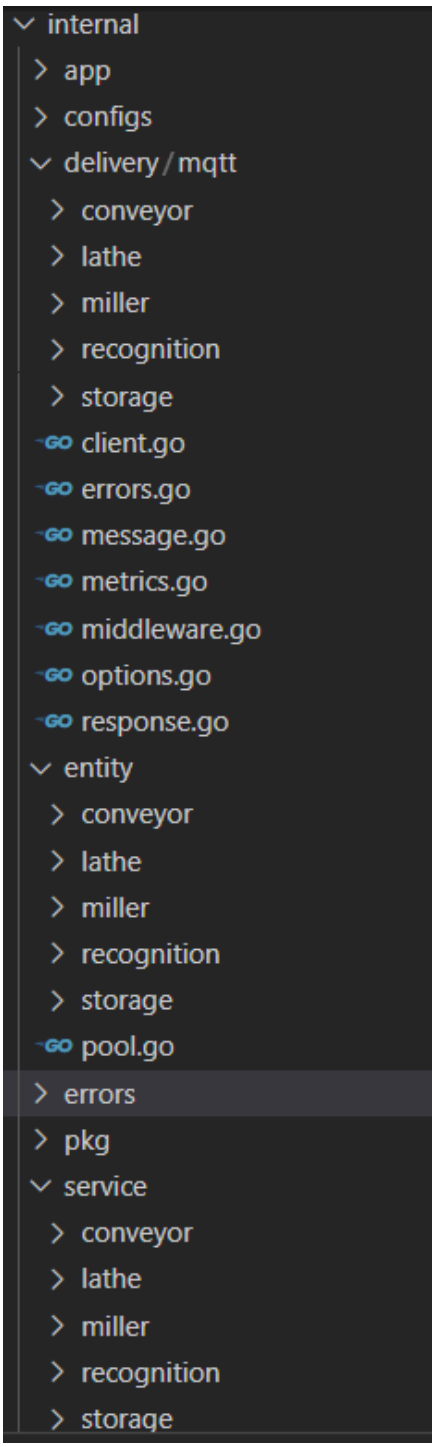

Рисунок 12 — демонстрация организации внутренней структуры проекта согласно «Чистой архитектуре»

5.4. Слой Entity

Всего представлены пять основных сущностей сервиса – Склад (Storage), Конвейер (Conveyor), Токарный станок (Lathe), Фрезерный станок (Miller) и Модуль распознавания (Recognition). В проекте объявлены контракты для каждой из сущностей – перечень методов, который должен присутствовать у

каждой конкретной реализации каждой сущности. Иначе говоря, в коде объявлены интерфейсы, описывающие ключевую логику приложения. [17]

Интерфейсный тип в Go декларирует конкретные методы, которые должны быть у какого-то другого типа, называемого реализацией интерфейса.[11] Чтобы использовать интерфейс, необходима какая-либо реализация, поскольку у самого интерфейсного типа не могут быть описаны методы.

Объявлен следующий перечень методов для каждой сущности:

Сущность «Склад»:

- Объявить о готовности/неготовности к работе
- Предоставить заготовку
- Принять изделие
- Предоставить метрики состояния

```
type Storage interface {
    IsReady(context.Context) error
    ProvideRawWorkpiece(context.Context) error
    AcceptFinishedWorkpiece(context.Context) error
    Metrics(context.Context) (*Metrics, error)
}
```

Листинг 4 – перечень методов сущности «Склад»

Сущность «Конвейер»:

- Объявить о готовности/неготовности к работе
- Изменить положение заготовки/изделия
- Предоставить метрики состояния

```
type Conveyor interface {
    IsReady(context.Context) (*WorkpieceLocation, error)
  SetWorkpieceLocation(context.Context, WorkpieceLocation, bool)
error
    Metrics(context.Context) (*Metrics, error)
```

```
}
```

Листинг 5 – перечень методов сущности «Конвейер»

Сущность «Токарный станок»:

- Объявить о готовности/неготовности к работе
- Обработать заготовку
- Предоставить метрики состояния

```
type Lathe interface {
    IsReady(context.Context) error
    HandleWorkpiece(context.Context) error
    Metrics(context.Context) (*Metrics, error)
}
```

Листинг 6 – перечень методов сущности «Токарный станок»

Сущность «Фрезерный станок»:

- Объявить о готовности/неготовности к работе
- Обработать заготовку
- Предоставить метрики состояния

```
type Miller interface {
    IsReady(context.Context) error
    HandleWorkpiece(context.Context) error
    Metrics(context.Context) (*Metrics, error)
}
```

Листинг 7 – перечень методов сущности «Фрезерный станок»

Сущность «Модуль распознавания»:

- Объявить о готовности/неготовности к работе
- Распознать заготовку
- Предоставить метрики состояния

```
type Recognition interface {
    IsReady(context.Context) error
```

```
        Recognize(context.Context) (WorkpieceType, error)
        Metrics(context.Context) (*Metrics, error)
    }
```

Листинг 8 – перечень методов сущности «Модуль распознавания»

Для каждого из интерфейсов написана конкретная реализация, эмулирующая работу модуля ГПС, обработку команд и состояния. Методы реализаций возвращают объявленные сервисные ошибки в случае неисправностей. Исходный код реализаций для каждой из сущностей представлен в приложениях 1-5.

```
type mockLathe struct{}

func NewLathe() *mockLathe {
    return &mockLathe{}
}
func (l *mockLathe) IsReady(ctx context.Context) error {
    switch rand.Intn(5000) {
    case 0:
        return &errors.TimeoutExceededError{Service:
errors.Lathe}
    default:
        return nil
    }
}
func (l *mockLathe) HandleWorkpiece(context.Context) error {
    return nil
}
func (l *mockLathe) Metrics(ctx context.Context) (*Metrics,
error) {
    return &Metrics{
        Ready: l.IsReady(ctx) == nil,
    }, nil
}
```

Листинг 9 – Исходный код реализации сущности «Токарный станок»

Альтернативная реализация интерфейса – структура, содержащая параметры подключения к центру управления модулем и инкапсулирующая за теми же методами сетевое взаимодействие.

Следует заметить, что каждая из представленных реализаций содержит специальные инструменты для синхронизации доступа к данным реализации, которые помогают управлять доступом к общим ресурсам и координировать работу между горутинами. В данном случае это структура Mutex из пакета sync. Мьютекс гарантирует, что только одна горутина может войти в критическую секцию кода, где происходит доступ или изменение общего ресурса, такого как глобальная переменная или структура данных.

При отсутствии примитивов синхронизации конкурентный доступ к данным может спровоцировать так называемое «состояние гонки» - возникает, когда два или более потока или процесса в многопоточной или распределенной системе одновременно пытаются получить доступ к общему ресурсу, при этом порядок доступа к этому ресурсу влияет на результат выполнения программы, то есть результат выполнения операций становится непредсказуемым и зависит от того, какой поток или процесс получает доступ к ресурсу первым.

5.5. Слой Service

На уровне Service определены 5 сервисов, ассоциированных с ключевыми сущностями проекта. Сервисы проекта описывают ключевые сценарии использования и взаимодействия основных сущностей из слоя entity. Смысл методов сервиса – охарактеризовать логику работы не конкретного модуля как такового, а описать алгоритм функционирования всей ГПС через взаимодействие модулей в разных сценариях.

Перечислим сценарии, характерные для каждой из сущностей согласно контрактам слоя Service:

Сервис «Склад»:

- Положить необработанную заготовку на конвейер
- Принять изделие от конвейера

- Предоставить метрики состояния

```
type Service interface {
    ProvideWorkpiece(context.Context) error
    AcceptWorkpiece(context.Context) error
    GetMetrics(context.Context) (*storage.Metrics, error)
}
```

Листинг 10 – перечень методов сервиса «Склад»

Сервис «Конвейер»:

- Переместить заготовку к токарному станку
- Переместить заготовку к фрезерному станку
- Переместить изделие к модулю распознавания
- Переместить изделие к складу
- Предоставить метрики состояния

```
type Service interface {
    MoveWorkpieceToLathe(context.Context) error
    MoveWorkpieceToMiller(context.Context) error
    MoveWorkpieceToRecognition(context.Context) error
    MoveWorkpieceToStorage(context.Context) error
    GetMetrics(context.Context) (*conveyor.Metrics, error)
}
```

Листинг 11 – перечень методов сервиса «Конвейер»

Сервис «Токарный станок»:

- Обработать заготовку
- Предоставить метрики состояния

```
type Service interface {
    HandleWorkpiece(context.Context) error
    GetMetrics(context.Context) (*lathe.Metrics, error)
}
```

Листинг 12 – перечень методов сервиса «Токарный станок»

Сервис «Фрезерный станок»:

- Обработать заготовку
- Предоставить метрики состояния

```
type Service interface {
    HandleWorkpiece(context.Context) error
    GetMetrics(context.Context) (*lathe.Metrics, error)
}
```

Листинг 13 – перечень методов сервиса «Фрезерный станок»

Сервис «Модуль распознавания»:

- Распознать изделие
- Предоставить метрики состояния

```
type Service interface {
  RecognizeWorkpiece(context.Context)
(recognition.WorkpieceType, error)
    GetMetrics(context.Context) (*recognition.Metrics, error)
}
```

Листинг 14 – перечень методов сервиса «Модуль распознавания»

Каждая из реализаций сервиса содержит специальную структуру, хранящую все 5 ключевых сущностей уровня entity, это необходимо как раз для вызова проверок зависимостей.

```
type Pool struct {
    Storage     storage.Storage
    Conveyor    conveyor.Conveyor
    Recognition recognition.Recognition
    Lathe       lathe.Lathe
    Miller      miller.Miller
}
```

Листинг 15 – структура со всеми ключевыми сущностями, необходимая каждому сервису

Ниже представлена часть реализации сервиса «Конвейер».

```
type service struct {
    log  *zap.SugaredLogger
```

```
    pool *entity.Pool
}

func NewService(log *zap.SugaredLogger, pool *entity.Pool)
*service {
    return &service{log: log, pool: pool}
}

func (c *service) MoveWorkpieceToRecognition(ctx
context.Context) error {
    if _, err := c.pool.Conveyor.IsReady(ctx); err != nil {
        return err
    }
    if err := c.pool.Recognition.IsReady(ctx); err != nil {
        return &errors.DepsNotReadyError{Service:
errors.Conveyor, Dependency: errors.Recognition, Reason: err}
    }
    return c.pool.Conveyor.SetWorkpieceLocation(ctx,
conveyor.WorkpieceLocation{
        X: locationForRecognitionX,
        Y: locationForRecognitionY,
    }, false)
}
```

Листинг 16 – часть реализации сервиса «Конвейер»

В каждом из методов конвейера по перемещению заготовки/изделия проверяется готовность модуля ГПС, к которому необходимо переместить объект. После получения статуса зависимости возможны два сценария: если зависимость не готова, возвращается ошибка соответствующего типа, в противном случае отрабатывает метод основной сущности сервиса. Полный исходный код реализаций представлен в приложениях 7-11.

5.6. Слой Delivery

На этом уровне описана логика промежуточного слоя между внешними сервисами и внутренней структурой приложения. Основное содержимое слоя – работа с сетевым протоколом MQTT.

Можно выделить несколько главных пунктов:

- Инициализация MQTT клиентов
- Описание промежуточного слоя функций, выполняющихся перед началом основного алгоритма обработки
- Описание обработчика, отправляющего метрики модулей с определенной периодичностью
- Описание обработчиков сообщений топиков, на которые подписаны модули ГПС
- Описание формата ответов внешнему клиенту, метода сериализации полезной нагрузки

5.6.1. Инициализация MQTT клиента

```
type ClientMQTT struct {
    mqtt.Client
    Log *zap.SugaredLogger
}

func NewClientMQTT(opts *mqtt.ClientOptions, log
*zap.SugaredLogger) *ClientMQTT {
    client := mqtt.NewClient(opts)
    return &ClientMQTT{Client: client, Log: log}
}

func (c *ClientMQTT) Publish(topic string, qos byte, retained
bool, payload interface{}) mqtt.Token {
    token := c.Client.Publish(topic, qos, retained, payload)
    pubID, err := uuid.NewRandom()
    if err != nil {
        c.Log.Warnln("Generate PublishID: ", err)
    }
    c.Log.Infow(
        "Published message:",
        zap.String("ID", pubID.String()),
        zap.String("topic", topic),
```

```
            zap.Int8("QOS", int8(qos)),
            zap.Bool("retained", retained),
        )
        return token
    }
```

Листинг 17 – инициализация MQTT клиента

Листинг 17 содержит код объявления структуры MQTT клиента и переопределение метода Publish, реализуемого стандартным MQTT клиентом библиотеки для работы с MQTT, на альтернативный вариант, при котором публикация любого отправляемого сообщения уникально идентифицируется и логируется сервисом для удобства отладки и мониторинга.

Инициализация MQTT клиента требует наличия специальных MQTT опций – параметров подключения. Ниже демонстриуреся работа с данными опциями.

```
   var (
    _connectTimeout = 5 * time.Second
    _writeTimeout   = 5 * time.Second
    _keepAlive      = 50 * time.Second
    _pingTimeout    = 5 * time.Second
)
type DefaultHandlers struct {
    OnConnect         mqtt.OnConnectHandler
    OnConnectionLost  mqtt.ConnectionLostHandler
    OnReconnect       mqtt.ReconnectHandler
    ConnectionAttempt mqtt.ConnectionAttemptHandler
}
func setDefaultHandlers(opts *mqtt.ClientOptions, defaultHandlers
DefaultHandlers) {
    opts.OnConnectionLost = defaultHandlers.OnConnectionLost
    opts.OnConnect = defaultHandlers.OnConnect
    opts.OnReconnecting = defaultHandlers.OnReconnect
}
func NewClientOptions(brokerCfg *configs.BrokerConfig, tlsCfg
```

```
*tls.Config, defaultHandlers DefaultHandlers, clientID string)
*mqtt.ClientOptions {
    opts := mqtt.NewClientOptions()
    opts.SetTLSConfig(tlsCfg)

    opts.AddBroker(brokerCfg.Address)
    opts.SetUsername(brokerCfg.Username)
    opts.SetPassword(brokerCfg.Password)

    opts.SetClientID(clientID)
    opts.SetOrderMatters(true)
    opts.SetCleanSession(false)

    opts.SetConnectTimeout(_connectTimeout)
    opts.SetWriteTimeout(_writeTimeout)
    opts.SetKeepAlive(_keepAlive)
    opts.SetPingTimeout(_pingTimeout)

    opts.SetConnectRetry(true)
    opts.SetAutoReconnect(true)

    setDefaultHandlers(opts, defaultHandlers)
    return opts
}
```

Листинг 18 – инициализация опций MQTT клиента

Инициализация опций подразумевает несколько этапов:

- Создание стандартных MQTT опций
- Настройка параметров брокера – адрес, имя пользователя, пароль
- Настройка клиентской сессии – ID клиента, порядок сообщений, обьявление новой «чистой» сессии
- Установка таймаутов подключения
- Установка флагов для автоматического подключения в случае потери соединения

- Установка служебных обработчиков подключения, потери подключения, повторного подключения

Каждый из вышеперечисленных этапов присутствует в листнге 18

5.6.2. Описание промежуточного слоя функций, выполняющихся перед началом основного алгоритма обработки

В контексте разработки серверного ПО часто прибегают к подходу, при котором последовательность действий, повторяющихся перед началом работы всех или группы основных обработчиков, выносится в промежуточный слой отдельных функций, такие функции называются middleware – промежуточное программное обеспечение.

Основные аспекты middleware:

- Посредник между запросом и ответом - middleware действует как промежуточный слой между получением запроса от клиента и отправкой ответа от сервера.
- Модульность и повторное использование - middleware предоставляет модульный способ повторного использования общего кода, что помогает избегать дублирования и упрощает управление кодом
- Цепочка обработки - middleware можно организовать в цепочку, где каждый элемент цепочки передает обработку следующему, позволяя создать гибкую и легко настраиваемую конвейерную обработку запросов.

Типичные задачи middleware:

- Аутентификация и авторизация - проверка пользовательских данных и определение прав доступа.
- Логирование - запись информации о запросах и ответах для отладки.
- Обработка тела запроса при наличии - анализ данных запроса и преобразование их в удобный для работы формат.
- Управление сессиями - управление пользовательскими сессиями и состояниями.
- Обработка ошибок - централизованная обработка ошибок и исключений.

- Управление политиками безопасности для запросов из разных источников.
- Кэширование и компрессия - кэширование ответов и их компрессия для улучшения производительности.

Список реализованных в проекте функций-middleware:

- Функция журналирования – логирует каждое приходящее из интересующих топиков сообщение, фиксируются топик сообщения, уникальный идентификатор сообщения, время обработки сообщения и другие служебные параметры.

```
func LoggingMiddleware(next mqtt.MessageHandler)
mqtt.MessageHandler {
    return func(client mqtt.Client, msg mqtt.Message) {
        customClient, ok := client.(*ClientMQTT)

        if !ok {
            log.Println("Logging middleware: can't extract
client from the interface")
            next(client, msg)
            return
        }

        start := time.Now()
        next(client, msg)

        customClient.Log.Infow(
            "Got message",
            zap.String("topic", msg.Topic()),
            zap.Int8("QOS", int8(msg.Qos())),
            zap.Bool("retained", msg.Retained()),
            zap.Uint16("MessageID", uint16(msg.MessageID())),
            zap.Duration("processing_time",
time.Since(start)),
        )
```

```
        }
    }
```

Листинг 19 – исходный код функции-middleware для логирования

- Функция восстановления после критической ошибки сервиса – фиксирует непредвиденную ошибку приложения, которую невозможно обнаружить на этапе компиляции программы, только в процессе ее исполнения. Примеры подобных ошибок – деление на ноль, обращение к участку памяти, к которой у приложения нет доступа, неправильное приведение типов, попытка закрытия закрытого канала. В случае обнаружения подобной ошибки функция восстановит состояние системы и предотвратит экстренное завершение программы, после восстановления сервис сообщит клиенту о критической ошибке в сообщении, которое публикуется в предусмотренный для этого служебный топик.

```
var (
    topicForPanicMiddleware    = "/aux/accidents/panic"
    retainedForPanicMiddleware = false
    qosForPanicMiddleware      = 1
)
func RecoverMiddleware(next mqtt.MessageHandler)
mqtt.MessageHandler {
    return func(client mqtt.Client, msg mqtt.Message) {
        defer func() {
            if err := recover(); err != nil {
                ResponseError(
                    struct{}{},
                    client, msg.Topic(),
                    topicForPanicMiddleware,
                    byte(qosForPanicMiddleware),
                    retainedForPanicMiddleware,
                    &serviceGotPanicError{msg: msg},
                )
            }
        }()
```

```
        next(client, msg)
    }
}
```

Листинг 20 – исходный код функции-middleware для восстановления системы

- Функция замены стандартных сущностей библиотеки MQTT на расширенные реализации - служебная функция, необходимая для предоставления контекста события обработчикам, корректного логирования и т.д.

```
func ReplaceMessageClientMiddleware(ctx context.Context,
customClient *ClientMQTT) Middleware {
    return func(next mqtt.MessageHandler) mqtt.MessageHandler {
        return func(client mqtt.Client, msg mqtt.Message) {
            customMsg := &MessageWithCtx{Message: msg, ctx: ctx}
            next(customClient, customMsg)
        }
    }
}
```

Листинг 21 – исходный код функции-middleware для замены стандартных сущностей библиотеки MQTT

- Функция проверки произведенной замены - проверяет, действительно ли была произведена подмена сущностей. Если расширения функционала не было, функция публикует сообщение об ошибке в служебный топик и принудительно завершает обработку.

```
var (
    topicForCheckMsgMiddleware = "/aux/accidents/internals"
    retainedForCheckMsgMiddleware = false
    qosForCheckMsgMiddleware      = 1
)
func CheckMsgWithCtxMiddleware(next mqtt.MessageHandler)
mqtt.MessageHandler {
    return func(client mqtt.Client, msg mqtt.Message) {
```

```
        _, gotClient := client.(*ClientMQTT)
        if !gotClient {
            log.Println("Check msg and client middleware: can't
extract custom client")
        }

        _, gotMsg := msg.(*MessageWithCtx)
        if !gotMsg {
            log.Println("Check msg and client middleware: can't
extract custom message")
        }

        if !gotClient || !gotMsg {
            ResponseError(
                struct{}{},
                client, msg.Topic(),
                topicForCheckMsgMiddleware,
                byte(qosForCheckMsgMiddleware),
                retainedForCheckMsgMiddleware,
                &customMsgClientNotFoundError{msg: msg},
            )
            return
        }
        next(client, msg)
    }
}
```

Листинг 22 – исходный код функции-middleware для проверки замены сущностей

```
type Middleware func(mqtt.MessageHandler) mqtt.MessageHandler

func ApplyMiddlewareStack(initialHandler mqtt.MessageHandler,
middlewares ...Middleware) mqtt.MessageHandler {
    for _, middleware := range middlewares {
        initialHandler = middleware(initialHandler)
    }
```

```
    return initialHandler
}
```

Листинг 23 – исходный код функции для применения middleware к исходному обработчику

Функция листинга 23 принимает на вход исходный обработчик и возвращает цепочку примененных функций-middleware, где последнее звено цепочки – исходный обработчик.

5.6.3. Описание обработчика, отправляющего метрики модулей с определенной периодичностью

Реализован следующий алгоритм мониторинга системы: для каждого из модулей запускается фоновый обработчик в своем легковесном потоке выполнения – горутине. Обработчик публикует метрики модуля в специальный топик с определенной периодичностью, в случае поступления служебного сигнала операционной системы, свидетельствующего о штатном завершении работы, обработчики завершают работу.

```
var (
    metricsPeroid = 3 * time.Second
)
type MetricsProvider interface {
    PushMetrics(context.Context, mqtt.Client)
}

func RunMetricsWorker(ctx context.Context, client mqtt.Client,
provider MetricsProvider, service string, log *zap.SugaredLogger,
wg *sync.WaitGroup) {
    defer wg.Done()
    log.Infof("Service %s beginning to push it's metrics",
service)
    for {
        select {
        case <-time.After(metricsPeroid):
            provider.PushMetrics(ctx, client)
        case <-ctx.Done():
```

```
            log.Infof("Service %s finishing pushing metrics",
service)
            return
        }
    }
}
```

Листинг 24 – исходный код функции обработчика, публикующего метрики модулей

5.6.4. Описание обработчиков сообщений топиков, на которые подписаны модули ГПС

В этом пункте речь пойдет про алгоритмы обработки команд, поступающих от внешнего клиента в адрес того или иного модуля.

В проекте реализована функциональная прослойка, главная задача которой – принять опубликованное в определенный топик сообщение, извлечь из него всю необходимую информацию, в том числе и элементы, предоставленные в промежуточных функциях-middleware, провалидировать содержимое сообщения при наличии и вызвать методы соответствующих сервисов из следующего внутреннего уровня **service**.

После получения результатов работы внутренней логики приложения, описываемые функции возвращают ответ со статусом «ok» или «error», о формате которых будет сказано в следующем пункте. Полный исходный код обработчиков представлен в приложениях 11-16.

```
type HandlerMQTT struct {
    Service conveyor.Service
    log     *zap.SugaredLogger
}
func NewHandlerMQTT(service conveyor.Service, log
*zap.SugaredLogger) *HandlerMQTT {
    return &HandlerMQTT{
        Service: service,
        log:     log,
```

```
    }
}
func (h *HandlerMQTT) ProvideWorkpieceToRecognition(client
mqtt.Client, msg mqtt.Message) {
    ctx, err := deliveryMQTT.GetContextFromMessage(msg)
    reqTopic, respTopic := msg.Topic(),
requestResponseTopics[msg.Topic()]

    if err != nil {
        deliveryMQTT.ResponseError(h.log, client, reqTopic,
respTopic, 1, false, err)
        return
    }
    if err = h.Service.MoveWorkpieceToRecognition(ctx); err !=
nil {
        deliveryMQTT.ResponseError(h.log, client, reqTopic,
respTopic, 1, false, err)
    } else if err = deliveryMQTT.ResponseOk(h.log, client,
reqTopic, respTopic, 1, false, "conveyor provided workpiece to
the recognition successfully", nil); err != nil {
        deliveryMQTT.ResponseError(h.log, client, reqTopic,
respTopic, 1, false, err)
    }
}
```

Листинг 25 – часть исходного кода обработчиков уровня delivery для Конвейера

Важно упомянуть установленные правила маршрутизации: для каждого модуля определены специальные топики для принятия команд, публикации ответа на соответствующую команду и публикации метрик. Полный исходный код представлен в приложениях 11-16.

```
var (
    requestCmdTopicPrefix  = "/conveyor/cmd/request"
    responseCmdTopicPrefix = "/conveyor/cmd/response"
)
var (
```

```
    provideToRecReq  = requestCmdTopicPrefix + "/recognition"
    provideToRecResp = responseCmdTopicPrefix + "/recognition"

    provideToLatheReq  = requestCmdTopicPrefix + "/lathe"
    provideToLatheResp = responseCmdTopicPrefix + "/lathe"

    provideToMillerReq  = requestCmdTopicPrefix + "/miller"
    provideToMillerResp = responseCmdTopicPrefix + "/miller"

    pushMetrics = "/conveyor/metrics"

    requestResponseTopics = map[string]string{
        provideToRecReq:    provideToRecResp,
        provideToLatheReq:  provideToLatheResp,
        provideToMillerReq: provideToMillerResp,
    }
)
```

Листинг 26 – демонстрация маршрутизации уровня delivery для Конвейера

Для каждого модуля устанавливаются служебные обработчики по умолчанию, о которых шла речь в пункте про инициализацию MQTT опций. Также в соответствие указанным выше маршрутам (топикам) ставятся описанные выше функции обработчики, к которым применены промежуточные функции-middleware из соответствующего пункта. Полный исходный код представен в приложениях 11-16.

```
func SetSubscribeRouter(subCtx context.Context, client
*deliveryMQTT.ClientMQTT, handler *HandlerMQTT) []mqtt.Token {
    tokens := make([]mqtt.Token, 0, 1)

    tokens = append(tokens, client.Subscribe(hanldeReq, 1,
deliveryMQTT.ApplyMiddlewareStack(
        handler.HandleWorkpiece,
        deliveryMQTT.LoggingMiddleware,
        deliveryMQTT.CheckMsgWithCtxMiddleware,
        deliveryMQTT.ReplaceMessageClientMiddleware(subCtx,
```

```
client),
        deliveryMQTT.RecoverMiddleware,
    )))

    return tokens
}
```

Листинг 27 – демонстрация установки обработчиков для каждого топика и применения к обработчикам функций-middleware (Фрезерный станок)

```
var (
    OnConnect mqtt.OnConnectHandler = func(client mqtt.Client) {
        log.Println("on connect recognition: successfully")
    }
    OnConnectionLost mqtt.ConnectionLostHandler = func(client
mqtt.Client, err error) {
        log.Printf("on connection lost recognition: %s\n",
err.Error())
    }
    OnReconnect mqtt.ReconnectHandler = func(client mqtt.Client,
_ *mqtt.ClientOptions) {
        log.Println("on reconnect recognition: trying to
reconnect")
    }
    ConnAtempt mqtt.ConnectionAttemptHandler = func(broker
*url.URL, tlsCfg *tls.Config) *tls.Config {
        log.Printf("conntAtempt recognition: broker address -
%q\n", broker.String())
        return nil
    }
)
```

Листинг 28 – демонстрация установки служебных обработчиков по умолчанию для Модуля Распознавания

5.6.5. Описание формата ответов внешнему клиенту, метода сериализации полезной нагрузки

В качестве варианта сериализации сообщений с ответом выбран JSON (JavaScript Object Notation) – текстовый формат обмена данными, основанный на JavaScript. Один из наиболее распространенных форматов, легко читается людьми.

Определены следующие форматы ответов:

1. Ответ, сообщающий об успешном завершении операции, содержит:

- Уникальный идентификатор ответа
- Исходный топик, в ответ на который отправляетсясообщение
- Статус ответа («ok»)
- Сообщение
- Полезная нагрузка

```
type JSONResponse struct {
    RespUUID     string `json:"responseID"`
    RequestTopic string `json:"request_topic"`
    Status       string `json:"status"`
    Message      string `json:"message"`
    Payload      any    `json:"payload"`
}
```

Листинг 29 – демонстрация формата сообщений со статусом «ok»

2. Ответ, сообщающий о какой-либо ошибке в результате выполнения операции, содержит:

- Уникальный идентификатор ответа
- Исходный топик, в ответ на который отправляется сообщение
- Статус ответа («error»)
- Сообщение
- Код ошибки

```
• type JSONErrResponse struct {
•      RespUUID     string `json:"responseID"`
•      RequestTopic string `json:"request_topic"`
•      Status       string `json:"status"`
•      Message      string `json:"message"`
```

```
•       Code          string `json:"code"`
•  }
```

Листинг 30 – демонстрация формата сообщений со статусом «error»

Полный исходный код представлен в приложении 20.

6. Апробирование

6.1. Понятие интеграционного тестирования

Интеграционное тестирование в разработке программного обеспечения — это процесс тестирования, в ходе которого отдельные модули или компоненты системы объединяются и тестируются как группа. Цель интеграционного тестирования — выявить любые ошибки в взаимодействии между интегрированными компонентами.

Основные аспекты интеграционного тестирования:

- Тестирование взаимодействия - интеграционное тестирование фокусируется на взаимодействиях между компонентами или системами.
- Выявление ошибок интеграции - основная цель — выявление проблем, которые могут возникнуть при интеграции различных частей системы, включая проблемы с интерфейсами, сетевыми взаимодействиями, базами данных и другими внешними компонентами или системами.
- Тестовые сценарии - тестовые сценарии обычно включают проверку потоков данных, контроля процессов, обработки ошибок и бизнес-логики.

6.2. Подготовка среды для проведения интеграционного тестирования программного кода приложения

Для проведения интеграционных тестов был написан программный код, эмулирующий операции реального клиента-оператора ГПС. В коде создается MQTT клиент, устанавливается подключение к платформе Yandex IoT Core.

Для клиента описываются наборы пар типа «топик-обработчик», где каждому топику, на который подпишется клиент соответствует определенный обработчик. Согласно сценария тестирования клиент подписывается на топики, в которые модули ГПС публикуют свои метрики, а также на топики, куда публикуются результаты выполнения

команд. Исходный код клиента для тестирования представлен в приложении 27.

```
subs := []struct {
        topic   string
        handler mqtt.MessageHandler
    }{
        {
            topic: "/storage/metrics",
            handler: func(c mqtt.Client, m mqtt.Message) {
                fmt.Println("Got storage metrics:")
                fmt.Println(string(m.Payload()))
            },
        },
        {
            topic: "/conveyor/metrics",
            handler: func(c mqtt.Client, m mqtt.Message) {
                fmt.Println("Got conveyor metrics:")
                fmt.Println(string(m.Payload()))
            },
        },
    }
```

Листинг 31 – часть топиков, на которые подписывается тестовый клиент

Далее объявляется список топиков команд, куда клиент будет публиковать сообщения в определенной последовательности для проверки работы модулей ГПС. Сообщения отправляются последовательно в рамках цикла for.

```
pubs := []string{
        "/storage/cmd/request/provide",
        "/conveyor/cmd/request/lathe",
        "/lathe/cmd/request/handle",
        "/conveyor/cmd/request/miller",
        "/miller/cmd/request/handle",
        "/conveyor/cmd/request/recognition",
        "/recognition/cmd/request/recognize",
```

```
            "/storage/cmd/request/accept",
        }
```

Листинг 32 – пример списка топиков команд, в которые клиент публикует сообщения

```
    for _, subCase := range subs {
            if token := client.Subscribe(subCase.topic, 1,
    subCase.handler); token.Wait() && token.Error() != nil {
                log.Fatal(token.Error())
            }
        }

        for _, topic := range pubs {
            time.Sleep(6 * time.Second)
            if token := client.Publish(topic, 1, false, "");
    token.Wait() && token.Error() != nil {
                log.Fatal(token.Error())
            }
        }
```

Листинг 32 – цикл for, в рамках которого сообщения публикуются в определенном порядке

6.3. Сценарий работы и результаты интеграционного тестирования

Рассмотрим следующий сценарий работы:

1. В наличии 1 заготовка
2. Складу поступает команда положить заготовку на конвейер
3. Конвейеру поступает команда доставить заготовку к токарному станку
4. Токарному станку поступает команда обработать заготовку
5. Конвейеру поступает команда доставить заготовку к фрезерному станку
6. Фрезерному станку поступает команда обработать заготовку

7. Конвейеру поступает команда доставить изделие к модулю распознавания
8. Модулю распознавания поступает команда распознать изделие
9. Складу поступает команда принять изделие

Результаты работы представлены в виде логов сервиса в приложениях 28-29

Судя по логам, программа запустилась в штатном режиме, началась периодическая публикация метрик, все команды были получены и обработаны, в результате произошло успешное завершение работы.

В результате все метрики были получены, модули ГПС корректно обработали поступившие команды. Когда все имеющиеся заготовки уже были обработаны, модули ГПС выдавали характерные сообщения с ошибками.

```
...
...
Got response from conveyor on moving workpiece to the lathe:
{"responseID":"e10fd9db-3c36-46bb-8e28-
9e5c38ba4375","request_topic":"/conveyor/cmd/request/lathe","statu
s":"error","message":"workpiece is not on the
conveyor","code":"invalid_cmd_state"}
...
...
Got response from lathe on handling the workpiece:
{"responseID":"ec8e0918-ab09-4fe3-bb55-
fc02ee54705a","request_topic":"/lathe/cmd/request/handle","status"
:"error","message":"There is no workpiece for lathe to
handle","code":"invalid_cmd_state"}
...
...
Got response from conveyor on moving workpiece to the miller:
{"responseID":"ec8e0918-ab09-4fe3-bb55-
fc02ee54705a","request_topic":"/lathe/cmd/request/handle","status"
:"error","message":"There is no workpiece for lathe to
```

```
handle","code":"invalid_cmd_state"}
...
...
Got response from miller on handling the workpiece:
{"responseID":"5cde4c98-5031-4913-aa27-
56cf932ed154","request_topic":"/miller/cmd/request/handle","status
":"error","message":"There is no workpiece for miller to
handle","code":"invalid_cmd_state"}
...
...
Got response from conveyor on moving workpiece to the recognition:
{"responseID":"6b657865-b868-4b8a-b27a-
d3421a621293","request_topic":"/conveyor/cmd/request/recognition",
"status":"error","message":"workpiece is not on the
conveyor","code":"invalid_cmd_state"}
...
...
```

Листинг 35 – логи на стороне клиента после того, как все заготовки были обработаны

ЗАКЛЮЧЕНИЕ

В ходе выполнения ВКР были выполнены следующие задачи:

1. Была проанализирована существующая система, в результате были выявлены некоторые недостатки.
2. Произведен анализ сетевых протоколов, выбор наиболее оптимального решения, удовлетворяющего всем требованиям, - протокола MQTT.
3. Произведен выбор платформы для реализации проекта, было принято решение проанализировать существующие облачные технологии. Основываясь на финансовых и географических факторах, выбор был сделан в пользу Yandex Cloud.
4. Разработан алгоритм взаимодействия модулей ГПС с клиентом.
5. В ходе исследования вопроса распределенных и централизованных систем было принято решение утвердить централизованную систему в качестве дизайна системы, концепцию коммуникации клиента и ГПС реализовать, основываясь на принципах многопоточного программирования.
6. Разработан алгоритм внутренней коммуникации элементов ГПС.
7. Произведена настройка облачного сервиса Yandex IoT Core: были выписаны сертификаты формата x.509, был создан MQTT брокер.
8. В процессе исследования наиболее часто используемых языков программирования было принято решение выбрать Golang в качестве инструмента разработки.
9. Была организована общая структура проекта, соответствующая современным стандартам Go разработки.
10. В ходе изучения понятия «Чистая архитектура» Роберта С. Мартина было принято решение организовать внутреннюю структуру проекта согласно данной концепции для обеспечения гибкости и удобства разработки.

11. Реализован уровень ключевых доменных сущностей приложения – «Склад», «Конвейер», «Токарный станок», «Фрезерный станок», «Модуль распознавания».
12. Разработан уровень сервисов, инкапсулирующий основные сценарии работы приложения.
13. Реализован уровень работы с сетевым протоколом MQTT.
14. Проведено интеграционное тестирование приложения.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГОСТ 34.602-89. Техническое задание на создание автоматизированной системы.
2. Житковский Е. А., Ельников Е. П., Тонко И. А. Использование протокола MQTT в телкоммуникационных системах //Новые информационные технологии в научных исследованиях. – 2019. – С. 364-366.
3. Кришнамурти Б., Рексфорд Д. Web-протоколы. Теория и практика //М.: БИНОМ. – 2002. – С. 58-61.
4. Овсянников М.В., Буханов С. А., Подкопаев С. А. Организация индустриального интернета на основе SAP HANA CLOUD PLATFORM // М.: Издательство МГТУ им. Н. Э. Баумана. – 2019. – С. 6-10.
5. Полуэктова Н. Разработка веб-приложений. Учебное пособие для СПО. – Litres, 2021.
6. Пошаговые инструкции Yandex.Cloud – Документация [Электронный ресурс] // Cloud.Yandex [сайт]. (2023) URL: https://cloud.yandex.ru/docs/iotcore/operations/publish (дата обращения 30.12.2023)
7. Селезнёв С. П., Яковлев В. В. Архитектура промышленных приложений IoT и протоколы amqp, mqtt, jms, rest, CoAP, xmpp, dds //International Journal of Open Information Technologies. – 2019. – Vol. 7. – №. 5.
8. Таненбаум Э. и др. Распределенные системы. Принципы и парадигмы. – 2003.
9. Роберт Мартин Чистая архитектура. Искусство разработки программного обеспечения. – 2018.
10. Алан А.А. Донован, Брайан Керниган – The Go Programming Language (1st edition) – 2015.
11. Мэтью А. Титмус - "Cloud Native Go: Building Web Applications and Microservices for the Cloud with Go and React"
12. Таненбаум Э. и др. Компьютерные сети. – 2005.

13. Адам Вудбек - "Network Programming with Go" – 2016.
14. Уильям Кеннеди, Брайн Кетелсон - "Go in Action"- 2012.
15. Томас Кормен - Алгоритмы: построение и анализ – 2000.
16. Официальная документация протокола MQTT – 2014
17. Тейва Харшани – «100 ошибок GO и как их избежать» - 2020
18. Newham C., Rosenblatt B. Learning the bash shell: Unix shell programming. – O'Reilly Media, Inc., 2005.

ПРИЛОЖЕНИЕ А

Программный код с общим описанием доменной сущности «Склад», конкретной реализацией сущности и характерными сервисными ошибками

```
package storage

import (
    "context"
)

type Metrics struct {
    Ready             bool
    Workpieces        int
    RawWorkpieces     int
    HandledWorkpieces int
}

type Storage interface {
    IsReady(context.Context) error
    ProvideRawWorkpiece(context.Context) error
    AcceptFinishedWorkpiece(context.Context) error
    Metrics(context.Context) (*Metrics, error)
}

package storage

import (
    "context"
    "math/rand"
    "sync"

    "github.com/wonderf00l/fms-control-system/internal/errors"
)

var _ Storage = (*mockStorage)(nil)
```

```
var (
    maxWorkpieceNum   = 1
    rawWorkpieces     = 1
    handledWorkpieces = 0
)

type mockStorage struct {
    maxWorkpieceNum   int
    rawWorkpieces     int
    handledWorkpieces int

    mu sync.Mutex
}

func NewStorage() *mockStorage {
    return &mockStorage{
        maxWorkpieceNum:   maxWorkpieceNum,
        rawWorkpieces:     rawWorkpieces,
        handledWorkpieces: handledWorkpieces,
    }
}

func (s *mockStorage) IsReady(ctx context.Context) error {
    switch rand.Intn(5000) {
    case 0:
        return &errors.TimeoutExceededError{Service:
errors.Storage}
    default:
        return nil
    }
}

func (s *mockStorage) ProvideRawWorkpiece(ctx context.Context)
error {
    s.mu.Lock()
```

```
    defer s.mu.Unlock()
    if s.handledWorkpieces+s.rawWorkpieces < s.maxWorkpieceNum {
        return &workpieceAlreadyProcessingError{}
    }
    if s.handledWorkpieces == s.maxWorkpieceNum {
        return &allWorkpiecesHandledError{}
    }
    s.rawWorkpieces--
    return nil
}

func (s *mockStorage) AcceptFinishedWorkpiece(ctx context.Context)
error {
    s.mu.Lock()
    defer s.mu.Unlock()
    if s.handledWorkpieces == s.maxWorkpieceNum {
        return &allWorkpiecesHandledError{}
    }
    if s.rawWorkpieces+s.handledWorkpieces == s.maxWorkpieceNum {
        return &noWorkpieceProcessingError{}
    }
    s.handledWorkpieces++
    return nil
}

func (s *mockStorage) Metrics(ctx context.Context) (*Metrics,
error) {
    err := s.IsReady(ctx)
    return &Metrics{
        Ready:             err == nil,
        Workpieces:        s.maxWorkpieceNum,
        RawWorkpieces:     s.rawWorkpieces,
        HandledWorkpieces: s.handledWorkpieces,
    }, nil
}
```

```
package storage

import (
    "fmt"

    "github.com/wonderf00l/fms-control-system/internal/errors"
)

var (
    errPreifx = "Storage"
)

type allWorkpiecesHandledError struct{}

func (e *allWorkpiecesHandledError) Error() string {
    return fmt.Sprintf("%s: all workpieces are already handled",
errPreifx)
}

func (e *allWorkpiecesHandledError) Type() errors.ErrorType {
    return errors.ErrInavlidStateForCMD
}

type workpieceAlreadyProcessingError struct{}

func (e *workpieceAlreadyProcessingError) Error() string {
    return fmt.Sprintf("%s: workpiece is already processing",
errPreifx)
}

func (e *workpieceAlreadyProcessingError) Type() errors.ErrorType
{
    return errors.ErrInavlidStateForCMD
}

type noWorkpieceProcessingError struct{}
```

```
func (e *noWorkpieceProcessingError) Error() string {
    return fmt.Sprintf("%s: no workpieces processing", errPreifx)
}

func (e *noWorkpieceProcessingError) Type() errors.ErrorType {
    return errors.ErrInavlidStateForCMD
}
```

ПРИЛОЖЕНИЕ Б

Программный код с общим описанием доменной сущности «Конвейер», конкретной реализацией сущности и характерными сервисными ошибками

```
package conveyor

import "context"

type Metrics struct {
    Ready             bool
    WorkpieceLocation any
}

type WorkpieceLocation struct {
    X int
    Y int
}

type Conveyor interface {
    IsReady(context.Context) (*WorkpieceLocation, error)
    SetWorkpieceLocation(context.Context, WorkpieceLocation, bool)
error
    Metrics(context.Context) (*Metrics, error)
}
package conveyor

import (
    "context"
    "math/rand"
    "sync"

    "github.com/wonderf00l/fms-control-system/internal/errors"
)
```

```
var _ Conveyor = (*mockConveyor)(nil)

var (
    maxX = 50
    maxY = 50
)

type mockConveyor struct {
    workpieceLocation WorkpieceLocation

    mu sync.Mutex
}

func NewConveyor() *mockConveyor {
    return &mockConveyor{}
}

func (c *mockConveyor) IsReady(ctx context.Context)
(*WorkpieceLocation, error) {
    switch rand.Intn(5000) {
    case 0:
        return nil, &errors.TimeoutExceededError{Service:
errors.Conveyor}
    default:
        return &c.workpieceLocation, nil
    }
}

func (c *mockConveyor) SetWorkpieceLocation(ctx context.Context,
location WorkpieceLocation, fromStorage bool) error {
    c.mu.Lock()
    defer c.mu.Unlock()
    if !fromStorage && c.workpieceLocation.X == 0 &&
c.workpieceLocation.Y == 0 {
        return &workpieceNotOnConveyorError{}
    }
```

```
    if location.X > maxX || location.X < 0 || location.Y > maxY ||
location.Y < 0 {
        return &invalidLocationError{location: location}
    }
    c.workpieceLocation.X = location.X
    c.workpieceLocation.Y = location.Y
    return nil
}

func (c *mockConveyor) Metrics(ctx context.Context) (*Metrics,
error) {
    location, err := c.IsReady(ctx)
    return &Metrics{
        Ready:             err == nil,
        WorkpieceLocation: location,
    }, nil
}
```

ПРИЛОЖЕНИЕ В

Программный код с общим описанием доменной сущности «Токарный станок», конкретной реализацией сущности

```
package lathe

import "context"

type Metrics struct {
    Ready bool
}

type Lathe interface {
    IsReady(context.Context) error
    HandleWorkpiece(context.Context) error
    Metrics(context.Context) (*Metrics, error)
}
package lathe

import (
    "context"
    "math/rand"

    "github.com/wonderf00l/fms-control-system/internal/errors"
)

var _ Lathe = (*mockLathe)(nil)

type mockLathe struct{}

func NewLathe() *mockLathe {
    return &mockLathe{}
}

func (l *mockLathe) IsReady(ctx context.Context) error {
```

```
    switch rand.Intn(5000) {
    case 0:
        return &errors.TimeoutExceededError{Service: errors.Lathe}
    default:
        return nil
    }
}

func (l *mockLathe) HandleWorkpiece(context.Context) error {
    return nil
}

func (l *mockLathe) Metrics(ctx context.Context) (*Metrics, error)
{
    return &Metrics{
        Ready: l.IsReady(ctx) == nil,
    }, nil
}
```

ПРИЛОЖЕНИЕ Г

Программный код с общим описанием доменной сущности «Фрезерный станок», конкретной реализацией сущности

```
package miller

import "context"

type Metrics struct {
    Ready bool
}

type Miller interface {
    IsReady(context.Context) error
    HandleWorkpiece(context.Context) error
    Metrics(context.Context) (*Metrics, error)
}

package miller

import (
    "context"
    "math/rand"

    "github.com/wonderf00l/fms-control-system/internal/errors"
)

var _ Miller = (*mockMiller)(nil)

type mockMiller struct{}

func NewMiller() *mockMiller {
    return &mockMiller{}
}
```

```
func (m *mockMiller) IsReady(ctx context.Context) error {
    switch rand.Intn(5000) {
    case 0:
        return &errors.TimeoutExceededError{Service:
errors.Miller}
    default:
        return nil
    }
}

func (m *mockMiller) HandleWorkpiece(ctx context.Context) error {
    return nil
}

func (m *mockMiller) Metrics(ctx context.Context) (*Metrics,
error) {
    return &Metrics{
        Ready: m.IsReady(ctx) == nil,
    }, nil
}
```

ПРИЛОЖЕНИЕ Д

Программный код с общим описанием доменной сущности «Модуль распознавания», конкретной реализацией сущности

```
package recognition

import "context"

type WorkpieceType uint8

const (
    Unknown WorkpieceType = iota
    King
    Queen
    Bishop
    Knight
    Castle
    Pawn
)

type Metrics struct {
    Ready bool
}

type Recognition interface {
    IsReady(context.Context) error
    Recognize(context.Context) (WorkpieceType, error)
    Metrics(context.Context) (*Metrics, error)
}


package recognition

import (
```

```
    "context"
    "math/rand"

    "github.com/wonderf00l/fms-control-system/internal/errors"
)

var _ Recognition = (*mockRecognition)(nil)

type mockRecognition struct{}

func NewRecognition() *mockRecognition {
    return &mockRecognition{}
}

func (r *mockRecognition) IsReady(context.Context) error {
    switch rand.Intn(5000) {
    case 0:
        return &errors.TimeoutExceededError{Service:
errors.Recognition}
    default:
        return nil
    }
}

func (r *mockRecognition) Recognize(ctx context.Context)
(WorkpieceType, error) {
    return WorkpieceType(2), nil
}

func (r *mockRecognition) Metrics(ctx context.Context) (*Metrics,
error) {
    return &Metrics{
        Ready: r.IsReady(ctx) == nil,
    }, nil
}
```

ПРИЛОЖЕНИЕ Е

Программный код со структурой, содержащей все доменные сущности. Необходима на уровне Service.

```
package entity

import (
    "github.com/wonderf00l/fms-control-
system/internal/entity/conveyor"
    "github.com/wonderf00l/fms-control-
system/internal/entity/lathe"
    "github.com/wonderf00l/fms-control-
system/internal/entity/miller"
    "github.com/wonderf00l/fms-control-
system/internal/entity/recognition"
    "github.com/wonderf00l/fms-control-
system/internal/entity/storage"
)
type Pool struct {
    Storage     storage.Storage
    Conveyor    conveyor.Conveyor
    Recognition recognition.Recognition
    Lathe       lathe.Lathe
    Miller      miller.Miller
}
```

ПРИЛОЖЕНИЕ Ж

Программный код сервиса «Склад»

```
package storage

import (
    "context"

    "github.com/wonderf00l/fms-control-system/internal/entity"
    "github.com/wonderf00l/fms-control-
system/internal/entity/conveyor"
    storage "github.com/wonderf00l/fms-control-
system/internal/entity/storage"
    "github.com/wonderf00l/fms-control-system/internal/errors"
    "go.uber.org/zap"
)
var _ Service = (*service)(nil)
type Service interface {
    ProvideWorkpiece(context.Context) error
    AcceptWorkpiece(context.Context) error
    GetMetrics(context.Context) (*storage.Metrics, error)
}
type service struct {
    log  *zap.SugaredLogger
    pool *entity.Pool
}
func NewService(log *zap.SugaredLogger, pool *entity.Pool)
*service {
    return &service{log: log, pool: pool}
}

func (s *service) ProvideWorkpiece(ctx context.Context) error {
    if err := s.pool.Storage.IsReady(ctx); err != nil {
        return err
    }
```

```
    if _, err := s.pool.Conveyor.IsReady(ctx); err != nil {
        return &errors.DepsNotReadyError{Service: errors.Storage,
Dependency: errors.Conveyor, Reason: err}
    }
    if err := s.pool.Storage.ProvideRawWorkpiece(ctx); err != nil
{
        return err
    }
    if err := s.pool.Conveyor.SetWorkpieceLocation(ctx,
conveyor.WorkpieceLocation{X: 1, Y: 0}, true); err != nil {
        return err
    }
    return nil
}
func (s *service) AcceptWorkpiece(ctx context.Context) error {
    if err := s.pool.Storage.IsReady(ctx); err != nil {
        return err
    }
    if _, err := s.pool.Conveyor.IsReady(ctx); err != nil {
        return &errors.DepsNotReadyError{Service: errors.Storage,
Dependency: errors.Conveyor, Reason: err}
    }
    if err := s.pool.Storage.AcceptFinishedWorkpiece(ctx); err !=
nil {
        return err
    }
    if err := s.pool.Conveyor.SetWorkpieceLocation(ctx,
conveyor.WorkpieceLocation{X: 0, Y: 0}, true); err != nil {
        return err
    }
    return nil
}

func (s *service) GetMetrics(ctx context.Context)
(*storage.Metrics, error) {
    return s.pool.Storage.Metrics(ctx)
```

```
}
```

ПРИЛОЖЕНИЕ 3

Программный код сервиса «Конвейер» с координатами заготовки для всех модулей ГПС

```
package conveyor

import (
    "context"

    "github.com/wonderf00l/fms-control-system/internal/entity"
    "github.com/wonderf00l/fms-control-
system/internal/entity/conveyor"
    "github.com/wonderf00l/fms-control-system/internal/errors"
    "go.uber.org/zap"
)

var _ Service = (*service)(nil)

type Service interface {
    MoveWorkpieceToLathe(context.Context) error
    MoveWorkpieceToMiller(context.Context) error
    MoveWorkpieceToRecognition(context.Context) error
    MoveWorkpieceToStorage(context.Context) error
    GetMetrics(context.Context) (*conveyor.Metrics, error)
}

type service struct {
    log  *zap.SugaredLogger
    pool *entity.Pool
}

func NewService(log *zap.SugaredLogger, pool *entity.Pool)
*service {
    return &service{log: log, pool: pool}
}
```

```
func (c *service) MoveWorkpieceToRecognition(ctx context.Context)
error {
    if _, err := c.pool.Conveyor.IsReady(ctx); err != nil {
        return err
    }
    if err := c.pool.Recognition.IsReady(ctx); err != nil {
        return &errors.DepsNotReadyError{Service: errors.Conveyor,
Dependency: errors.Recognition, Reason: err}
    }
    return c.pool.Conveyor.SetWorkpieceLocation(ctx,
conveyor.WorkpieceLocation{
        X: locationForRecognitionX,
        Y: locationForRecognitionY,
    }, false)
}

func (c *service) MoveWorkpieceToLathe(ctx context.Context) error
{
    if _, err := c.pool.Conveyor.IsReady(ctx); err != nil {
        return err
    }
    if err := c.pool.Lathe.IsReady(ctx); err != nil {
        return &errors.DepsNotReadyError{Service: errors.Conveyor,
Dependency: errors.Lathe, Reason: err}
    }
    return c.pool.Conveyor.SetWorkpieceLocation(ctx,
conveyor.WorkpieceLocation{
        X: locationForLatheX,
        Y: locationForLatheY,
    }, false)
}

func (c *service) MoveWorkpieceToMiller(ctx context.Context) error
{
    if _, err := c.pool.Conveyor.IsReady(ctx); err != nil {
```

```
        return err
    }
    if err := c.pool.Miller.IsReady(ctx); err != nil {
        return &errors.DepsNotReadyError{Service: errors.Conveyor,
Dependency: errors.Miller, Reason: err}
    }
    return c.pool.Conveyor.SetWorkpieceLocation(ctx,
conveyor.WorkpieceLocation{
        X: locationForMillerX,
        Y: locationForMillerY,
    }, false)
}

func (c *service) MoveWorkpieceToStorage(ctx context.Context)
error {
    if _, err := c.pool.Conveyor.IsReady(ctx); err != nil {
        return err
    }
    if err := c.pool.Storage.IsReady(ctx); err != nil {
        return &errors.DepsNotReadyError{Service: errors.Conveyor,
Dependency: errors.Storage, Reason: err}
    }
    return c.pool.Conveyor.SetWorkpieceLocation(ctx,
conveyor.WorkpieceLocation{
        X: locationForStorageX,
        Y: locationForStorageY,
    }, false)
}

func (c *service) GetMetrics(ctx context.Context)
(*conveyor.Metrics, error) {
    return c.pool.Conveyor.Metrics(ctx)
}
package conveyor

var (
```

```
    locationForRecognitionX = 0
    locationForRecognitionY = 25

    locationForLatheX = 50
    locationForLatheY = 50

    locationForMillerX = 0
    locationForMillerY = 50

    locationForStorageX = 0
    locationForStorageY = 0
)
```

ПРИЛОЖЕНИЕ И

Программный код сервиса «Токарный станок» с координатами заготовки для данного модуля ГПС, сервисными ошибками

```
package lathe

import (
    "context"

    "github.com/wonderf00l/fms-control-system/internal/entity"
    "github.com/wonderf00l/fms-control-
system/internal/entity/lathe"
    "go.uber.org/zap"
)

var _ Service = (*service)(nil)

type Service interface {
    HandleWorkpiece(context.Context) error
    GetMetrics(context.Context) (*lathe.Metrics, error)
}

type service struct {
    log  *zap.SugaredLogger
    pool *entity.Pool
}

func NewService(log *zap.SugaredLogger, pool *entity.Pool)
*service {
    return &service{log: log, pool: pool}
}

func (m *service) HandleWorkpiece(ctx context.Context) error {
    if err := m.pool.Lathe.IsReady(ctx); err != nil {
        return err
```

```
    }

    location, err := m.pool.Conveyor.IsReady(ctx)
    if err != nil {
        return err
    }
    if location.X != locationForLatheX || location.Y !=
locationForLatheY {
        return &noWorkpieceForLatheToHandleError{}
    }
    return m.pool.Lathe.HandleWorkpiece(ctx)
}
func (m *service) GetMetrics(ctx context.Context) (*lathe.Metrics,
error) {
    return m.pool.Lathe.Metrics(ctx)
}
package lathe

var (
    locationForLatheX = 50
    locationForLatheY = 50
)

package lathe

import "github.com/wonderf00l/fms-control-system/internal/errors"

type noWorkpieceForLatheToHandleError struct{}

func (e *noWorkpieceForLatheToHandleError) Error() string {
    return "There is no workpiece for lathe to handle"
}
func (e *noWorkpieceForLatheToHandleError) Type() errors.ErrorType
{
    return errors.ErrInavlidStateForCMD
}
```

ПРИЛОЖЕНИЕ К

Программный код сервиса «Фрезерный станок» с координатами заготовки для данного модуля ГПС, сервисными ошибками

```
package miller

import (
    "context"

    "github.com/wonderf00l/fms-control-system/internal/entity"
    "github.com/wonderf00l/fms-control-
system/internal/entity/miller"
    "go.uber.org/zap"
)

var _ Service = (*service)(nil)

type Service interface {
    HandleWorkpiece(context.Context) error
    GetMetrics(context.Context) (*miller.Metrics, error)
}

type service struct {
    log  *zap.SugaredLogger
    pool *entity.Pool
}

func NewService(log *zap.SugaredLogger, pool *entity.Pool)
*service {
    return &service{log: log, pool: pool}
}

func (m *service) HandleWorkpiece(ctx context.Context) error {
    if err := m.pool.Miller.IsReady(ctx); err != nil {
        return err
```

```
    }

    location, err := m.pool.Conveyor.IsReady(ctx)
    if err != nil {
        return err
    }
    if location.X != locationForMillerX || location.Y !=
locationForMillerY {
        return &noWorkpieceForMillerToHandleError{}
    }

    return m.pool.Miller.HandleWorkpiece(ctx)
}
func (m *service) GetMetrics(ctx context.Context)
(*miller.Metrics, error) {
    return m.pool.Miller.Metrics(ctx)
}

package miller
var (
    locationForMillerX = 0
    locationForMillerY = 50
)

package miller

import "github.com/wonderf00l/fms-control-system/internal/errors"

type noWorkpieceForMillerToHandleError struct{}
func (e *noWorkpieceForMillerToHandleError) Error() string {
    return "There is no workpiece for miller to handle"
}
func (e *noWorkpieceForMillerToHandleError) Type()
errors.ErrorType {
    return errors.ErrInavlidStateForCMD
}
```

ПРИЛОЖЕНИЕ Л

Программный код сервиса «Модуль распознавания» с координатами заготовки для данного модуля ГПС, сервисными ошибками

```
package recognition

import (
    "context"

    "github.com/wonderf00l/fms-control-system/internal/entity"
    "github.com/wonderf00l/fms-control-
system/internal/entity/recognition"
    "go.uber.org/zap"
)

var _ Service = (*service)(nil)

type Service interface {
    RecognizeWorkpiece(context.Context)
(recognition.WorkpieceType, error)
    GetMetrics(context.Context) (*recognition.Metrics, error)
}

type service struct {
    log  *zap.SugaredLogger
    pool *entity.Pool
}

func NewService(log *zap.SugaredLogger, pool *entity.Pool)
*service {
    return &service{log: log, pool: pool}
}

func (r *service) RecognizeWorkpiece(ctx context.Context)
(recognition.WorkpieceType, error) {
    if err := r.pool.Recognition.IsReady(ctx); err != nil {
```

```
        return recognition.Unknown, err
    }

    location, err := r.pool.Conveyor.IsReady(ctx)
    if err != nil {
        return recognition.Unknown, err
    }

    if location.X != locationForRecognitionX || location.Y !=
locationForRecognitionY {
        return recognition.Unknown, &noWorkpieceToRecognizeError{}
    }

    return r.pool.Recognition.Recognize(ctx)
}
func (r *service) GetMetrics(ctx context.Context)
(*recognition.Metrics, error) {
    return r.pool.Recognition.Metrics(ctx)
}

package recognition
var (
    locationForRecognitionX = 0
    locationForRecognitionY = 25
)
package recognition

import "github.com/wonderf00l/fms-control-system/internal/errors"

type noWorkpieceToRecognizeError struct{}
func (e *noWorkpieceToRecognizeError) Error() string {
    return "There is no workpiece for recognition module"
}
func (e *noWorkpieceToRecognizeError) Type() errors.ErrorType {
    return errors.ErrInavlidStateForCMD
}
```

ПРИЛОЖЕНИЕ М

Программный код MQTT обработчика «Склад» со служебными MQTT обработчиками, маршрутизатором

```
package storage

import (
    "context"

    mqtt "github.com/eclipse/paho.mqtt.golang"
    deliveryMQTT "github.com/wonderf00l/fms-control-
system/internal/delivery/mqtt"
    "github.com/wonderf00l/fms-control-
system/internal/service/storage"
    "go.uber.org/zap"
)

type HandlerMQTT struct {
    Service storage.Service
    log     *zap.SugaredLogger
}

func NewHandlerMQTT(service storage.Service, log
*zap.SugaredLogger) *HandlerMQTT {
    return &HandlerMQTT{
        Service: service,
        log:     log,
    }
}

func (h *HandlerMQTT) ProvideWorkpiece(client mqtt.Client, msg
mqtt.Message) {
    ctx, err := deliveryMQTT.GetContextFromMessage(msg)
    reqTopic, respTopic := msg.Topic(),
requestResponseTopics[msg.Topic()]
```

```
    if err != nil {
        deliveryMQTT.ResponseError(h.log, client, reqTopic,
respTopic, 1, false, err)
        return
    }

    if err := h.Service.ProvideWorkpiece(ctx); err != nil {
        deliveryMQTT.ResponseError(h.log, client, reqTopic,
respTopic, 1, false, err)
    } else if err = deliveryMQTT.ResponseOk(h.log, client,
reqTopic, respTopic, 1, false, "storage has provided workpiece
successfully", nil); err != nil {
        deliveryMQTT.ResponseError(h.log, client, reqTopic,
respTopic, 1, false, err)
    }
}

func (h *HandlerMQTT) AcceptWorkpiece(client mqtt.Client, msg
mqtt.Message) {
    ctx, err := deliveryMQTT.GetContextFromMessage(msg)
    reqTopic, respTopic := msg.Topic(),
requestResponseTopics[msg.Topic()]

    if err != nil {
        deliveryMQTT.ResponseError(h.log, client, reqTopic,
respTopic, 1, false, err)
        return
    }

    if err := h.Service.AcceptWorkpiece(ctx); err != nil {
        deliveryMQTT.ResponseError(h.log, client, reqTopic,
respTopic, 1, false, err)
    } else if err = deliveryMQTT.ResponseOk(h.log, client,
reqTopic, respTopic, 1, false, "storage has accepted workpiece
successfully", nil); err != nil {
```

```
        deliveryMQTT.ResponseError(h.log, client, reqTopic,
respTopic, 1, false, err)
    }
}

func (h *HandlerMQTT) PushMetrics(ctx context.Context, client
mqtt.Client) {
    if metrics, err := h.Service.GetMetrics(ctx); err != nil {
        deliveryMQTT.ResponseError(h.log, client, *new(string),
pushMetrics, 1, false, err)
    } else if err = deliveryMQTT.ResponseOk(h.log, client,
*new(string), pushMetrics, 1, true, "got storage metrics
sucessfully", metrics); err != nil {
        deliveryMQTT.ResponseError(h.log, client, *new(string),
pushMetrics, 1, false, err)
    }
}
package storage

import (
    "context"

    mqtt "github.com/eclipse/paho.mqtt.golang"
    deliveryMQTT "github.com/wonderf00l/fms-control-
system/internal/delivery/mqtt"
)

var (
    requestCmdTopicPrefix  = "/storage/cmd/request"
    responseCmdTopicPrefix = "/storage/cmd/response"
)

var (
    provideWorkpieceReq  = requestCmdTopicPrefix + "/provide"
    provideWorkpieceResp = responseCmdTopicPrefix + "/provide"
```

```
    acceptWorkpieceReq  = requestCmdTopicPrefix + "/accept"
    acceptWorkpieceResp = responseCmdTopicPrefix + "/accept"

    pushMetrics = "/storage/metrics"

    requestResponseTopics = map[string]string{
        provideWorkpieceReq: provideWorkpieceResp,
        acceptWorkpieceReq:  acceptWorkpieceResp,
    }
)

func SetSubscribeRouter(subCtx context.Context, client
*deliveryMQTT.ClientMQTT, handler *HandlerMQTT) []mqtt.Token {
    tokens := make([]mqtt.Token, 0, 2)

    tokens = append(tokens,
        client.Subscribe(provideWorkpieceReq, 1,
deliveryMQTT.ApplyMiddlewareStack(
            handler.ProvideWorkpiece,
            deliveryMQTT.LoggingMiddleware,
            deliveryMQTT.CheckMsgWithCtxMiddleware,
            deliveryMQTT.ReplaceMessageClientMiddleware(subCtx,
client),
            deliveryMQTT.RecoverMiddleware),
        ),
        client.Subscribe(acceptWorkpieceReq, 1,
deliveryMQTT.ApplyMiddlewareStack(
            handler.AcceptWorkpiece,
            deliveryMQTT.LoggingMiddleware,
            deliveryMQTT.CheckMsgWithCtxMiddleware,
            deliveryMQTT.ReplaceMessageClientMiddleware(subCtx,
client),
            deliveryMQTT.RecoverMiddleware,
        )))

    return tokens
```

```
}
package storage

import (
    "crypto/tls"
    "log"
    "net/url"

    mqtt "github.com/eclipse/paho.mqtt.golang"
)

var (
    OnConnect mqtt.OnConnectHandler = func(client mqtt.Client) {
        log.Println("on connect storage: successfully")
    }
    OnConnectionLost mqtt.ConnectionLostHandler = func(client
mqtt.Client, err error) {
        log.Printf("on connection lost storage: %s\n",
err.Error())
    }
    OnReconnect mqtt.ReconnectHandler = func(client mqtt.Client, _
*mqtt.ClientOptions) {
        log.Println("on reconnect storage: trying to reconnect")
    }
    ConnAtempt mqtt.ConnectionAttemptHandler = func(broker
*url.URL, tlsCfg *tls.Config) *tls.Config {
        log.Printf("conntAtempt storage: broker address - %q\n",
broker.String())
        return nil
    }
)
```

ПРИЛОЖЕНИЕ Н

Программный код MQTT обработчика «Токарный станок» со служебными MQTT обработчиками, маршрутизатором

```
 package lathe

import (
    "context"

    mqtt "github.com/eclipse/paho.mqtt.golang"
    deliveryMQTT "github.com/wonderf00l/fms-control-
system/internal/delivery/mqtt"
    "github.com/wonderf00l/fms-control-
system/internal/service/lathe"
    "go.uber.org/zap"
)

type HandlerMQTT struct {
    Service lathe.Service
    log     *zap.SugaredLogger
}

func NewHandlerMQTT(service lathe.Service, log *zap.SugaredLogger)
*HandlerMQTT {
    return &HandlerMQTT{
        Service: service,
        log:     log,
    }
}

func (h *HandlerMQTT) HandleWorkpiece(client mqtt.Client, msg
mqtt.Message) {
    ctx, err := deliveryMQTT.GetContextFromMessage(msg)
    reqTopic, respTopic := msg.Topic(),
requestResponseTopics[msg.Topic()]
```

```
    if err != nil {
        deliveryMQTT.ResponseError(h.log, client, reqTopic,
respTopic, 1, false, err)
        return
    }

    if err = h.Service.HandleWorkpiece(ctx); err != nil {
        deliveryMQTT.ResponseError(h.log, client, reqTopic,
respTopic, 1, false, err)
    } else if err = deliveryMQTT.ResponseOk(h.log, client,
reqTopic, respTopic, 1, false, "lathe handled workpiece
sucessfully", nil); err != nil {
        deliveryMQTT.ResponseError(h.log, client, reqTopic,
respTopic, 1, false, err)
    }
}

func (h *HandlerMQTT) PushMetrics(ctx context.Context, client
mqtt.Client) {
    metrics, err := h.Service.GetMetrics(ctx)
    if err != nil {
        deliveryMQTT.ResponseError(h.log, client, *new(string),
pushMetrics, 1, false, err)
        return
    }
    if err = deliveryMQTT.ResponseOk(h.log, client, *new(string),
pushMetrics, 1, true, "got lathe metrics sucessfully", metrics);
err != nil {
        deliveryMQTT.ResponseError(h.log, client, *new(string),
pushMetrics, 1, false, err)
        return
    }
}

 package lathe
```

```
import (
    "context"

    mqtt "github.com/eclipse/paho.mqtt.golang"
    deliveryMQTT "github.com/wonderf00l/fms-control-
system/internal/delivery/mqtt"
)

var (
    requestCmdTopicPrefix  = "/lathe/cmd/request"
    responseCmdTopicPrefix = "/lathe/cmd/response"
)

var (
    hanldeReq  = requestCmdTopicPrefix + "/handle"
    handleResp = responseCmdTopicPrefix + "/handle"

    pushMetrics = "/lathe/metrics"

    requestResponseTopics = map[string]string{
        hanldeReq: handleResp,
    }
)

func SetSubscribeRouter(subCtx context.Context, client
*deliveryMQTT.ClientMQTT, handler *HandlerMQTT) []mqtt.Token {
    tokens := make([]mqtt.Token, 0, 1)

    tokens = append(tokens, client.Subscribe(hanldeReq, 1,
deliveryMQTT.ApplyMiddlewareStack(
        handler.HandleWorkpiece,
        deliveryMQTT.LoggingMiddleware,
        deliveryMQTT.CheckMsgWithCtxMiddleware,
        deliveryMQTT.ReplaceMessageClientMiddleware(subCtx,
client),
        deliveryMQTT.RecoverMiddleware,
```

```
    )))

    return tokens
}
package lathe

import (
    "crypto/tls"
    "log"
    "net/url"

    mqtt "github.com/eclipse/paho.mqtt.golang"
)

var (
    OnConnect mqtt.OnConnectHandler = func(client mqtt.Client) {
        log.Println("on connect lathe: successfully")
    }
    OnConnectionLost mqtt.ConnectionLostHandler = func(client
mqtt.Client, err error) {
        log.Printf("on connection lost lathe: %s\n", err.Error())
    }
    OnReconnect mqtt.ReconnectHandler = func(client mqtt.Client, _
*mqtt.ClientOptions) {
        log.Println("on reconnect lathe: trying to reconnect")
    }
    ConnAtempt mqtt.ConnectionAttemptHandler = func(broker
*url.URL, tlsCfg *tls.Config) *tls.Config {
        log.Printf("conntAtempt lathe: broker address - %q\n",
broker.String())
        return nil
    }
)
```

ПРИЛОЖЕНИЕ О

Программный код MQTT обработчика «Фрезерный станок» со служебными MQTT обработчиками, маршрутизатором

```
package miller

import (
    "context"

    mqtt "github.com/eclipse/paho.mqtt.golang"
    deliveryMQTT "github.com/wonderf00l/fms-control-
system/internal/delivery/mqtt"
    "github.com/wonderf00l/fms-control-
system/internal/service/miller"
    "go.uber.org/zap"
)

type HandlerMQTT struct {
    Service miller.Service
    log     *zap.SugaredLogger
}

func NewHandlerMQTT(service miller.Service, log
*zap.SugaredLogger) *HandlerMQTT {
    return &HandlerMQTT{
        Service: service,
        log:     log,
    }
}

func (h *HandlerMQTT) HandleWorkpiece(client mqtt.Client, msg
mqtt.Message) {
    ctx, err := deliveryMQTT.GetContextFromMessage(msg)
    reqTopic, respTopic := msg.Topic(),
requestResponseTopics[msg.Topic()]
```

```
    if err != nil {
        deliveryMQTT.ResponseError(h.log, client, reqTopic,
respTopic, 1, false, err)
        return
    }

    if err = h.Service.HandleWorkpiece(ctx); err != nil {
        deliveryMQTT.ResponseError(h.log, client, reqTopic,
respTopic, 1, false, err)
    } else if err = deliveryMQTT.ResponseOk(h.log, client,
reqTopic, respTopic, 1, false, "miller handled workpiece
successfully", nil); err != nil {
        deliveryMQTT.ResponseError(h.log, client, reqTopic,
respTopic, 1, false, err)
    }
}

func (h *HandlerMQTT) PushMetrics(ctx context.Context, client
mqtt.Client) {
    metrics, err := h.Service.GetMetrics(ctx)
    if err != nil {
        deliveryMQTT.ResponseError(h.log, client, *new(string),
pushMetrics, 1, false, err)
        return
    }
    if err = deliveryMQTT.ResponseOk(h.log, client, *new(string),
pushMetrics, 1, true, "got miller metrics sucessfully", metrics);
err != nil {
        deliveryMQTT.ResponseError(h.log, client, *new(string),
pushMetrics, 1, false, err)
        return
    }
}
package miller

import (
```

```
    "context"

    mqtt "github.com/eclipse/paho.mqtt.golang"
    deliveryMQTT "github.com/wonderf00l/fms-control-
system/internal/delivery/mqtt"
)

var (
    requestCmdTopicPrefix  = "/miller/cmd/request"
    responseCmdTopicPrefix = "/miller/cmd/response"
)

var (
    hanldeReq  = requestCmdTopicPrefix + "/handle"
    handleResp = responseCmdTopicPrefix + "/handle"

    pushMetrics = "/miller/metrics"

    requestResponseTopics = map[string]string{
        hanldeReq: handleResp,
    }
)

func SetSubscribeRouter(subCtx context.Context, client
*deliveryMQTT.ClientMQTT, handler *HandlerMQTT) []mqtt.Token {
    tokens := make([]mqtt.Token, 0, 1)

    tokens = append(tokens, client.Subscribe(hanldeReq, 1,
deliveryMQTT.ApplyMiddlewareStack(
        handler.HandleWorkpiece,
        deliveryMQTT.LoggingMiddleware,
        deliveryMQTT.CheckMsgWithCtxMiddleware,
        deliveryMQTT.ReplaceMessageClientMiddleware(subCtx,
client),
        deliveryMQTT.RecoverMiddleware,
    )))
```

```
    return tokens
}
package miller

import (
    "crypto/tls"
    "log"
    "net/url"

    mqtt "github.com/eclipse/paho.mqtt.golang"
)

var (
    OnConnect mqtt.OnConnectHandler = func(client mqtt.Client) {
        log.Println("on connect miller: successfully")
    }
    OnConnectionLost mqtt.ConnectionLostHandler = func(client
mqtt.Client, err error) {
        log.Printf("on connection lost miller: %s\n", err.Error())
    }
    OnReconnect mqtt.ReconnectHandler = func(client mqtt.Client, _
*mqtt.ClientOptions) {
        log.Println("on reconnect miller: trying to reconnect")
    }
    ConnAtempt mqtt.ConnectionAttemptHandler = func(broker
*url.URL, tlsCfg *tls.Config) *tls.Config {
        log.Printf("conntAtempt miller: broker address - %q\n",
broker.String())
        return nil
    }
)
```

ПРИЛОЖЕНИЕ П

Программный код MQTT обработчика «Модуль распознавания» со служебными MQTT обработчиками, маршрутизатором

```
package recognition

import (
    "context"
    "fmt"

    mqtt "github.com/eclipse/paho.mqtt.golang"
    deliveryMQTT "github.com/wonderf00l/fms-control-
system/internal/delivery/mqtt"
    "github.com/wonderf00l/fms-control-
system/internal/service/recognition"
    "go.uber.org/zap"
)

type HandlerMQTT struct {
    Service recognition.Service
    log     *zap.SugaredLogger
}

func NewHandlerMQTT(service recognition.Service, log
*zap.SugaredLogger) *HandlerMQTT {
    return &HandlerMQTT{
        Service: service,
        log:     log,
    }
}

func (h *HandlerMQTT) WorkpieceRecognition(client mqtt.Client, msg
mqtt.Message) {
    ctx, err := deliveryMQTT.GetContextFromMessage(msg)
    reqTopic, respTopic := msg.Topic(),
requestResponseTopics[msg.Topic()]
```

```
    if err != nil {
        deliveryMQTT.ResponseError(h.log, client, reqTopic,
respTopic, 1, false, err)
        return
    }

    if workpieceType, err := h.Service.RecognizeWorkpiece(ctx);
err != nil {
        deliveryMQTT.ResponseError(h.log, client, reqTopic,
respTopic, 1, false, err)
    } else if err = deliveryMQTT.ResponseOk(h.log, client,
reqTopic, respTopic, 1, false, fmt.Sprint("recognized workpiece:
", workpieceType), nil); err != nil {
        deliveryMQTT.ResponseError(h.log, client, reqTopic,
respTopic, 1, false, err)
    }
}

func (h *HandlerMQTT) PushMetrics(ctx context.Context, client
mqtt.Client) {
    if metrics, err := h.Service.GetMetrics(ctx); err != nil {
        deliveryMQTT.ResponseError(h.log, client, *new(string),
pushMetrics, 1, false, err)
    } else if err = deliveryMQTT.ResponseOk(h.log, client,
*new(string), pushMetrics, 1, true, "got recognition metrics
sucessfully", metrics); err != nil {
        deliveryMQTT.ResponseError(h.log, client, *new(string),
pushMetrics, 1, false, err)
    }
}
package recognition

import (
    "context"
```

```
    mqtt "github.com/eclipse/paho.mqtt.golang"
    deliveryMQTT "github.com/wonderf00l/fms-control-
system/internal/delivery/mqtt"
)

var (
    requestCmdTopicPrefix  = "/recognition/cmd/request"
    responseCmdTopicPrefix = "/recognition/cmd/response"
)

var (
    recognizeWorkpieceReq  = requestCmdTopicPrefix + "/recognize"
    recognizeWorkpieceResp = responseCmdTopicPrefix + "/recognize"

    pushMetrics = "/recognition/metrics"

    requestResponseTopics = map[string]string{
        recognizeWorkpieceReq: recognizeWorkpieceResp,
    }
)

func SetSubscribeRouter(subCtx context.Context, client
*deliveryMQTT.ClientMQTT, handler *HandlerMQTT) []mqtt.Token {
    tokens := make([]mqtt.Token, 0, 1)

    tokens = append(tokens,
client.Subscribe(recognizeWorkpieceReq, 1,
deliveryMQTT.ApplyMiddlewareStack(
        handler.WorkpieceRecognition,
        deliveryMQTT.LoggingMiddleware,
        deliveryMQTT.CheckMsgWithCtxMiddleware,
        deliveryMQTT.ReplaceMessageClientMiddleware(subCtx,
client),
        deliveryMQTT.RecoverMiddleware,
    )))
```

```
    return tokens
}

package recognition

import (
    "crypto/tls"
    "log"
    "net/url"

    mqtt "github.com/eclipse/paho.mqtt.golang"
)

var (
    OnConnect mqtt.OnConnectHandler = func(client mqtt.Client) {
        log.Println("on connect recognition: successfully")
    }
    OnConnectionLost mqtt.ConnectionLostHandler = func(client
mqtt.Client, err error) {
        log.Printf("on connection lost recognition: %s\n",
err.Error())
    }
    OnReconnect mqtt.ReconnectHandler = func(client mqtt.Client, _
*mqtt.ClientOptions) {
        log.Println("on reconnect recognition: trying to
reconnect")
    }
    ConnAtempt mqtt.ConnectionAttemptHandler = func(broker
*url.URL, tlsCfg *tls.Config) *tls.Config {
        log.Printf("conntAtempt recognition: broker address -
%q\n", broker.String())
        return nil
    }
)
```

ПРИЛОЖЕНИЕ Р

Программный код MQTT клиента и MQTT сообщения с расширенным функционалом

```
package mqtt

import (
    "context"
    "sync"

    mqtt "github.com/eclipse/paho.mqtt.golang"

    "github.com/google/uuid"
    "go.uber.org/zap"
)

type ClientMQTT struct {
    mqtt.Client
    Log *zap.SugaredLogger
}

func NewClientMQTT(opts *mqtt.ClientOptions, log
*zap.SugaredLogger) *ClientMQTT {
    client := mqtt.NewClient(opts)
    return &ClientMQTT{Client: client, Log: log}
}

func (c *ClientMQTT) Publish(topic string, qos byte, retained
bool, payload interface{}) mqtt.Token {
    token := c.Client.Publish(topic, qos, retained, payload)
    pubID, err := uuid.NewRandom()
    if err != nil {
        c.Log.Warnln("Generate PublishID: ", err)
    }
    c.Log.Infow(
```

```
        "Published message:",
        zap.String("ID", pubID.String()),
        zap.String("topic", topic),
        zap.Int8("QOS", int8(qos)),
        zap.Bool("retained", retained),
    )
    return token
}

func CheckMQTTToken(ctx context.Context, token mqtt.Token, msg
string, log *zap.SugaredLogger, wg *sync.WaitGroup, errCh chan<-
struct{}) {
    defer wg.Done()
    select {
    case <-token.Done():
        if token.Error() != nil {
            errCh <- struct{}{}
        }
    case <-ctx.Done():
        log.Warnf("%s: OS signal was received before token
release", msg)
    }
}

package mqtt

import (
    "context"
    "errors"

    mqtt "github.com/eclipse/paho.mqtt.golang"
)

type MessageWithCtx struct {
    mqtt.Message
    ctx context.Context
```

```
}

func (m *MessageWithCtx) Context() context.Context {
    return m.ctx
}

func GetContextFromMessage(msg mqtt.Message) (context.Context,
error) {
    customMsg, ok := msg.(*MessageWithCtx)
    if !ok {
        return nil, errors.New("failed custom message conversion")
    }
    return customMsg.Context(), nil
}
```

ПРИЛОЖЕНИЕ С

Программный код обработчика, публикующего метрики с заданной периодичностью

```
package mqtt
import (
    "context"
    "sync"
    "time"
    mqtt "github.com/eclipse/paho.mqtt.golang"
    "go.uber.org/zap"
)
var (
    metricsPeroid = 4 * time.Second
)
type MetricsProvider interface {
    PushMetrics(context.Context, mqtt.Client)
}
func RunMetricsWorker(ctx context.Context, client mqtt.Client,
provider MetricsProvider, service string, log *zap.SugaredLogger,
wg *sync.WaitGroup) {
    defer wg.Done()
    log.Infof("Service %s beginning to push it's metrics",
service)
    for {
        select {
        case <-time.After(metricsPeroid):
            provider.PushMetrics(ctx, client)
        case <-ctx.Done():
            log.Infof("Service %s finishing pushing metrics",
service)
            return
        }
    }
}
```

ПРИЛОЖЕНИЕ Т

Программный код промежуточных функций-обработчиков – middleware

```
package mqtt

import (
    "context"
    "log"
    "time"

    mqtt "github.com/eclipse/paho.mqtt.golang"
    "go.uber.org/zap"
)
var (
    topicForPanicMiddleware    = "/aux/accidents/panic"
    retainedForPanicMiddleware = false
    qosForPanicMiddleware      = 1

    topicForCheckMsgMiddleware    = "/aux/accidents/internals"
    retainedForCheckMsgMiddleware = false
    qosForCheckMsgMiddleware      = 1
)
type Middleware func(mqtt.MessageHandler) mqtt.MessageHandler
func ApplyMiddlewareStack(initialHandler mqtt.MessageHandler,
middlewares ...Middleware) mqtt.MessageHandler {
    for _, middleware := range middlewares {
        initialHandler = middleware(initialHandler)
    }
    return initialHandler
}
func ReplaceMessageClientMiddleware(ctx context.Context,
customClient *ClientMQTT) Middleware {
    return func(next mqtt.MessageHandler) mqtt.MessageHandler {
        return func(client mqtt.Client, msg mqtt.Message) {
            customMsg := &MessageWithCtx{Message: msg, ctx: ctx}
            next(customClient, customMsg)
```

```
        }
    }
}
func CheckMsgWithCtxMiddleware(next mqtt.MessageHandler)
mqtt.MessageHandler {
    return func(client mqtt.Client, msg mqtt.Message) {
        _, gotClient := client.(*ClientMQTT)
        if !gotClient {
            log.Println("Check msg and client middleware: can't
extract custom client")
        }
        _, gotMsg := msg.(*MessageWithCtx)
        if !gotMsg {
            log.Println("Check msg and client middleware: can't
extract custom message")
        }
        if !gotClient || !gotMsg {
            ResponseError(
                struct{}{},
                client, msg.Topic(),
                topicForCheckMsgMiddleware,
                byte(qosForCheckMsgMiddleware),
                retainedForCheckMsgMiddleware,
                &customMsgClientNotFoundError{msg: msg},
            )
            return
        }
        next(client, msg)
    }
}
func LoggingMiddleware(next mqtt.MessageHandler)
mqtt.MessageHandler {
    return func(client mqtt.Client, msg mqtt.Message) {
        customClient, ok := client.(*ClientMQTT)
        if !ok {
```

```
            log.Println("Logging middleware: can't extract client
from the interface")
            next(client, msg)
            return
        }
        start := time.Now()
        next(client, msg)
        customClient.Log.Infow(
            "Got message",
            zap.String("topic", msg.Topic()),
            zap.Int8("QOS", int8(msg.Qos())),
            zap.Bool("retained", msg.Retained()),
            zap.Uint16("MessageID", uint16(msg.MessageID())),
            zap.Duration("processing_time", time.Since(start)),
        )
    }
}
func RecoverMiddleware(next mqtt.MessageHandler)
mqtt.MessageHandler {
    return func(client mqtt.Client, msg mqtt.Message) {
        defer func() {
            if err := recover(); err != nil {
                ResponseError(
                    struct{}{},
                    client, msg.Topic(),
                    topicForPanicMiddleware,
                    byte(qosForPanicMiddleware),
                    retainedForPanicMiddleware,
                    &serviceGotPanicError{msg: msg},
                )
            }
        }()

        next(client, msg)
    }
}
```

ПРИЛОЖЕНИЕ У

Программный код опций MQTT клиента

```
package mqtt

import (
    "crypto/tls"
    "time"

    mqtt "github.com/eclipse/paho.mqtt.golang"
    "github.com/wonderf00l/fms-control-system/internal/configs"
)

var (
    _connectTimeout = 5 * time.Second
    _writeTimeout   = 5 * time.Second
    _keepAlive      = 50 * time.Second
    _pingTimeout    = 5 * time.Second
)

type DefaultHandlers struct {
    OnConnect         mqtt.OnConnectHandler
    OnConnectionLost  mqtt.ConnectionLostHandler
    OnReconnect       mqtt.ReconnectHandler
    ConnectionAttempt mqtt.ConnectionAttemptHandler
}

func setDefaultHandlers(opts *mqtt.ClientOptions, defaultHandlers
DefaultHandlers) {
    opts.OnConnectionLost = defaultHandlers.OnConnectionLost
    opts.OnConnect = defaultHandlers.OnConnect
    opts.OnReconnecting = defaultHandlers.OnReconnect
}
```

```
func NewClientOptions(brokerCfg *configs.BrokerConfig, tlsCfg
*tls.Config, defaultHandlers DefaultHandlers, clientID string)
*mqtt.ClientOptions {
    opts := mqtt.NewClientOptions()
    opts.SetTLSConfig(tlsCfg)

    opts.AddBroker(brokerCfg.Address)
    opts.SetUsername(brokerCfg.Username)
    opts.SetPassword(brokerCfg.Password)

    opts.SetClientID(clientID)
    opts.SetOrderMatters(true)
    opts.SetCleanSession(false)

    opts.SetConnectTimeout(_connectTimeout)
    opts.SetWriteTimeout(_writeTimeout)
    opts.SetKeepAlive(_keepAlive)
    opts.SetPingTimeout(_pingTimeout)

    opts.SetConnectRetry(true)
    opts.SetAutoReconnect(true)
    setDefaultHandlers(opts, defaultHandlers)
    return opts
}
```

ПРИЛОЖЕНИЕ Ф

Программный код функций формирования ответа со статусом «ok» / «error»

```
package mqtt

import (
    "encoding/json"
    "errors"
    "log"

    mqtt "github.com/eclipse/paho.mqtt.golang"
    "github.com/google/uuid"
    errorsPkg "github.com/wonderf00l/fms-control-
system/internal/errors"
    "go.uber.org/zap"
)
var (
    StatusOK            = "ok"
    StatusError         = "error"
    InternalErrorCode   = "internal_error"
    InternalErorMessage = "internal service error occured"
)
type JSONResponse struct {
    RespUUID     string `json:"responseID"`
    RequestTopic string `json:"request_topic"`
    Status       string `json:"status"`
    Message      string `json:"message"`
    Payload      any    `json:"payload"`
}
type JSONErrResponse struct {
    RespUUID     string `json:"responseID"`
    RequestTopic string `json:"request_topic"`
    Status       string `json:"status"`
    Message      string `json:"message"`
    Code         string `json:"code"`
}
```

```
func ConvertErrorMQTT(err error) (code string, msg string) {
    var declaredErr errorsPkg.DeclaredError
    if errors.As(err, &declaredErr) {
        switch declaredErr.Type() {
        case errorsPkg.ErrNotFound:
            return "not_found", err.Error()
        case errorsPkg.ErrAlreadyExists:
            return "already_exists", err.Error()
        case errorsPkg.ErrInvalidInput:
            return "invalid_input", err.Error()
        case errorsPkg.ErrTimeout:
            return "timeout", err.Error()
        case errorsPkg.ErrServiceNotReady:
            return "service_not_ready", err.Error()
        case errorsPkg.ErrServiceNotAvailable:
            return "not_available", err.Error()
        case errorsPkg.ErrDepsNotReady:
            return "deps_not_ready", err.Error()
        case errorsPkg.ErrInavlidStateForCMD:
            return "invalid_cmd_state", err.Error()
        }
    }
    return InternalErrorCode, InternalErorMessage
}
func ResponseOk(log *zap.SugaredLogger, client mqtt.Client,
reqTopic, respTopic string, qos byte, retained bool, msg string,
payload any) error {
    serialized, err := json.Marshal(JSONResponse{
        RespUUID:     uuid.NewString(),
        RequestTopic: reqTopic,
        Status:       "ok",
        Message:      msg,
        Payload:      payload,
    })
    if err != nil {
        return err
```

```
    }
    token := client.Publish(respTopic, qos, retained, serialized)
    go func() {
        <-token.Done()
        if err := token.Error(); err != nil {
            log.Errorf("publish ok response: topic - %s, error -
%s", respTopic, err.Error())
        }
    }()
    return nil
}
func ResponseError(serviceLog any, client mqtt.Client, reqTopic,
respTopic string, qos byte, retained bool, err error) {
    logger, ok := serviceLog.(*zap.SugaredLogger)

    code, msg := ConvertErrorMQTT(err)
    if code == InternalErrorCode {
        if ok {
            logger.Errorf("Internal service error occured: %s",
err.Error())
        } else {
            log.Printf("Internal service error occured: %s",
err.Error())
        }
    }

    serialized, _ := json.Marshal(JSONErrResponse{
        RespUUID:     uuid.NewString(),
        RequestTopic: reqTopic,
        Status:       "error",
        Message:      msg,
        Code:         code,
    })
    token := client.Publish(respTopic, qos, retained, serialized)
    go func() {
        <-token.Done()
```

```
        if err := token.Error(); err != nil {
            if ok {
                logger.Errorf("publish error response: topic - %s,
error - %s", respTopic, err.Error())
            } else {
                log.Printf("publish error response: topic - %s,
error - %s", respTopic, err.Error())
            }

        }
    }() }
```

ПРИЛОЖЕНИЕ Х

Программный код с объявленными типизированными ошибками сервиса

```
package errors

import "fmt"

type ErrorType uint8

type ServiceName string

const (
    _ ErrorType = iota
    ErrNotFound
    ErrAlreadyExists
    ErrInvalidInput
    ErrNotImplemented
    ErrTimeout
    ErrServiceNotAvailable
    ErrServiceNotReady
    ErrDepsNotReady
    ErrInavlidStateForCMD
)

const (
    Storage     ServiceName = "storage"
    Conveyor    ServiceName = "conveyor"
    Recognition ServiceName = "recognition"
    Lathe       ServiceName = "lathe"
    Miller      ServiceName = "miller"
)

type DeclaredError interface {
    Type() ErrorType
}
```

```
type InternalError struct {
    Message string
    Layer   string
}

func (e *InternalError) Error() string {
    return fmt.Sprintf("Internal error occured. Message: '%s'.
Layer: %s", e.Message, e.Layer)
}

type TimeoutExceededError struct {
    Service ServiceName
}

func (e *TimeoutExceededError) Error() string {
    return fmt.Sprintf("service %s: timeout exceeded", e.Service)
}

func (e *TimeoutExceededError) Type() ErrorType {
    return ErrTimeout
}

type ServiceOfflineError struct {
    Service ServiceName
}

func (e *ServiceOfflineError) Error() string {
    return fmt.Sprintf("service %s is offline right now",
e.Service)
}

func (e *ServiceOfflineError) Type() ErrorType {
    return ErrServiceNotAvailable
}
```

```
type ServiceNotReadyError struct {
    Service ServiceName
}

func (e *ServiceNotReadyError) Error() string {
    return fmt.Sprintf("service %s is not ready", e.Service)
}

func (e *ServiceNotReadyError) Type() ErrorType {
    return ErrServiceNotReady
}

type DepsNotReadyError struct {
    Service    ServiceName
    Dependency ServiceName
    Reason     error
}

func (e *DepsNotReadyError) Error() string {
    return fmt.Sprintf("%s: dependency %v is not ready -- %s",
e.Service, e.Dependency, e.Reason.Error())
}

func (e *DepsNotReadyError) Type() ErrorType {
    return ErrDepsNotReady
}
```

ПРИЛОЖЕНИЕ Ц

Программный код с логгером сервиса и опциями логирования

```
package logger

import "go.uber.org/zap"

func New(config *zap.Config) (*zap.SugaredLogger, error) {
    logger, err := config.Build()
    if err != nil {
        return nil, err
    }
    return logger.Sugar(), nil
}

package logger

import (
    "go.uber.org/zap"
    "go.uber.org/zap/zapcore"
)

type Option func(*zap.Config)

func ConfigureTime(field, layout string) Option {
    return func(cfg *zap.Config) {
        cfg.EncoderConfig.TimeKey = field
        cfg.EncoderConfig.EncodeTime =
zapcore.TimeEncoderOfLayout(layout)
    }
}

func ConfigureEncoding(encoding string) Option {
    return func(cfg *zap.Config) {
        cfg.Encoding = encoding
    }
```

```
}

func ConfigureOutput(paths []string) Option {
    return func(cfg *zap.Config) {
        cfg.OutputPaths = paths
    }
}

func ConfigureErrorOutput(paths []string) Option {
    return func(cfg *zap.Config) {
        cfg.ErrorOutputPaths = paths
    }
}

func ConfigureTimeKey(key string) Option {
    return func(cfg *zap.Config) {
        cfg.EncoderConfig.TimeKey = key
    }
}

func NewConfig(opts ...Option) *zap.Config {
    cfg := zap.NewProductionConfig()
    for _, opt := range opts {
        opt(&cfg)
    }
    return &cfg
}
```

ПРИЛОЖЕНИЕ Ч

Программный код с алгоритмом конфигурирования сервиса: MQTT, TLS конфигурация из одного файла формата yaml

```
package configs

import (
    "go.uber.org/config"
)

var (
    envKey                   = "app.env_file"
    brokerKey      ClientKey = "app.broker"
    StorageKey     ClientKey = "app.clients.storage"
    ConveyorKey    ClientKey = "app.clients.conveyor"
    RecognitionKey ClientKey = "app.clients.recognition"
    LatheKey       ClientKey = "app.clients.lathe"
    MillerKey      ClientKey = "app.clients.miller"
    clientsCfgKeys           = [5]ClientKey{StorageKey,
ConveyorKey, RecognitionKey, LatheKey, MillerKey}
)

type brokerYAML struct {
    Scheme  string `yaml:"scheme"`
    Address string `yaml:"address"`
    Port    int    `yaml:"port"`
}

type clientYAML struct {
    ID       string `yaml:"ID"`
    BrokerCA string `yaml:"brokerCA"`
    ClientCA string `yaml:"clientCA"`
    Cert     string `yaml:"cert"`
    Key      string `yaml:"key"`
}
```

```
func NewYAML(filename string) (*config.YAML, error) {
    cfg, err := config.NewYAML(config.File(filename))
    if err != nil {
        return nil, &yamlConfigError{filename: filename, inner:
err}
    }
    return cfg, nil
}
package configs

import (
    "crypto/tls"
    "crypto/x509"
    "os"
)

type clientParamsTLS struct {
    brokerCA         string
    clientCA         string
    clientCert       string
    clientPrivateKey string
}

func NewTLSConfig(params clientParamsTLS) (*tls.Config, error) {
    rootCAs := x509.NewCertPool()
    clientCAs := x509.NewCertPool()

    brokerCert, err := os.ReadFile(params.brokerCA)
    if err != nil {
        return nil, &tlsConfigError{err}
    }
    clientCert, err := os.ReadFile(params.clientCA)
    if err != nil {
        return nil, &tlsConfigError{err}
    }
```

```
    rootCAs.AppendCertsFromPEM(brokerCert)
    clientCAs.AppendCertsFromPEM(clientCert)

    cert, err := tls.LoadX509KeyPair(params.clientCert,
params.clientPrivateKey)
    if err != nil {
        return nil, &tlsConfigError{err}
    }

    cert.Leaf, err = x509.ParseCertificate(cert.Certificate[0])
    if err != nil {
        return nil, &tlsConfigError{err}
    }

    return &tls.Config{
        RootCAs:      rootCAs,
        ClientAuth:   tls.RequireAndVerifyClientCert,
        ClientCAs:    clientCAs,
        Certificates: []tls.Certificate{cert},
    }, nil

}
package configs

import (
    "fmt"
    "os"
)

type BrokerConfig struct {
    Address  string
    Username string
    Password string
}

func brokerAddress(brokerParams *brokerYAML) string {
```

```
    return fmt.Sprintf("%s://%s:%d",
        brokerParams.Scheme,
        brokerParams.Address,
        brokerParams.Port,
    )
}

func NewBrokerConfig(address string) (*BrokerConfig, error) {
    username := os.Getenv("BROKER_USERNAME")
    password := os.Getenv("BROKER_PASSWORD")
    if len(username) == 0 || len(password) == 0 {
        return nil, fmt.Errorf("incorrect .env broker
credentials")
    }

    return &BrokerConfig{
        Address:  address,
        Username: username,
        Password: password,
    }, nil
}
package configs

import (
    "crypto/tls"
    "fmt"

    "go.uber.org/config"
)

type ClientKey string

type Configs struct {
    BrokerConfig *BrokerConfig
    TLSConfigs   map[ClientKey]*tls.Config
    ClientIDs    map[ClientKey]string
```

```
}

type Parameters struct {
    EnvFile       string
    BrokerAddress string
    ClientIDs     map[ClientKey]string
    TLSParams     map[ClientKey]clientParamsTLS
}

func ExtractCfgValues(appCfg *config.YAML) (*Parameters, error) {
    var (
        tlsParams = map[ClientKey]clientParamsTLS{}
        clientIDs = map[ClientKey]string{}
    )

    envExtracted := ""
    if err := appCfg.Get(envKey).Populate(&envExtracted); err !=
nil {
        return nil, &configsError{inner: err}
    }

    brokerExtracted := brokerYAML{}
    if err :=
appCfg.Get(string(brokerKey)).Populate(&brokerExtracted); err !=
nil {
        return nil, &configsError{inner: err}
    }

    for _, clKey := range clientsCfgKeys {
        extracted := clientYAML{}
        if err := appCfg.Get(string(clKey)).Populate(&extracted);
err != nil {
            return nil, &configsError{inner: fmt.Errorf("extract
values: %w", err)}
        }
        tlsParams[clKey] = clientParamsTLS{
```

```
            brokerCA:         extracted.BrokerCA,
            clientCA:         extracted.ClientCA,
            clientCert:       extracted.Cert,
            clientPrivateKey: extracted.Key,
        }
        clientIDs[clKey] = extracted.ID
    }

    return &Parameters{
        EnvFile:       envExtracted,
        BrokerAddress: brokerAddress(&brokerExtracted),
        ClientIDs:     clientIDs,
        TLSParams:     tlsParams,
    }, nil
}

func NewConfigs(params *Parameters) (*Configs, error) {
    tlsConfigs := make(map[ClientKey]*tls.Config,
len(params.TLSParams))
    for clKey, TLSparams := range params.TLSParams {
        tlsCfg, err := NewTLSConfig(TLSparams)
        if err != nil {
            return nil, &configsError{inner: fmt.Errorf("new
configs: %w", err)}
        }
        tlsConfigs[clKey] = tlsCfg
    }

    brokerCfg, err := NewBrokerConfig(params.BrokerAddress)
    if err != nil {
        return nil, &configsError{inner: err}
    }
    return &Configs{
        BrokerConfig: brokerCfg,
        TLSConfigs:   tlsConfigs,
        ClientIDs:    params.ClientIDs,
```

```
    }, nil
}
package configs
import "fmt"
type yamlConfigError struct {
    inner    error
    filename string
}
func (e *yamlConfigError) Error() string {
    return fmt.Sprintf("YAML config %q: %s", e.filename,
e.inner.Error())
}
type configsError struct {
    inner error
}
func (e *configsError) Error() string {
    return fmt.Sprintf("Configs: %s", e.inner.Error())
}
type tlsConfigError struct {
    inner error
}
func (e *tlsConfigError) Error() string {
    return fmt.Sprintf("TLS config: %s", e.inner.Error())
}
```

ПРИЛОЖЕНИЕ Ш

Программный код с алгоритмом инициализации первичных зависимостей, созданием файлов конфигурации для каждого MQTT клиента и запуском главного алгоритма сервиса, в котором создаются MQTT опции, MQTT клиенты, устанавливаются маршрутизаторы, применяются middleware, инициализируются подключения к брокеру Yandex Cloud, запускаются обработчики, публикающие метрики и т.д.

```
package app

import "fmt"

type initPrereqError struct {
    inner error
}

func (e *initPrereqError) Error() string {
    return fmt.Sprintf("Init prerequisites: %s", e.inner.Error())
}

type appRunError struct {
    inner error
}

func (e *appRunError) Error() string {
    return fmt.Sprintf("run application: %s", e.inner.Error())
}

package app

import (
    "context"
    "flag"
    "strings"
```

```
    "github.com/joho/godotenv"
    "github.com/wonderf00l/fms-control-system/internal/configs"
    "github.com/wonderf00l/fms-control-system/internal/pkg/logger"
    "go.uber.org/config"
    "go.uber.org/zap"
)

var (
    appConfigPath = "configs/app.yml"
)

var (
    timeKey             = "timestamp"
    logEncoding         = flag.String("logenc", "json", "preferred
logging format")
    logOutputPaths      = flag.String("log", "stdout,access.log",
"file paths to write logging output to")
    logErrorOutputPaths = flag.String("logerror",
"stderr,error.log", "path to write internal logger errors to.")
)

func InitPrerequisites(_ context.Context) (*zap.SugaredLogger,
*configs.Configs, error) {
    appCfg, err := config.NewYAML(config.File(appConfigPath))
    if err != nil {
        return nil, nil, &initPrereqError{err}
    }

    cfgParameters, err := configs.ExtractCfgValues(appCfg)
    if err != nil {
        return nil, nil, &initPrereqError{err}
    }

    err = godotenv.Load(cfgParameters.EnvFile)
    if err != nil {
        return nil, nil, &initPrereqError{err}
```

```
    }

    configs, err := configs.NewConfigs(cfgParameters)
    if err != nil {
        return nil, nil, &initPrereqError{err}
    }

    flag.Parse()

    log, err := logger.New(logger.NewConfig(
        logger.ConfigureTimeKey(timeKey),
        logger.ConfigureEncoding(*logEncoding),
        logger.ConfigureOutput(strings.Split(*logOutputPaths,
","")),

logger.ConfigureErrorOutput(strings.Split(*logErrorOutputPaths,
","")),
    ))
    if err != nil {
        return nil, nil, &initPrereqError{err}
    }

    return log, configs, nil
}
package app

import (
    "context"
    "errors"
    "sync"

    mqtt "github.com/eclipse/paho.mqtt.golang"
    "github.com/wonderf00l/fms-control-system/internal/configs"
    delivery "github.com/wonderf00l/fms-control-
system/internal/delivery/mqtt"
    "github.com/wonderf00l/fms-control-system/internal/entity"
```

```
    "go.uber.org/zap"

    entityConveyor "github.com/wonderf00l/fms-control-
system/internal/entity/conveyor"
    entityLathe "github.com/wonderf00l/fms-control-
system/internal/entity/lathe"
    entityMiller "github.com/wonderf00l/fms-control-
system/internal/entity/miller"
    entityRecognition "github.com/wonderf00l/fms-control-
system/internal/entity/recognition"
    entityStorage "github.com/wonderf00l/fms-control-
system/internal/entity/storage"
    deliveryConveyor "github.com/wonderf00l/fms-control-
system/internal/delivery/mqtt/conveyor"
    deliveryLathe "github.com/wonderf00l/fms-control-
system/internal/delivery/mqtt/lathe"
    deliveryMiller "github.com/wonderf00l/fms-control-
system/internal/delivery/mqtt/miller"
    deliveryRecognition "github.com/wonderf00l/fms-control-
system/internal/delivery/mqtt/recognition"
    deliveryStorage "github.com/wonderf00l/fms-control-
system/internal/delivery/mqtt/storage"
    serviceConveyor "github.com/wonderf00l/fms-control-
system/internal/service/conveyor"
    serviceLathe "github.com/wonderf00l/fms-control-
system/internal/service/lathe"
    serviceMiller "github.com/wonderf00l/fms-control-
system/internal/service/miller"
    serviceRecognition "github.com/wonderf00l/fms-control-
system/internal/service/recognition"
    serviceStorage "github.com/wonderf00l/fms-control-
system/internal/service/storage"
)
var (
    timeForDisconnect uint = 100
)
```

```
func Run(ctx context.Context, logger *zap.SugaredLogger, cfgs
*configs.Configs) error {
    wg := &sync.WaitGroup{}
    tokenErrCh := make(chan struct{}, 1024)
    entityPool := entity.Pool{
        Storage:     entityStorage.NewStorage(),
        Conveyor:    entityConveyor.NewConveyor(),
        Recognition: entityRecognition.NewRecognition(),
        Lathe:       entityLathe.NewLathe(),
        Miller:      entityMiller.NewMiller(),
    }
    storageHandler :=
deliveryStorage.NewHandlerMQTT(serviceStorage.NewService(logger,
&entityPool), logger)
    conveyorHandler :=
deliveryConveyor.NewHandlerMQTT(serviceConveyor.NewService(logger,
&entityPool), logger)
    recognitionHandler :=
deliveryRecognition.NewHandlerMQTT(serviceRecognition.NewService(l
ogger, &entityPool), logger)
    latheHandler :=
deliveryLathe.NewHandlerMQTT(serviceLathe.NewService(logger,
&entityPool), logger)
    millerHandler :=
deliveryMiller.NewHandlerMQTT(serviceMiller.NewService(logger,
&entityPool), logger)
    storageClient :=
delivery.NewClientMQTT(delivery.NewClientOptions(
        cfgs.BrokerConfig,
        cfgs.TLSConfigs[configs.StorageKey],
        delivery.DefaultHandlers{
            OnConnect:         deliveryStorage.OnConnect,
            OnConnectionLost:  deliveryStorage.OnConnectionLost,
            OnReconnect:       deliveryStorage.OnReconnect,
            ConnectionAttempt: deliveryStorage.ConnAtempt,
```

```
        }, cfgs.ClientIDs[configs.StorageKey]), logger)
    conveyorClient :=
delivery.NewClientMQTT(delivery.NewClientOptions(
        cfgs.BrokerConfig,
        cfgs.TLSConfigs[configs.ConveyorKey],
        delivery.DefaultHandlers{
            OnConnect:         deliveryConveyor.OnConnect,
            OnConnectionLost:  deliveryConveyor.OnConnectionLost,
            OnReconnect:       deliveryConveyor.OnReconnect,
            ConnectionAttempt: deliveryConveyor.ConnAtempt,
        }, cfgs.ClientIDs[configs.ConveyorKey]), logger)
    recognitionClient :=
delivery.NewClientMQTT(delivery.NewClientOptions(
        cfgs.BrokerConfig,
        cfgs.TLSConfigs[configs.RecognitionKey],
        delivery.DefaultHandlers{
            OnConnect:         deliveryRecognition.OnConnect,
            OnConnectionLost:
 deliveryRecognition.OnConnectionLost,
            OnReconnect:       deliveryRecognition.OnReconnect,
            ConnectionAttempt: deliveryRecognition.ConnAtempt,
        }, cfgs.ClientIDs[configs.RecognitionKey]), logger)
    latheClient :=
delivery.NewClientMQTT(delivery.NewClientOptions(
        cfgs.BrokerConfig,
        cfgs.TLSConfigs[configs.LatheKey],
        delivery.DefaultHandlers{
            OnConnect:         deliveryLathe.OnConnect,
            OnConnectionLost:  deliveryLathe.OnConnectionLost,
            OnReconnect:       deliveryLathe.OnReconnect,
            ConnectionAttempt: deliveryLathe.ConnAtempt,
        }, cfgs.ClientIDs[configs.LatheKey]), logger)
    millerClient :=
delivery.NewClientMQTT(delivery.NewClientOptions(
        cfgs.BrokerConfig,
        cfgs.TLSConfigs[configs.MillerKey],
```

```
        delivery.DefaultHandlers{
            OnConnect:         deliveryMiller.OnConnect,
            OnConnectionLost:  deliveryMiller.OnConnectionLost,
            OnReconnect:       deliveryMiller.OnReconnect,
            ConnectionAttempt: deliveryMiller.ConnAtempt,
        }, cfgs.ClientIDs[configs.MillerKey]), logger)
    wg.Add(5)
    go delivery.CheckMQTTToken(ctx, storageClient.Connect(),
"storage client connect", logger, wg, tokenErrCh)
    go delivery.CheckMQTTToken(ctx, conveyorClient.Connect(),
"conveyor client connect", logger, wg, tokenErrCh)
    go delivery.CheckMQTTToken(ctx, recognitionClient.Connect(),
"recognition client connect", logger, wg, tokenErrCh)
    go delivery.CheckMQTTToken(ctx, latheClient.Connect(), "lathe
client connect", logger, wg, tokenErrCh)
    go delivery.CheckMQTTToken(ctx, millerClient.Connect(),
"miller client connect", logger, wg, tokenErrCh)
    wg.Wait() // waiting is obligatory
    if len(tokenErrCh) > 0 {
        return &appRunError{inner: errors.New("one or more clients
couldn't establish the connection")}
    }
    defer storageClient.Disconnect(timeForDisconnect)
    defer recognitionClient.Disconnect(timeForDisconnect)
    defer conveyorClient.Disconnect(timeForDisconnect)
    defer latheClient.Disconnect(timeForDisconnect)
    defer millerClient.Disconnect(timeForDisconnect)
    subscribeTokens := make([]mqtt.Token, 0)
    subscribeTokens = append(subscribeTokens,
deliveryStorage.SetSubscribeRouter(ctx, storageClient,
storageHandler)...)
    subscribeTokens = append(subscribeTokens,
deliveryConveyor.SetSubscribeRouter(ctx, conveyorClient,
conveyorHandler)...)
```

```
    subscribeTokens = append(subscribeTokens,
deliveryRecognition.SetSubscribeRouter(ctx, recognitionClient,
recognitionHandler)...)
    subscribeTokens = append(subscribeTokens,
deliveryLathe.SetSubscribeRouter(ctx, latheClient,
latheHandler)...)
    subscribeTokens = append(subscribeTokens,
deliveryMiller.SetSubscribeRouter(ctx, millerClient,
millerHandler)...)
    wg.Add(len(subscribeTokens))
    for _, token := range subscribeTokens {
        go delivery.CheckMQTTToken(ctx, token, "making
subscription", logger, wg, tokenErrCh)
    }
    wg.Wait()

    if len(tokenErrCh) > 0 {
        return &appRunError{inner: errors.New("one or more clients
couldn't make a subscription")}
    }
    wg.Add(5)
    go delivery.RunMetricsWorker(ctx, storageClient,
storageHandler, "storage", logger, wg)
    go delivery.RunMetricsWorker(ctx, recognitionClient,
conveyorHandler, "conveyor", logger, wg)
    go delivery.RunMetricsWorker(ctx, recognitionClient,
recognitionHandler, "recognition", logger, wg)
    go delivery.RunMetricsWorker(ctx, latheClient, latheHandler,
"lathe", logger, wg)
    go delivery.RunMetricsWorker(ctx, millerClient, millerHandler,
"miller", logger, wg)

    <-ctx.Done()
    wg.Wait()
    logger.Infoln("Shutting down gracefully")
```

```
    return nil
}
```

ПРИЛОЖЕНИЕ Щ

Программный код точки входа в приложение, исполнение программы начинается отсюда

```
package main

import (
    "context"
    "log"
    "os"
    "os/signal"
    "syscall"

    "github.com/wonderf00l/fms-control-system/internal/app"
)
func main() {
    serviceCtx, cancel :=
signal.NotifyContext(context.Background(), os.Interrupt,
syscall.SIGTERM)
    defer cancel()

    serviceLogger, cfgFiles, err :=
app.InitPrerequisites(serviceCtx)
    if err != nil {
        log.Fatal(err)
    }
    defer func() {
        if err = serviceLogger.Sync(); err != nil {
            log.Fatal("Sync service logger: ", err)
        }
    }()
    if err = app.Run(serviceCtx, serviceLogger, cfgFiles); err !=
nil {
```

```
        serviceLogger.Fatal(err)
    }
}
```

ПРИЛОЖЕНИЕ Ы

Содержимое yaml файла конфигурации сервиса

```
app:
  env_file: .env
  broker:
    scheme: tls
    address: mqtt.cloud.yandex.net
    port: 8883
  clients:
    storage:
      ID: storage
      brokerCA: certs/rootCA.crt
      clientCA: certs/self-signed.crt
      cert: certs/storage.crt
      key: certs/storage.key
    conveyor:
      ID: conveyor
      brokerCA: certs/rootCA.crt
      clientCA: certs/self-signed.crt
      cert: certs/conveyor.crt
      key: certs/conveyor.key
    recognition:
      ID: recognition
      brokerCA: certs/rootCA.crt
      clientCA: certs/self-signed.crt
      cert: certs/recognition.crt
      key: certs/recognition.key
    lathe:
      ID: lathe
      brokerCA: certs/rootCA.crt
      clientCA: certs/self-signed.crt
      cert: certs/lathe.crt
```

```
    key: certs/lathe.key
  miller:
    ID: miller
    brokerCA: certs/rootCA.crt
    clientCA: certs/self-signed.crt
    cert: certs/miller.crt
    key: certs/miller.key
```

ПРИЛОЖЕНИЕ Э

Исходный код клиента для тестирования

```
package main

import (
    "fmt"
    "log"
    "os"
    "os/signal"
    "syscall"
    "time"

    mqtt "github.com/eclipse/paho.mqtt.golang"
)
func main() {
    opts := mqtt.NewClientOptions()
    opts.SetTLSConfig(NewTLSConfig())
    opts.SetUsername("a44p9bq63uah16qnocbi")
    opts.SetPassword("iot-course-worker22036")
    opts.AddBroker(SERVERADDRESS)
    opts.SetClientID(CLIENTID)

    opts.SetOrderMatters(true)
    opts.ConnectTimeout = time.Second
    opts.WriteTimeout = time.Second
    opts.KeepAlive = 10
    opts.PingTimeout = time.Second     should be quick

    opts.ConnectRetry = true
    opts.AutoReconnect = true

    opts.OnConnectionLost = func(cl mqtt.Client, err error) {
        fmt.Println("connection lost")
    }
    opts.OnConnect = func(mqtt.Client) {
```

```
        fmt.Println("connection established")
    }
    opts.OnReconnecting = func(mqtt.Client, *mqtt.ClientOptions) {
        fmt.Println("attempting to reconnect")
    }

    subs := []struct {
        topic   string
        handler mqtt.MessageHandler
    }{
        {
            topic: "/storage/metrics",
            handler: func(c mqtt.Client, m mqtt.Message) {
                fmt.Println("Got storage metrics:")
                fmt.Println(string(m.Payload()))
            },
        },
        {
            topic: "/conveyor/metrics",
            handler: func(c mqtt.Client, m mqtt.Message) {
                fmt.Println("Got conveyor metrics:")
                fmt.Println(string(m.Payload()))
            },
        },
        {
            topic: "/recognition/metrics",
            handler: func(c mqtt.Client, m mqtt.Message) {
                fmt.Println("Got recognition metrics")
                fmt.Println(string(m.Payload()))
            },
        },
        {
            topic: "/lathe/metrics",
            handler: func(c mqtt.Client, m mqtt.Message) {
                fmt.Println("Got lathe metrics")
                fmt.Println(string(m.Payload()))
```

```
            },
        },
        {
            topic: "/miller/metrics",
            handler: func(c mqtt.Client, m mqtt.Message) {
                fmt.Println("Got miller metrics")
                fmt.Println(string(m.Payload()))
            },
        },
        {
            topic: "/storage/cmd/response/provide",
            handler: func(c mqtt.Client, m mqtt.Message) {
                fmt.Println("Got response from storage on
workpiece provision:")
                fmt.Println(string(m.Payload()))
            },
        },
        {
            topic: "/conveyor/cmd/response/recognition",
            handler: func(c mqtt.Client, m mqtt.Message) {
                fmt.Println("Got response from conveyor on moving
workpiece to the recognition:")
                fmt.Println(string(m.Payload()))
            },
        },
        {
            topic: "/recognition/cmd/response/recognize",
            handler: func(c mqtt.Client, m mqtt.Message) {
                fmt.Println("Got recognition results from
recognition module:")
                fmt.Println(string(m.Payload()))
            },
        },
        {
            topic: "/conveyor/cmd/response/lathe",
            handler: func(c mqtt.Client, m mqtt.Message) {
```

```
                fmt.Println("Got response from conveyor on moving
workpiece to the lathe:")
                fmt.Println(string(m.Payload()))
            },
        },
        {
            topic: "/lathe/cmd/response/handle",
            handler: func(c mqtt.Client, m mqtt.Message) {
                fmt.Println("Got response from lathe on handling
the workpiece:")
                fmt.Println(string(m.Payload()))
            },
        },
        {
            topic: "/conveyor/cmd/response/miller",
            handler: func(c mqtt.Client, m mqtt.Message) {
                fmt.Println("Got response from conveyor on moving
workpiece to the miller:")
                fmt.Println(string(m.Payload()))
            },
        },
        {
            topic: "/miller/cmd/response/handle",
            handler: func(c mqtt.Client, m mqtt.Message) {
                fmt.Println("Got response from miller on handling
the workpiece:")
                fmt.Println(string(m.Payload()))
            },
        },
        {
            topic: "/storage/cmd/response/accept",
            handler: func(c mqtt.Client, m mqtt.Message) {
                fmt.Println("Got response from storage on
workpiece acception:")
                fmt.Println(string(m.Payload()))
            },
```

```
        },
    }

    pubs := []string{
        "/storage/cmd/request/provide",
        "/conveyor/cmd/request/lathe",
        "/lathe/cmd/request/handle",
        "/conveyor/cmd/request/miller",
        "/miller/cmd/request/handle",
        "/conveyor/cmd/request/recognition",
        "/recognition/cmd/request/recognize",
        "/storage/cmd/request/accept",

        "/storage/cmd/request/provide",
        "/conveyor/cmd/request/lathe",
        "/lathe/cmd/request/handle",
        "/conveyor/cmd/request/miller",
        "/miller/cmd/request/handle",
        "/conveyor/cmd/request/recognition",
        "/recognition/cmd/request/recognize",
        "/storage/cmd/request/accept",
    }

    client := mqtt.NewClient(opts)

    if token := client.Connect(); token.Wait() && token.Error() !=
nil {
        panic(token.Error())
    }
    fmt.Println("Connection is up")

    for _, subCase := range subs {
        if token := client.Subscribe(subCase.topic, 1,
subCase.handler); token.Wait() && token.Error() != nil {
            log.Fatal(token.Error())
        }
```

```
    }

    for _, topic := range pubs {
        time.Sleep(6 * time.Second)
        if token := client.Publish(topic, 1, false, "");
token.Wait() && token.Error() != nil {
            log.Fatal(token.Error())
        }
    }

    sig := make(chan os.Signal, 1)
    signal.Notify(sig, os.Interrupt)
    signal.Notify(sig, syscall.SIGTERM)

    <-sig
    fmt.Println("signal caught - exiting")
    fmt.Println("shutdown complete")
}
```

ПРИЛОЖЕНИЕ Ю

Логи на стороне приложения в результате работы тестового сценария (часть логов опущена, жирным шрифтом выделены логи об обработке команд и публикации ответа)


```
{"level":"info","timestamp":1706018453.7926035,"caller":"mqtt/metr
ics.go:22","msg":"Service miller beginning to push it's metrics"}
{"level":"info","timestamp":1706018453.7925704,"caller":"mqtt/metr
ics.go:22","msg":"Service conveyor beginning to push it's
metrics"}
{"level":"info","timestamp":1706018453.79266,"caller":"mqtt/metric
s.go:22","msg":"Service storage beginning to push it's metrics"}
{"level":"info","timestamp":1706018453.7927094,"caller":"mqtt/metr
ics.go:22","msg":"Service recognition beginning to push it's
metrics"}
{"level":"info","timestamp":1706018453.7927427,"caller":"mqtt/metr
ics.go:22","msg":"Service lathe beginning to push it's metrics"}

{"level":"info","timestamp":1706018457.7975922,"caller":"mqtt/clie
nt.go:29","msg":"Published message:","ID":"8002a2d0-8320-48f3-
9159-
10187c434bb7","topic":"/lathe/metrics","QOS":1,"retained":true}
...
...
{"level":"info","timestamp":1706018457.7977746,"caller":"mqtt/clie
nt.go:29","msg":"Published message:","ID":"1d312a38-6689-46aa-
bce1-
f9f717ba255b","topic":"/recognition/metrics","QOS":1,"retained":tr
ue}

{"level":"info","timestamp":1706018462.383261,"caller":"mqtt/clien
t.go:29","msg":"Published message:","ID":"9c8b1e67-20d5-4e80-8115-
583271e4a040","topic":"/storage/cmd/response/provide","QOS":1,"ret
ained":false}
{"level":"info","timestamp":1706018462.3833787,"caller":"mqtt/midd
leware.go:80","msg":"Got
```

message","topic":"/storage/cmd/request/provide","QOS":1,"retained":false,"MessageID":2,"processing_time":0.000209343}
...
...
{"level":"info","timestamp":1706018465.8008351,"caller":"mqtt/client.go:29","msg":"Published message:","ID":"fe93be92-7b73-42dd-afb4-f4fbd0c62d73","topic":"/miller/metrics","QOS":1,"retained":true}
...
...
{"level":"info","timestamp":1706018465.8009717,"caller":"mqtt/client.go:29","msg":"Published message:","ID":"7599b5a8-b115-49e4-8c3a-4703fb2ba52a","topic":"/conveyor/metrics","QOS":1,"retained":true}
...
...
{"level":"info","timestamp":1706018468.4592235,"caller":"mqtt/client.go:29","msg":"Published message:","ID":"c821849d-d573-4b87-880c-998990eb3ec1","topic":"/conveyor/cmd/response/lathe","QOS":1,"retained":false}
{"level":"info","timestamp":1706018468.4593961,"caller":"mqtt/middleware.go:80","msg":"Got message","topic":"/conveyor/cmd/request/lathe","QOS":1,"retained":false,"MessageID":2,"processing_time":0.000384602}
...
...
{"level":"info","timestamp":1706018474.5619912,"caller":"mqtt/client.go:29","msg":"Published message:","ID":"b44d3644-cb4d-46aa-ab7a-442c4a6967b5","topic":"/lathe/cmd/response/handle","QOS":1,"retained":false}
{"level":"info","timestamp":1706018474.5621405,"caller":"mqtt/middleware.go:80","msg":"Got message","topic":"/lathe/cmd/request/handle","QOS":1,"retained":false,"MessageID":2,"processing_time":0.000221245}

...
...
{"level":"info","timestamp":1706018480.7607992,"caller":"mqtt/client.go:29","msg":"Published message:","ID":"48280cfd-c13c-402f-9af4-024c9c4e02b4","topic":"/conveyor/cmd/response/miller","QOS":1,"retained":false}
{"level":"info","timestamp":1706018480.7609158,"caller":"mqtt/middleware.go:80","msg":"Got message","topic":"/conveyor/cmd/request/miller","QOS":1,"retained":false,"MessageID":3,"processing_time":0.000187332}
...
...
{"level":"info","timestamp":1706018486.9661276,"caller":"mqtt/client.go:29","msg":"Published message:","ID":"3b0c129d-f993-4095-88e5-baac9b98ad8a","topic":"/miller/cmd/response/handle","QOS":1,"retained":false}
{"level":"info","timestamp":1706018486.9662745,"caller":"mqtt/middleware.go:80","msg":"Got message","topic":"/miller/cmd/request/handle","QOS":1,"retained":false,"MessageID":2,"processing_time":0.000224693}
...
...
{"level":"info","timestamp":1706018493.1589499,"caller":"mqtt/client.go:29","msg":"Published message:","ID":"e27050f9-6552-4e7f-9cdc-c2f22664311b","topic":"/conveyor/cmd/response/recognition","QOS":1,"retained":false}
{"level":"info","timestamp":1706018493.1590827,"caller":"mqtt/middleware.go:80","msg":"Got message","topic":"/conveyor/cmd/request/recognition","QOS":1,"retained":false,"MessageID":4,"processing_time":0.000181431}
...
...

{"level":"info","timestamp":1706018499.3522718,"caller":"mqtt/client.go:29","msg":"Published message:","ID":"f556207b-925a-448d-b70b-c126bac0ea77","topic":"/recognition/cmd/response/recognize","QOS":1,"retained":false}
{"level":"info","timestamp":1706018499.3523707,"caller":"mqtt/middleware.go:80","msg":"Got message","topic":"/recognition/cmd/request/recognize","QOS":1,"retained":false,"MessageID":2,"processing_time":0.000196169}
...
...
{"level":"info","timestamp":1706018505.5656695,"caller":"mqtt/client.go:29","msg":"Published message:","ID":"55c96425-c492-4638-af1a-429b1259a65c","topic":"/storage/cmd/response/accept","QOS":1,"retained":false}
{"level":"info","timestamp":1706018505.5657969,"caller":"mqtt/middleware.go:80","msg":"Got message","topic":"/storage/cmd/request/accept","QOS":1,"retained":false,"MessageID":3,"processing_time":0.000169509}
...
...
{"level":"info","timestamp":1706018558.747005,"caller":"mqtt/metrics.go:28","msg":"Service lathe finishing pushing metrics"}
{"level":"info","timestamp":1706018558.747018,"caller":"mqtt/metrics.go:28","msg":"Service storage finishing pushing metrics"}
{"level":"info","timestamp":1706018558.7470841,"caller":"mqtt/metrics.go:28","msg":"Service conveyor finishing pushing metrics"}
{"level":"info","timestamp":1706018558.7470467,"caller":"mqtt/metrics.go:28","msg":"Service miller finishing pushing metrics"}
{"level":"info","timestamp":1706018558.7470767,"caller":"mqtt/metrics.go:28","msg":"Service recognition finishing pushing metrics"}
{"level":"info","timestamp":1706018558.7473092,"caller":"app/run.go:145","msg":"Shutting down gracefully"}

ПРИЛОЖЕНИЕ Я

Логи на стороне тестового клиента в результате работы тестового сценария (часть логов опущена, жирным шрифтом выделены ответы на команды клиента):

```
connection established
Connection is up

Got storage metrics:
{"responseID":"c0350c3f-8a4c-44b2-9874-
7a520a5c7def","request_topic":"","status":"ok","message":"got
storage metrics
sucessfully","payload":{"Ready":true,"Workpieces":1,"RawWorkpieces
":1,"HandledWorkpieces":0}}
Got conveyor metrics:
{"responseID":"d5c06c0b-e4b5-44fd-9f61-
7cc79b5a8529","request_topic":"","status":"ok","message":"got
conveyor metrics
sucessfully","payload":{"Ready":true,"WorkpieceLocation":{"X":0,"Y
":0}}}
Got recognition metrics
{"responseID":"95788199-314f-488f-ac65-
014d7519fd3b","request_topic":"","status":"ok","message":"got
recognition metrics sucessfully","payload":{"Ready":true}}
Got miller metrics
{"responseID":"762ae56d-ff14-45d1-8e0b-
ea7bcb344f0c","request_topic":"","status":"ok","message":"got
miller metrics sucessfully","payload":{"Ready":true}}
Got lathe metrics
{"responseID":"a9241455-5290-45bd-9847-
30f92fdd0127","request_topic":"","status":"ok","message":"got
lathe metrics sucessfully","payload":{"Ready":true}}
```

Got response from storage on workpiece provision:
{"responseID":"fa3e506d-941e-4d9d-8cc6-
c8b67ad73de5","request_topic":"/storage/cmd/request/provide","stat

us":"ok","message":"storage has provided workpiece successfully","payload":null}

Got miller metrics
{"responseID":"ba74da11-944a-4cd3-aae4-3c7531a58cb6","request_topic":"","status":"ok","message":"got miller metrics sucessfully","payload":{"Ready":true}}
Got storage metrics:
{"responseID":"53af56e0-cde0-4031-a133-a1fcd7e593f9","request_topic":"","status":"ok","message":"got storage metrics sucessfully","payload":{"Ready":true,"Workpieces":1,"RawWorkpieces":0,"HandledWorkpieces":0}}
Got recognition metrics
{"responseID":"a15682c0-f15b-4dbd-860f-59fd4f921bb7","request_topic":"","status":"ok","message":"got recognition metrics sucessfully","payload":{"Ready":true}}
Got conveyor metrics:
{"responseID":"ba5c02a7-435b-4092-a5d9-854a0c7eed20","request_topic":"","status":"ok","message":"got conveyor metrics sucessfully","payload":{"Ready":true,"WorkpieceLocation":{"X":1,"Y":0}}}
Got lathe metrics
{"responseID":"ffc07176-6108-4d5c-ab90-ad35bad27d39","request_topic":"","status":"ok","message":"got lathe metrics sucessfully","payload":{"Ready":true}}

Got response from conveyor on moving workpiece to the lathe: {"responseID":"ddf4e6a0-d69a-4a6b-b204-29acdfd6484a","request_topic":"/conveyor/cmd/request/lathe","status":"ok","message":"conveyor provided workpiece to the lathe successfully","payload":null}

Got storage metrics:

{"responseID":"18d0d336-c250-4ee1-a6d4-c83f10860926","request_topic":"","status":"ok","message":"got storage metrics sucessfully","payload":{"Ready":true,"Workpieces":1,"RawWorkpieces":0,"HandledWorkpieces":0}}
Got conveyor metrics:
{"responseID":"a63cdd5d-1488-441e-b240-b4b77fdd50e6","request_topic":"","status":"ok","message":"got conveyor metrics sucessfully","payload":{"Ready":true,"WorkpieceLocation":{"X":50,"Y":50}}}
Got recognition metrics
{"responseID":"cdcda72d-3eac-48ac-8459-15f57e11a0f2","request_topic":"","status":"ok","message":"got recognition metrics sucessfully","payload":{"Ready":true}}
Got miller metrics
{"responseID":"a80adaa7-4a0a-43fd-900c-5bb919501d72","request_topic":"","status":"ok","message":"got miller metrics sucessfully","payload":{"Ready":true}}
Got lathe metrics
{"responseID":"11ea3477-7372-4f8b-a5f2-beaf3b663614","request_topic":"","status":"ok","message":"got lathe metrics sucessfully","payload":{"Ready":true}}

Got response from lathe on handling the workpiece:
{"responseID":"83c87da5-22e1-4d13-92d4-1338eba05254","request_topic":"/lathe/cmd/request/handle","status":"ok","message":"lathe handled workpiece sucessfully","payload":null}

Got lathe metrics
{"responseID":"fd5cb87b-a4dc-4140-824e-f0bdbefb4910","request_topic":"","status":"ok","message":"got lathe metrics sucessfully","payload":{"Ready":true}}
Got miller metrics

{"responseID":"ec928ede-0017-4c45-a893-6c3225782435","request_topic":"","status":"ok","message":"got miller metrics sucessfully","payload":{"Ready":true}}
Got conveyor metrics:
{"responseID":"3f9548e9-67e5-4ae3-9f84-7824be6a7f44","request_topic":"","status":"ok","message":"got conveyor metrics sucessfully","payload":{"Ready":true,"WorkpieceLocation":{"X":50,"Y":50}}}
Got storage metrics:
{"responseID":"087aa25e-f9c4-452f-bef0-8d306e84935f","request_topic":"","status":"ok","message":"got storage metrics sucessfully","payload":{"Ready":true,"Workpieces":1,"RawWorkpieces":0,"HandledWorkpieces":0}}
Got recognition metrics
{"responseID":"34671ece-2214-404b-ae58-1416b183f9b3","request_topic":"","status":"ok","message":"got recognition metrics sucessfully","payload":{"Ready":true}}

Got response from conveyor on moving workpiece to the miller:
{"responseID":"62bc1324-5fd5-4c57-86f6-148f5310c3a9","request_topic":"/conveyor/cmd/request/miller","status":"ok","message":"conveyor provided workpiece to the miller successfully","payload":null}

Got conveyor metrics:
{"responseID":"31bd9754-a231-4bb6-83ea-c3e62704f1fe","request_topic":"","status":"ok","message":"got conveyor metrics sucessfully","payload":{"Ready":true,"WorkpieceLocation":{"X":0,"Y":50}}}
Got storage metrics:
{"responseID":"234869d2-076b-406d-8292-92282fb164c6","request_topic":"","status":"ok","message":"got storage metrics

sucessfully","payload":{"Ready":true,"Workpieces":1,"RawWorkpieces":0,"HandledWorkpieces":0}}
Got recognition metrics
{"responseID":"597d7573-db37-48e7-8141-1021523fa350","request_topic":"","status":"ok","message":"got recognition metrics sucessfully","payload":{"Ready":true}}
Got miller metrics
{"responseID":"ad6af064-d82a-44bb-a761-3afd4c57fb80","request_topic":"","status":"ok","message":"got miller metrics sucessfully","payload":{"Ready":true}}
Got lathe metrics
{"responseID":"ad1f7155-4ad4-4afd-95db-1d0f99ffbe17","request_topic":"","status":"ok","message":"got lathe metrics sucessfully","payload":{"Ready":true}}

Got response from miller on handling the workpiece:
{"responseID":"07dd9c85-9876-4a4e-b4e1-a9933d85da20","request_topic":"/miller/cmd/request/handle","status":"ok","message":"miller handled workpiece successfully","payload":null}

Got recognition metrics
{"responseID":"0b847be6-9f5f-4e7f-a8e2-801c64ebc5f5","request_topic":"","status":"ok","message":"got recognition metrics sucessfully","payload":{"Ready":true}}
Got conveyor metrics:
{"responseID":"91f41c46-ff95-43b2-b494-aa929beb5876","request_topic":"","status":"ok","message":"got conveyor metrics sucessfully","payload":{"Ready":true,"WorkpieceLocation":{"X":0,"Y":50}}}
Got storage metrics:
{"responseID":"1a63def5-bd43-4658-a7e9-58742b506f4c","request_topic":"","status":"ok","message":"got storage metrics

sucessfully","payload":{"Ready":true,"Workpieces":1,"RawWorkpieces":0,"HandledWorkpieces":0}}
Got lathe metrics
{"responseID":"7c14b399-e40d-4999-8619-f611536daffe","request_topic":"","status":"ok","message":"got lathe metrics sucessfully","payload":{"Ready":true}}
Got miller metrics
{"responseID":"4e4ac32e-9af6-41f7-85f7-9cf7d7a78581","request_topic":"","status":"ok","message":"got miller metrics sucessfully","payload":{"Ready":true}}

Got response from conveyor on moving workpiece to the recognition:
{"responseID":"416d30be-6210-4cd9-b6a9-789fd1beeaa0","request_topic":"/conveyor/cmd/request/recognition","status":"ok","message":"conveyor provided workpiece to the recognition successfully","payload":null}

Got recognition metrics
{"responseID":"b0f6f645-f572-46ba-abd0-0f73a5c9226e","request_topic":"","status":"ok","message":"got recognition metrics sucessfully","payload":{"Ready":true}}
Got conveyor metrics:
{"responseID":"14a1c68b-e76a-44c2-94bc-f1b4677f87b3","request_topic":"","status":"ok","message":"got conveyor metrics sucessfully","payload":{"Ready":true,"WorkpieceLocation":{"X":0,"Y":25}}}
Got storage metrics:
{"responseID":"84c8f0da-c6f1-4f20-aa9d-772fc3ca6bdf","request_topic":"","status":"ok","message":"got storage metrics sucessfully","payload":{"Ready":true,"Workpieces":1,"RawWorkpieces":0,"HandledWorkpieces":0}}
Got lathe metrics

{"responseID":"0d036a30-2aaa-48cc-8c9f-1f6454b2f86b","request_topic":"","status":"ok","message":"got lathe metrics sucessfully","payload":{"Ready":true}}
Got miller metrics
{"responseID":"d467b139-d100-4a3b-9586-1b4a9e8d40b4","request_topic":"","status":"ok","message":"got miller metrics sucessfully","payload":{"Ready":true}}

Got recognition results from recognition module:
{"responseID":"4a72d75d-80fc-4503-b11a-67fcbeee6068","request_topic":"/recognition/cmd/request/recognize","status":"ok","message":"recognized workpiece: 2","payload":null}

Got storage metrics:
{"responseID":"fb107ac0-0778-4418-8b34-5e9a8227d782","request_topic":"","status":"ok","message":"got storage metrics sucessfully","payload":{"Ready":true,"Workpieces":1,"RawWorkpieces":0,"HandledWorkpieces":0}}
Got conveyor metrics:
{"responseID":"e80db90d-b8a1-4ac7-8759-0367cc2f1eb2","request_topic":"","status":"ok","message":"got conveyor metrics sucessfully","payload":{"Ready":true,"WorkpieceLocation":{"X":0,"Y":25}}}
Got recognition metrics
{"responseID":"a2f7a3e1-21d2-48f8-89cb-1bda5250744e","request_topic":"","status":"ok","message":"got recognition metrics sucessfully","payload":{"Ready":true}}
Got miller metrics
{"responseID":"a67bb8ae-0add-49a0-a88e-80aaec17e117","request_topic":"","status":"ok","message":"got miller metrics sucessfully","payload":{"Ready":true}}
Got lathe metrics

{"responseID":"0949dd4b-ffff-4c8f-94b5-e0f41036bbab","request_topic":"","status":"ok","message":"got lathe metrics sucessfully","payload":{"Ready":true}}

Got response from storage on workpiece acception:
{"responseID":"91ede648-1047-4c4c-8e10-198ade9ef35c","request_topic":"/storage/cmd/request/accept","status":"ok","message":"storage has accepted workpiece successfully","payload":null}

Got storage metrics:
{"responseID":"75320649-a733-4724-af97-aa057bc87a11","request_topic":"","status":"ok","message":"got storage metrics sucessfully","payload":{"Ready":true,"Workpieces":1,"RawWorkpieces":0,"HandledWorkpieces":1}}
Got conveyor metrics:
{"responseID":"3c877a19-24ae-452b-8c0f-3e7012742fdb","request_topic":"","status":"ok","message":"got conveyor metrics sucessfully","payload":{"Ready":true,"WorkpieceLocation":{"X":0,"Y":0}}}
Got recognition metrics
{"responseID":"13c2e101-efee-4207-b9b3-8a28918fc38b","request_topic":"","status":"ok","message":"got recognition metrics sucessfully","payload":{"Ready":true}}
Got miller metrics
{"responseID":"0f6ada75-b144-42b8-90cc-2032caeb9ddb","request_topic":"","status":"ok","message":"got miller metrics sucessfully","payload":{"Ready":true}}
Got lathe metrics
{"responseID":"f5eae1b9-166a-4bb4-9b7f-8d80ce3b202d","request_topic":"","status":"ok","message":"got lathe metrics sucessfully","payload":{"Ready":true}}